

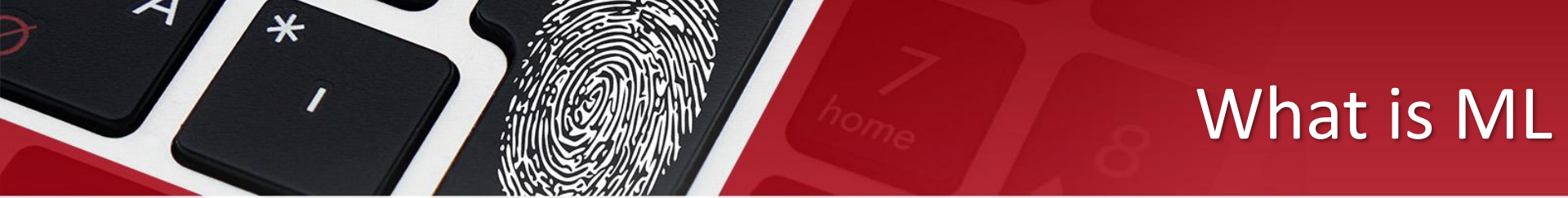


Machine Learning and its Algorithms

Data Science, Machine Learning Course

Fahad Hussain
MSCS (SMIU)
DAE (IT)
MCS(KU)

Instructor of Well-known INTERNATIONAL COMPUTER CENTER



What is ML

Machine learning is an application of artificial **intelligence** (AI) that provides systems the ability to automatically learn and improve from experience without being explicitly programmed. **Machine learning** focuses on the development of computer programs that can access data and use it learn for themselves.

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>

Types of Machine Learning – At a Glance

Supervised Learning

- Makes machine Learn explicitly
- Data with clearly defined output is given
- Direct feedback is given
- Predicts outcome/future
- Resolves classification and regression problems



Unsupervised Learning

- Machine understands the data (Identifies patterns/structures)
- Evaluation is qualitative or indirect
- Does not predict/find anything specific

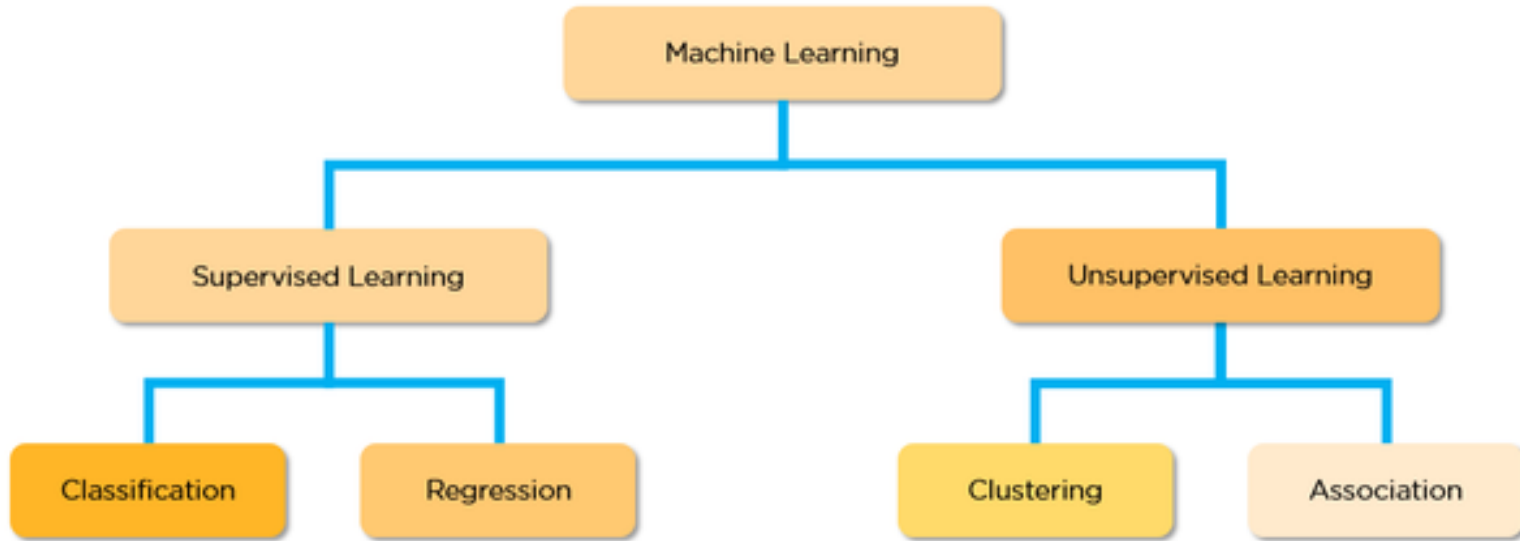


Reinforcement Learning

- An approach to AI
- Reward based learning
- Learning from +ve & -ve reinforcement
- Machine Learns how to act in a certain environment
- To maximize rewards



AI related Subjects



For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



What is Regression

Regression models (both linear and non-linear) are used for predicting a real value, like salary for example. If your independent variable is time, then you are forecasting future values, otherwise your model is predicting present but unknown values.

Types of Machine Learning Regression models:

- Simple Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Support Vector for Regression (SVR)
- Decision Tree Regression
- Random Forest Regression

Data Set

	A	B
1	YearsExperience	Salary
2	1.1	39343
3	1.3	46205
4	1.5	37731
5	2	43525
6	2.2	39891
7	2.9	56642
8	3	60150
9	3.2	54445
10	3.2	64445

Simple Linear Regression

Simple linear regression is a statistical method that allows us to summarize and study relationships between two continuous (quantitative) variables

Constant

Coefficient

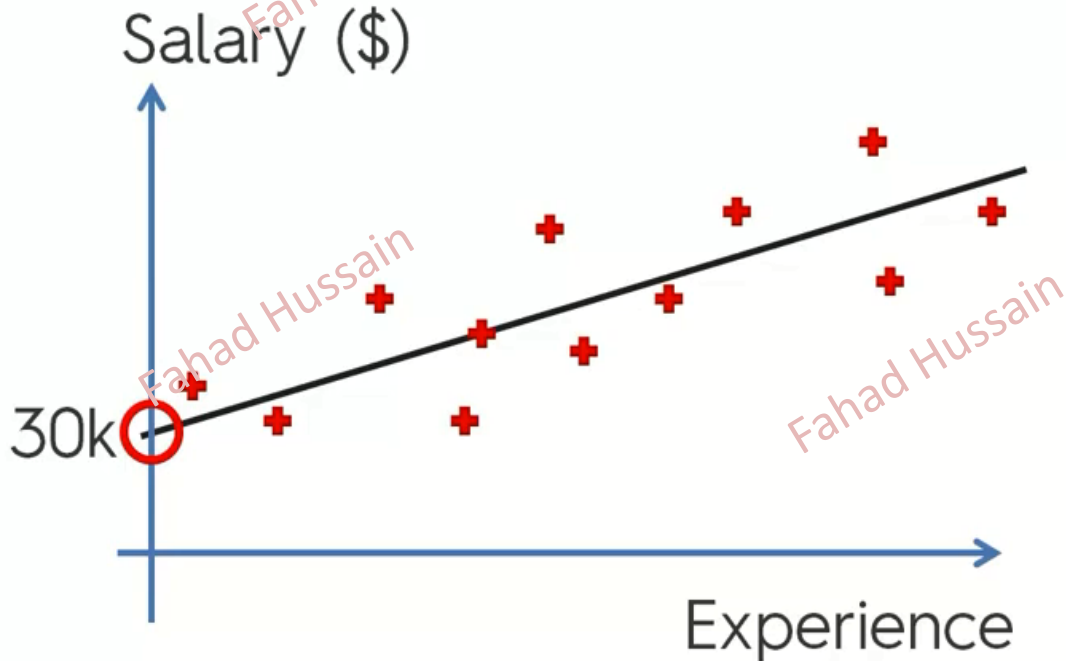
$$y = b_0 + b_1 * x_1$$

Dependent variable (DV)

Independent variable (IV)

What is Data Science

Simple Linear Regression:



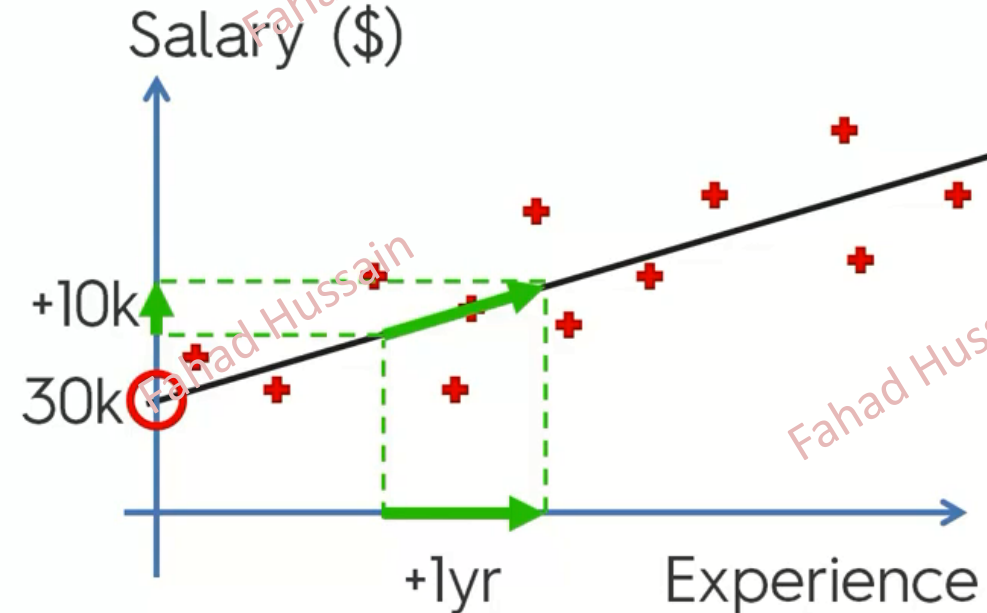
$$y = b_0 + b_1 * x$$



$$\text{Salary} = b_0 + b_1 * \text{Experience}$$

What is Data Science

Simple Linear Regression:



$$y = b_0 + b_1 * x$$

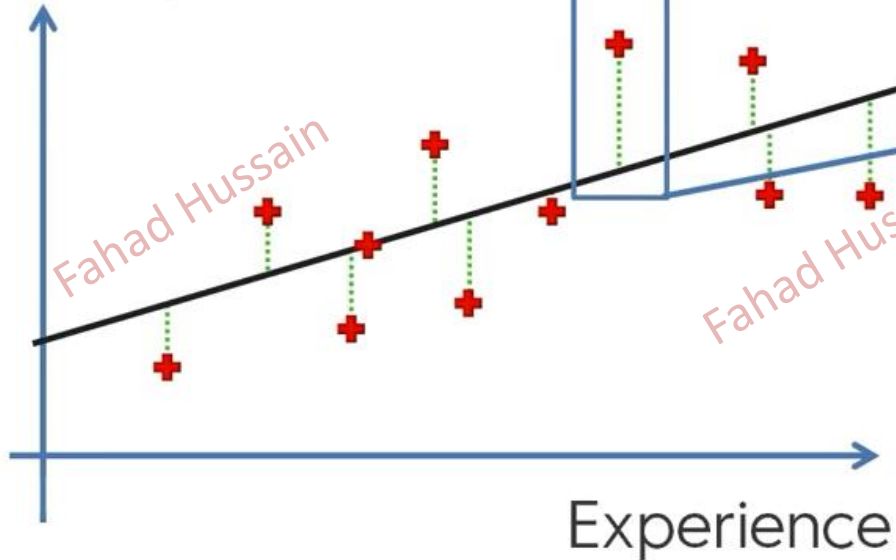


$$\text{Salary} = b_0 + b_1 * \text{Experience}$$

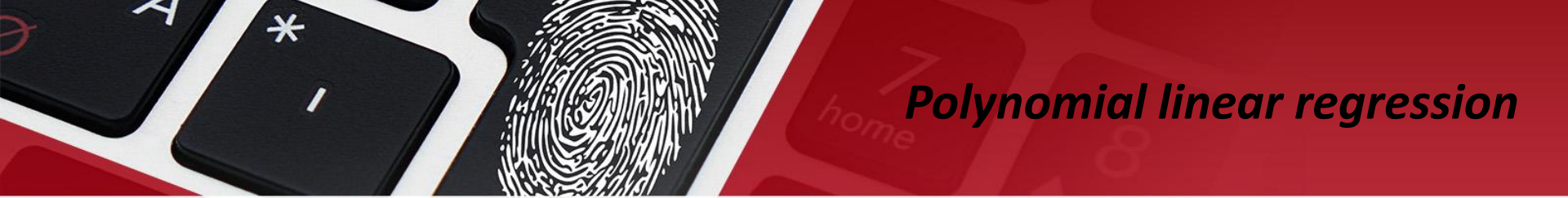
What is Data Science

Simple Linear Regression:

Salary (\$)



$$\text{SUM } (y - \hat{y})^2 \rightarrow \min$$



Polynomial linear regression

Simple
Linear
Regression

$$y = b_0 + b_1 x_1$$

Multiple
Linear
Regression

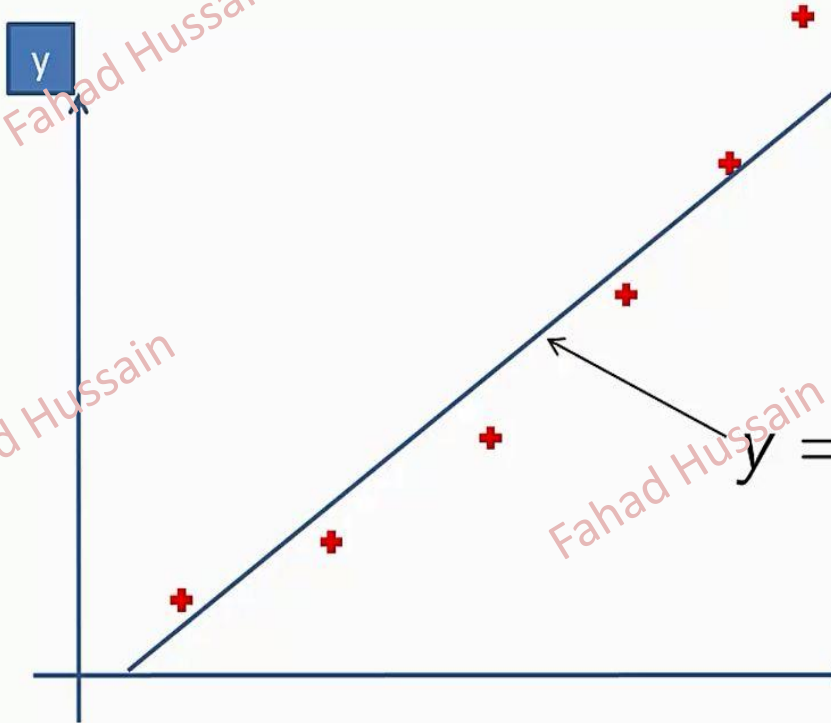
$$y = b_0 + b_1 x_1 + b_2 x_2 + \dots + b_n x_n$$

Polynomial
Linear
Regression

$$y = b_0 + b_1 x_1 + b_2 x_1^2 + \dots + b_n x_1^n$$

Polynomial linear regression

y



$$y = b_0 + b_1 x_1$$

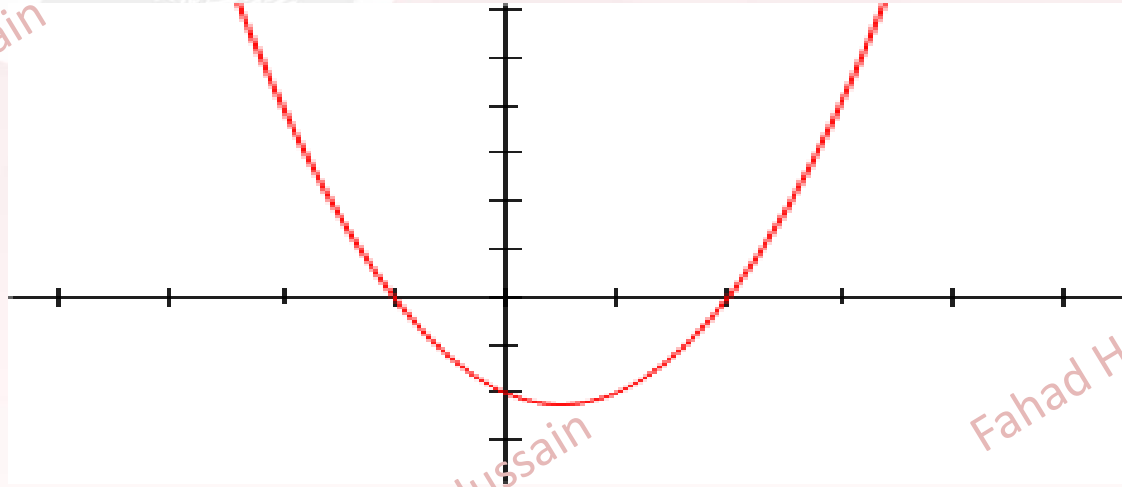
X1

Polynomial linear regression



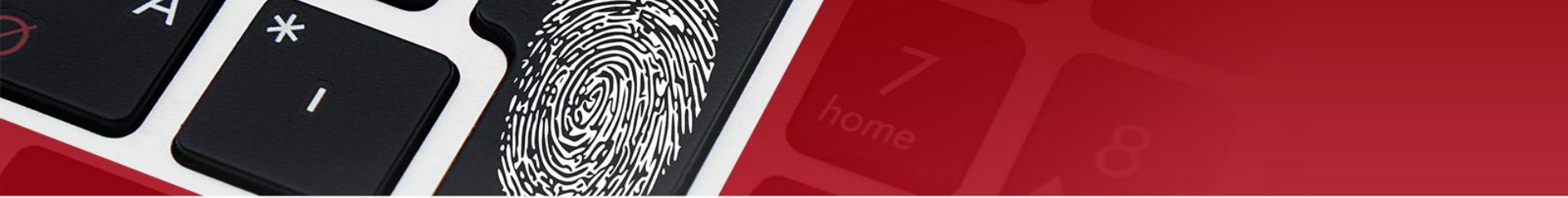
$$y = b_0 + b_1x_1 + b_2x_1^2$$

Non-linear regression



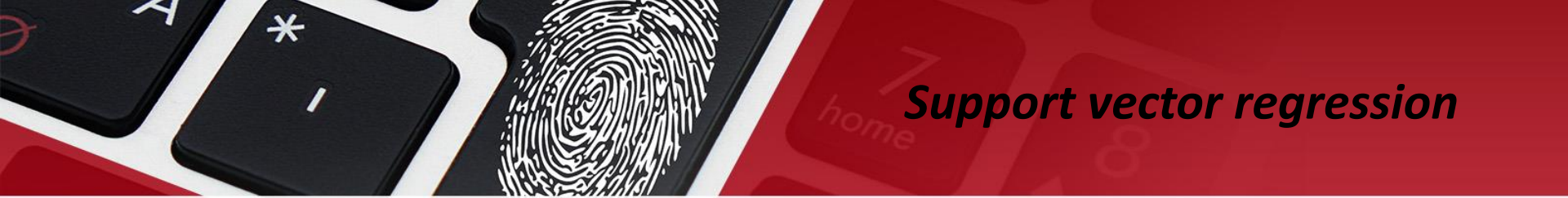
Nonlinear regression involves curves. This is partly true, and if you want a loose definition for the difference, you can probably stop right there. However, *linear equations can sometimes produce curves*.

In order to understand why, you need to take a look at the linear regression equation form.



SVM, which stands for Support Vector Machine, is a classifier. Classifiers perform classification, predicting discrete categorical labels. SVR, which stands for Support Vector Regressor, is a regressor. Regressors perform regression, predicting continuous ordered variables. Both use very similar algorithms, but predict different types of variables

In simple regression we try to minimise the error rate. While in SVR we try to fit the error within a certain threshold.



Support vector regression

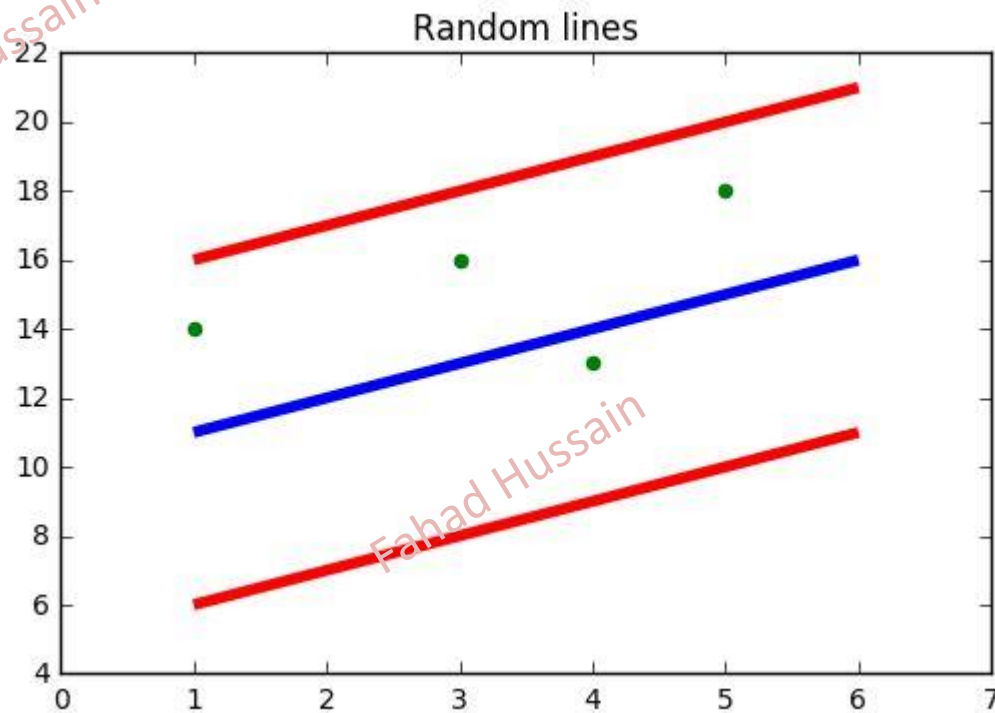
Kernel: The function used to map a lower dimensional data into a higher dimensional data.

Hyper Plane: In SVM this is basically the separation line between the data classes. Although in SVR we are going to define it as the line that will help us predict the continuous value or target value

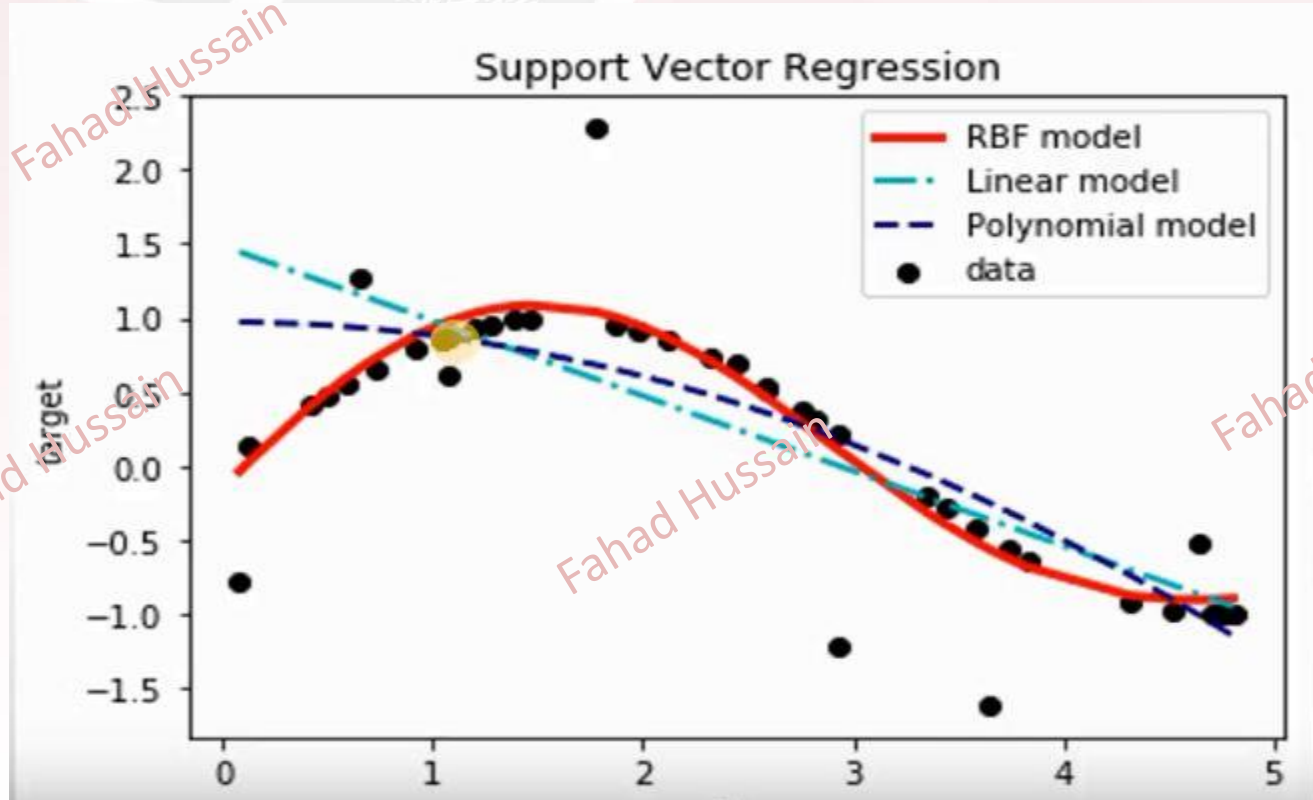
Boundary line: In SVM there are two lines other than Hyper Plane which creates a margin . The support vectors can be on the Boundary lines or outside it. This boundary line separates the two classes. In SVR the concept is same.

Support vectors: These are the data points which are closest to the boundary. The distance of the points is minimum or least.

Support vector regression

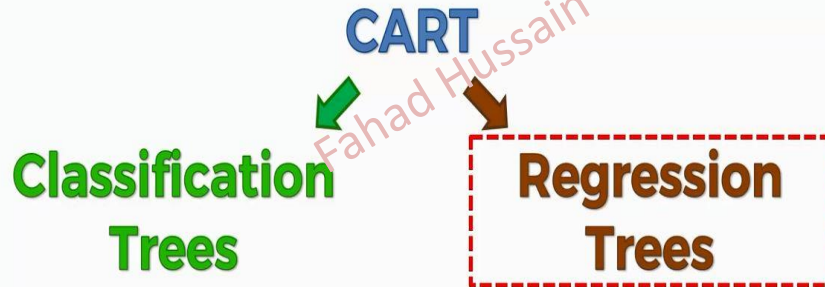


Support vector regression

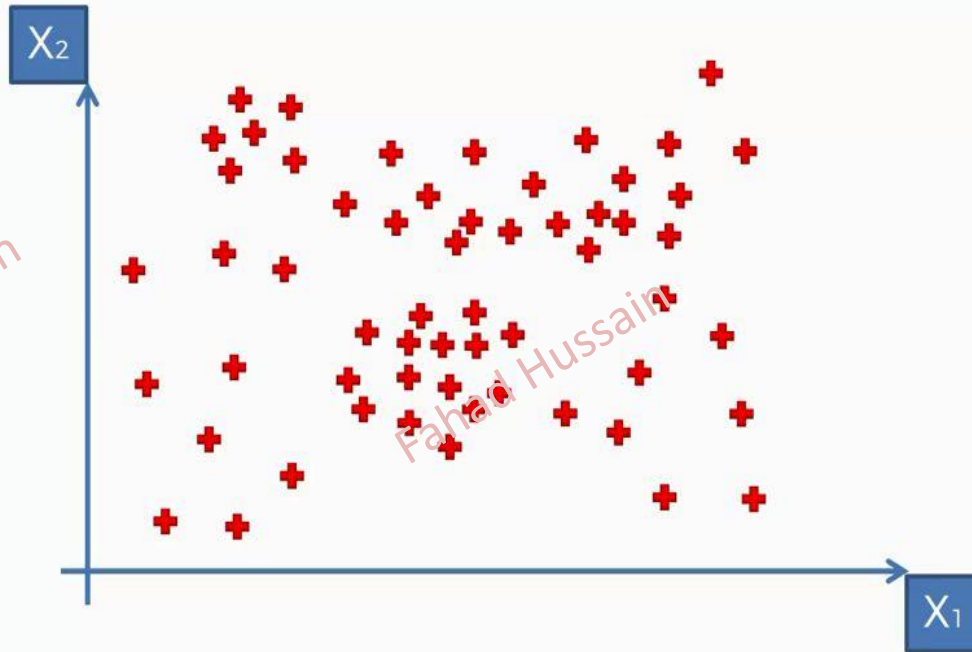


What is Cart

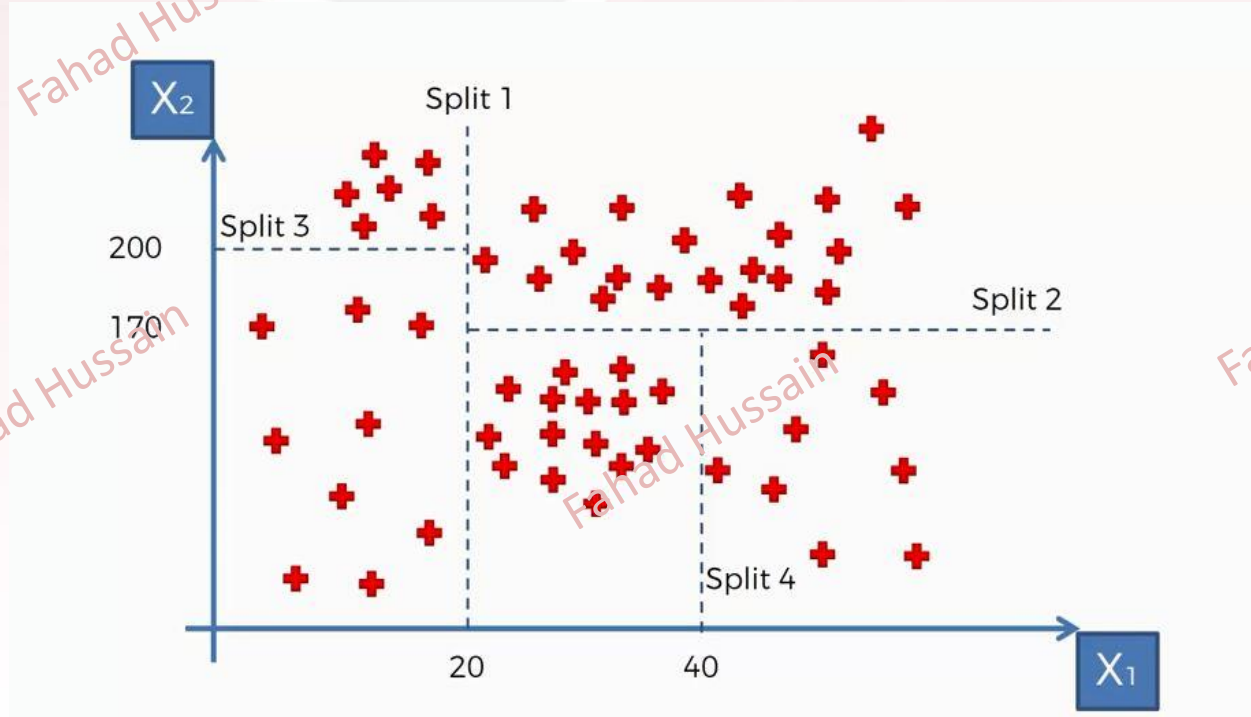
A Classification And Regression Tree (**CART**), is a predictive model, which explains how an outcome variable's values can be predicted based on other values. A **CART** output is a decision tree where each fork is a split in a predictor variable and each end node contains a prediction for the outcome variable



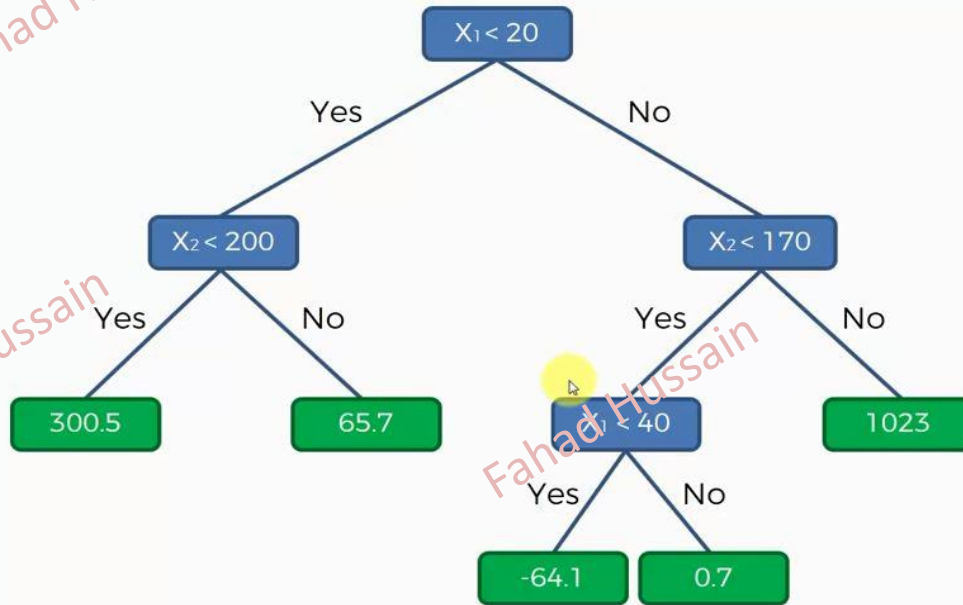
Support vector regression



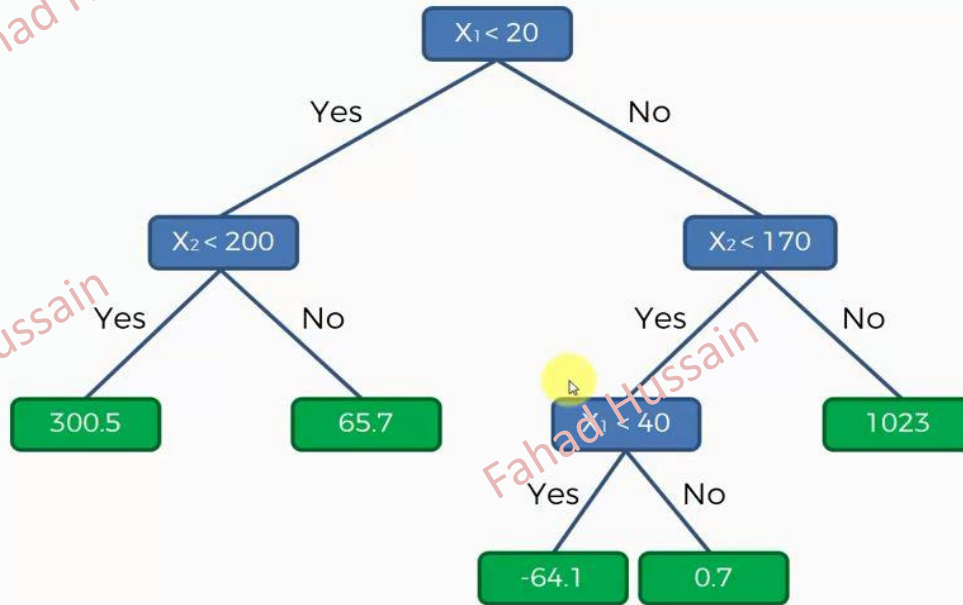
Decision Tree Regression



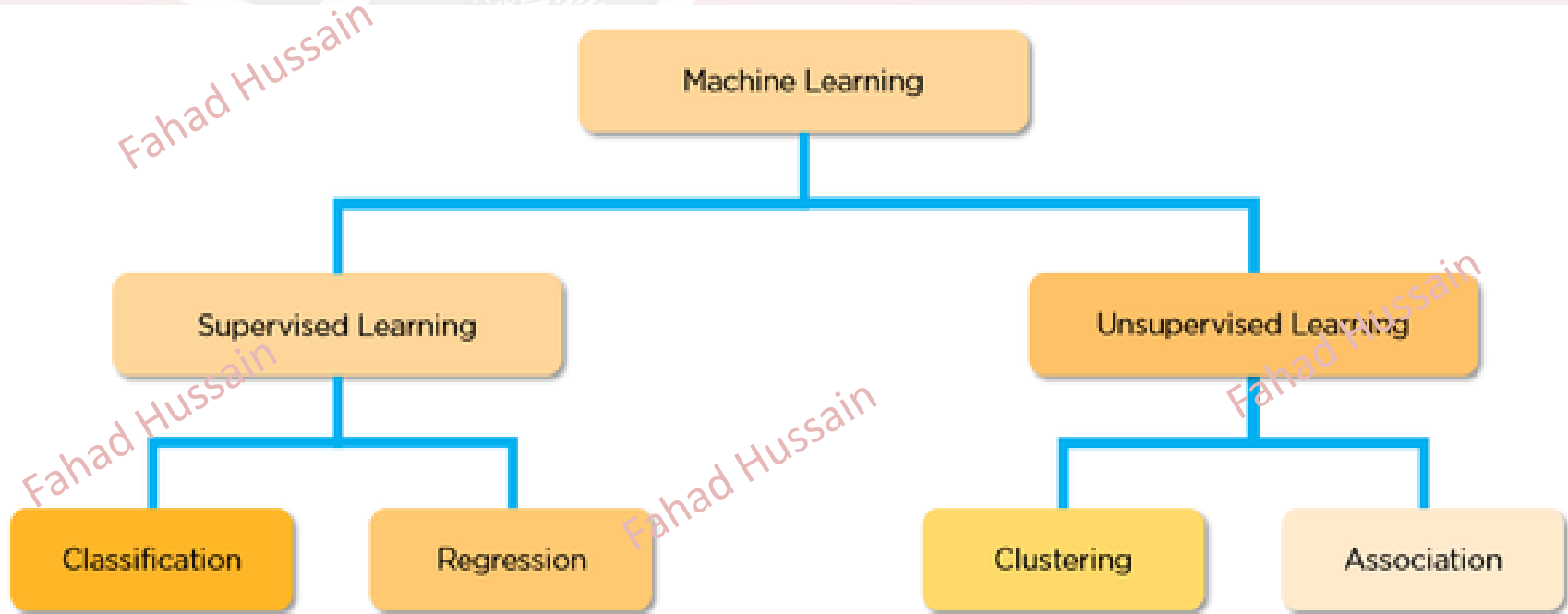
Decision Tree Regression



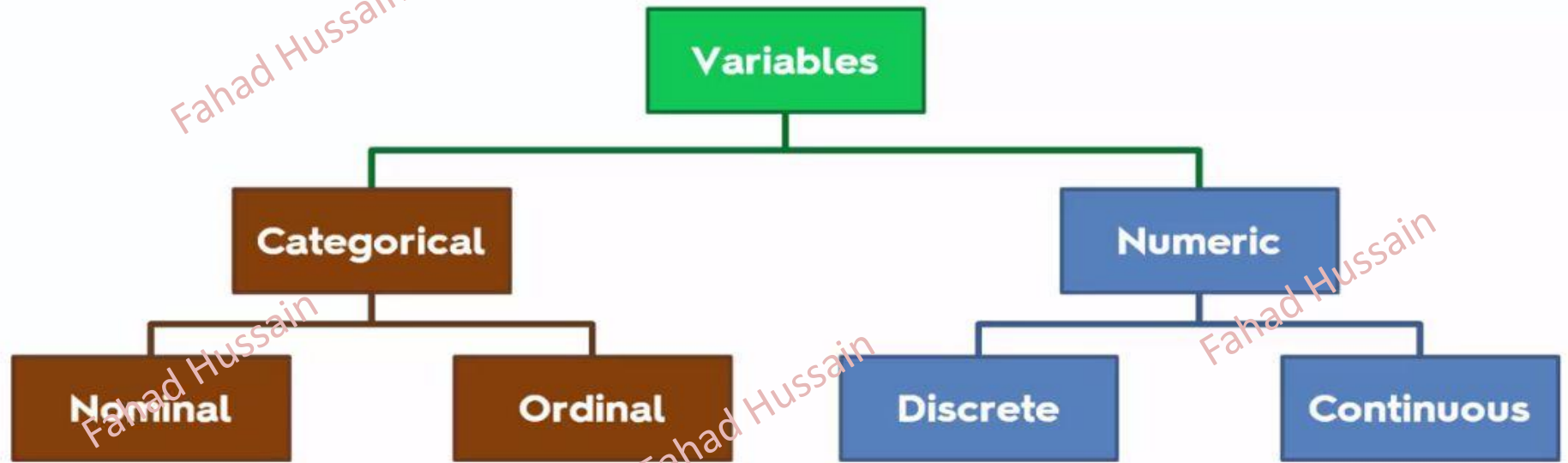
Random Forest Regression



Supervised Learning Of Machine Learning



Variable/Data its types in Machine Learning

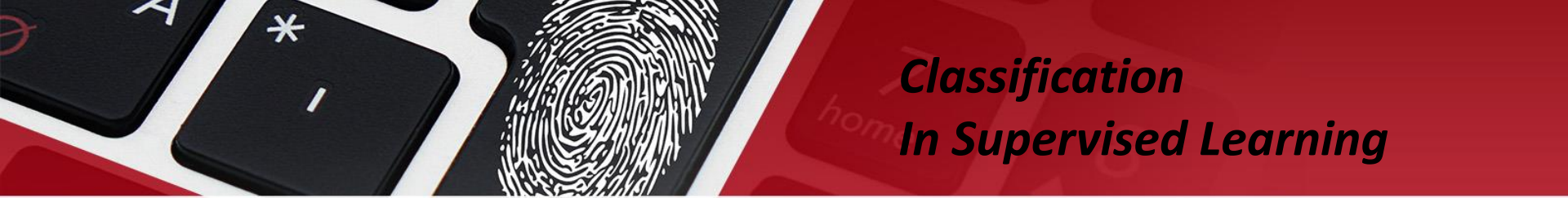


Male, Female
Red, Green, Black

Small, Medium, Large
A, B, C (grades)

1, 2, 3 businesses
568 people

Age
Height

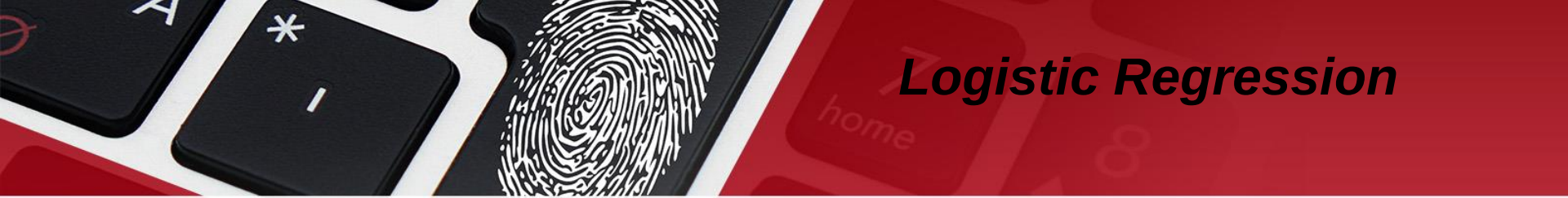


Classification In Supervised Learning

Unlike regression where you predict a continuous number, you use classification to predict a category. There is a wide variety of classification applications from medicine to marketing. Classification models include linear models like Logistic Regression, and nonlinear ones like K-NN, Kernel SVM and Random Forests.

Learning Classification models:

1. Logistic Regression
2. K-Nearest Neighbors (K-NN)
3. Support Vector Machine (SVM)
4. Kernel SVM
5. Naive Bayes
6. Decision Tree Classification
7. Random Forest Classification



Logistic Regression

Logistic regression is a statistical method for analyzing a dataset in which there are one or more independent variables that determine an outcome. The outcome is measured with a dichotomous variable (in which there are only two possible outcomes).

In logistic regression, the dependent variable is binary or dichotomous, i.e. it only contains data coded as 1 (TRUE, success, pregnant, etc.) or 0 (FALSE, failure, non-pregnant, etc.).
Also

Probability is the likelihood or chance of an event occurring. **Probability** = the number of ways of achieving success. the total number of possible outcomes.

Is it classification algo.?
So why is it called regression?



Logistic Regression

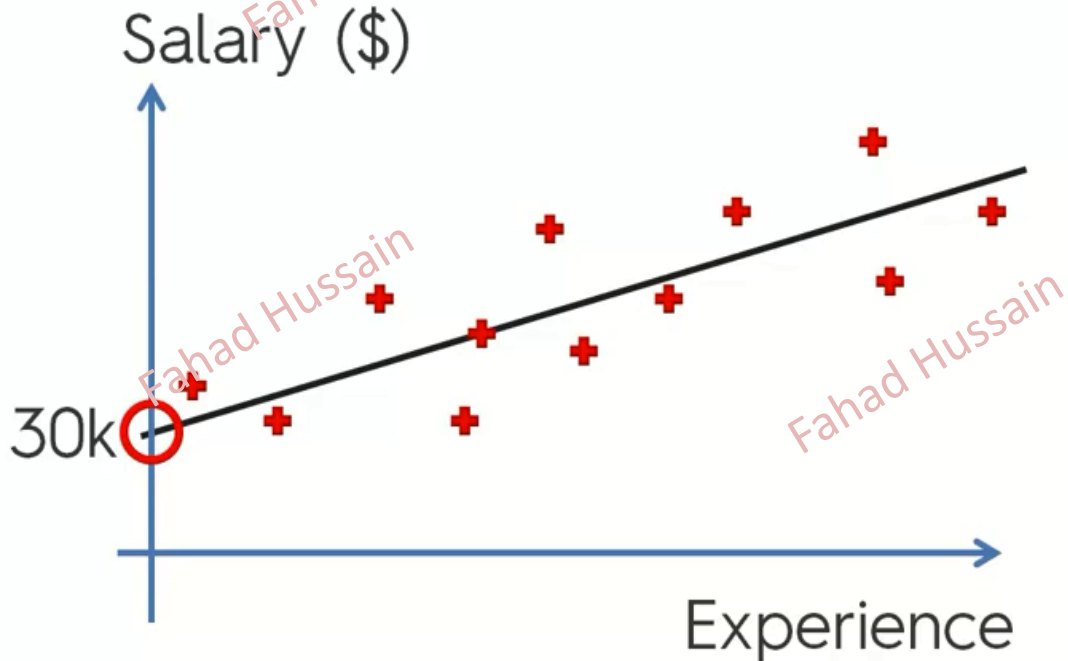
Logistic regression also called
binary regression

And also it can be Multiple...

So it become multiple logistic regression

Logistic Regression

Simple Linear Regression:



$$y = b_0 + b_1 * x$$

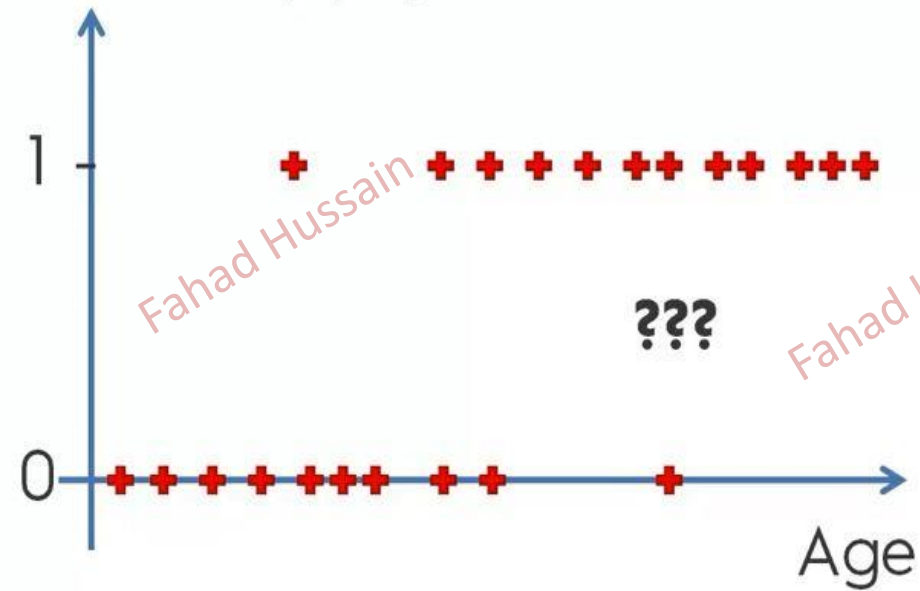


$$\text{Salary} = b_0 + b_1 * \text{Experience}$$

Logistic R vs Linear R

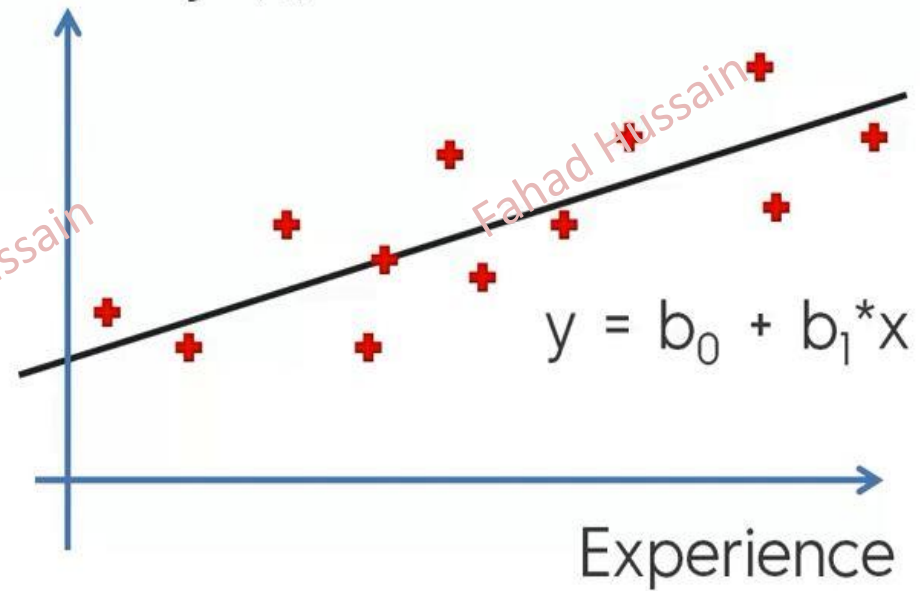
This is new:

Action (Y/N)



We know this:

Salary (\$)



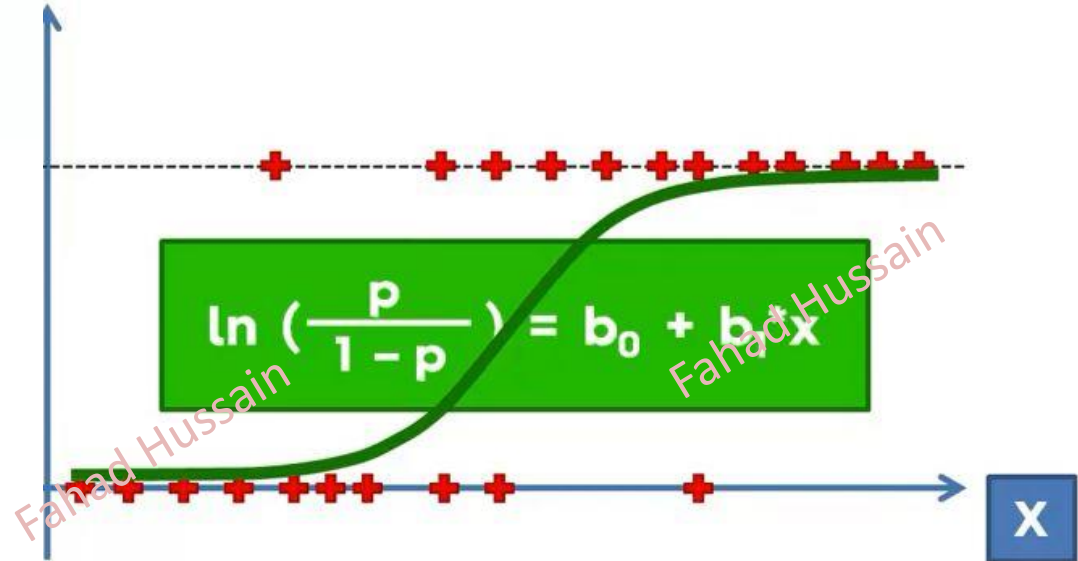
Logistic Regression Formula

$$y = b_0 + b_1 * x$$

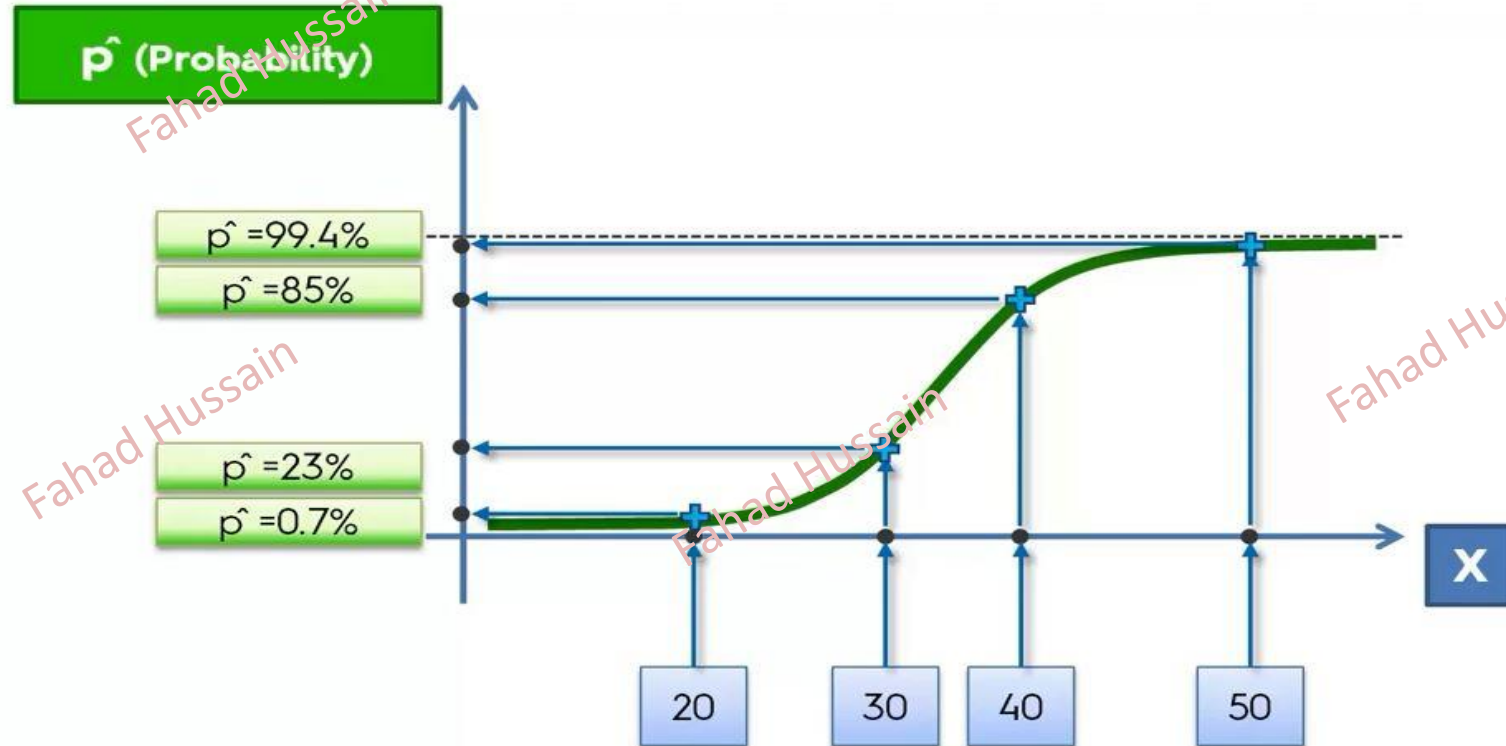
Sigmoid Function

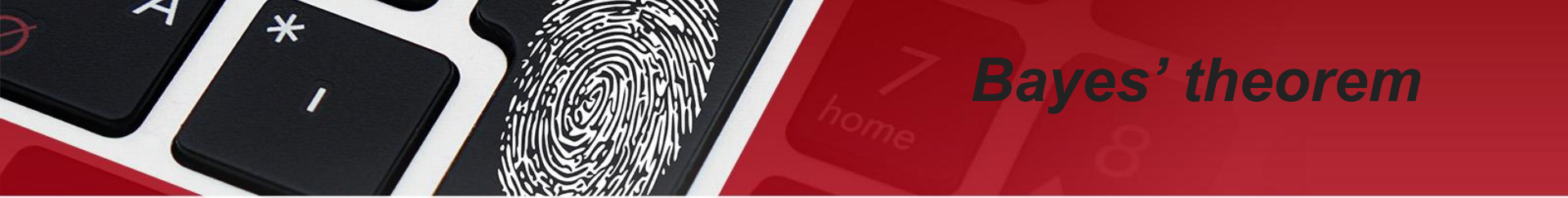
$$p = \frac{1}{1 + e^{-y}}$$

$$\ln \left(\frac{p}{1-p} \right) = b_0 + b_1 * x$$



Logistic Regression working



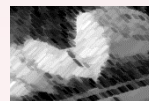


Bayes' theorem

In probability theory and statistics, Bayes' theorem describes the probability of an event, based on prior knowledge of conditions that might be related to the event

$$P(A|B) = \frac{P(A) P(B|A)}{P(B)}$$

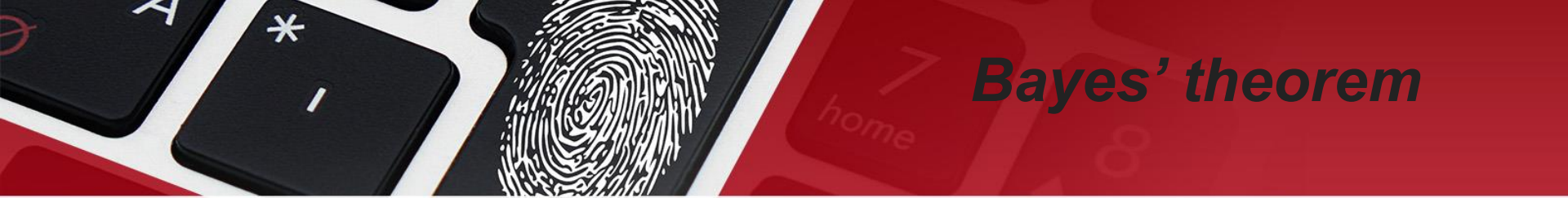
Bayes' theorem



Machine 1
30 Bread per hour



Machine 2
20 Bread per hour



Bayes' theorem

Machine1: 30 Breads / hr

Machine2: 20 Breads /hr

Out of all product parts:

We can SEE that 1% are defective

Out of all defective parts:

We can See that 50% came form Machine1

And 50% came from Machine2

What is probability that a part produced by machine1
Is defective =?

$$\Rightarrow P(\text{Machine1}) = 30/50 = 0.6$$

$$\Rightarrow P(\text{Machine2}) = 20/50 = 0.4$$

$$\Rightarrow P(\text{Defect}) = 1\%$$

$$\Rightarrow P(\text{Machine1} | \text{Defect}) = 50\%$$

$$\Rightarrow P(\text{Machine2} | \text{Defect}) = 50\%$$

$$\Rightarrow P(\text{Defect} | \text{Machine2}) = ?$$

$$P(\text{Defect} | \text{Machine2}) = \frac{P(\text{Machine2} | \text{Defect}) * P(\text{Defect})}{P(\text{Machine2})}$$

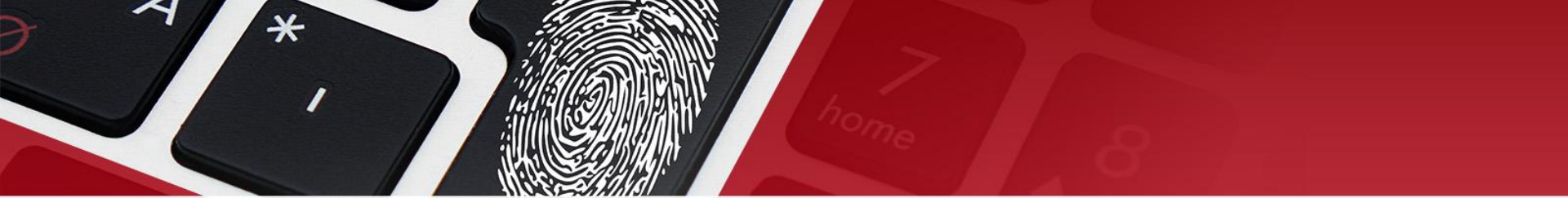
$$P(\text{Defect} | \text{Machine2}) = \frac{0.5 * 0.01}{0.4} = 0.0125 \Rightarrow 1.25\%$$



Naïve Bayes' theorem

It is a classification technique based on Bayes' Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature.

$$P(A|B) = \frac{P(A) P(B|A)}{P(B)}$$



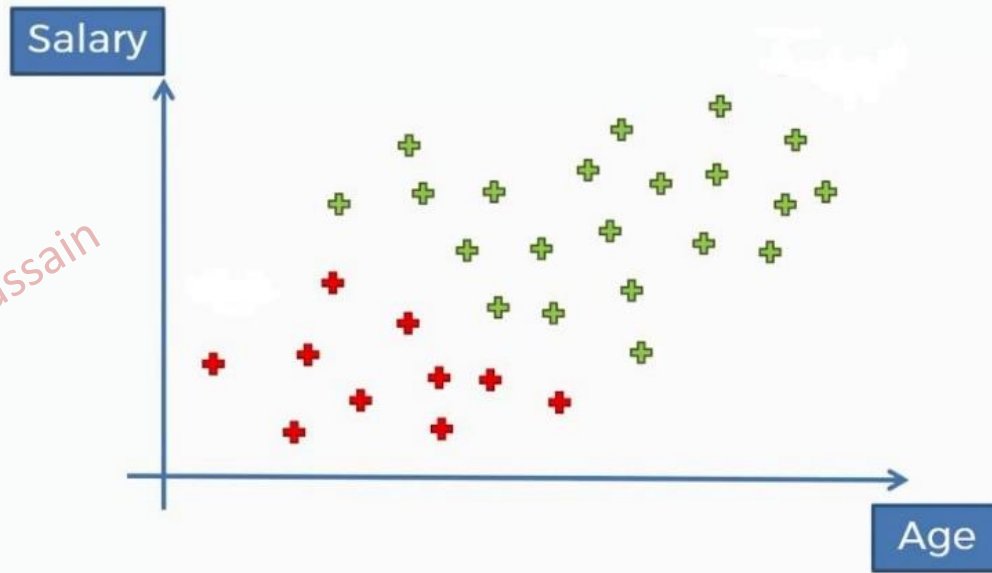
Posterior Probability

Likelihood 0.3

Prior Probability
0.33

$$P(\text{ClassA} \mid X) = \frac{P(X \mid \text{ClassA}) * P(\text{ClassA})}{P(X)} = 0.75$$

Marginal Probability
0.13



$$\underline{P(\text{Class A}) = 10 / 30 = 0.33}$$

$$\text{Using Circle } P(X) = 4 / 30 = 0.13$$

Using Circle with Red Data

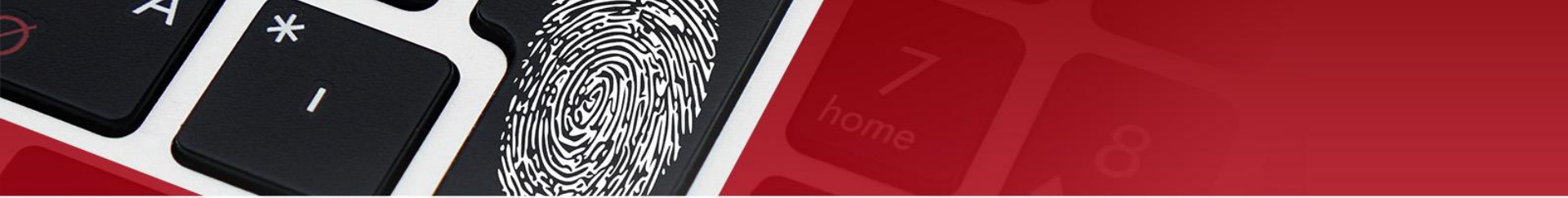
$$\underline{P(X | \text{Class A}) = 3 / 10 = 0.3}$$

$$\underline{P(\text{Class B}) = 20 / 30 = 0.66}$$

$$\text{Using Circle } P(X) = 4 / 30 = 0.13$$

Using Circle with Red Data

$$\underline{P(X | \text{Class B}) = 1 / 20 = 0.05}$$



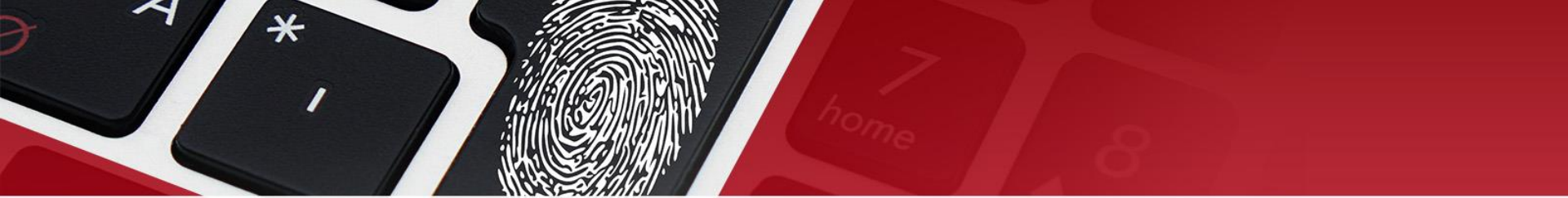
Posterior Probability

Likelihood

Prior Probability

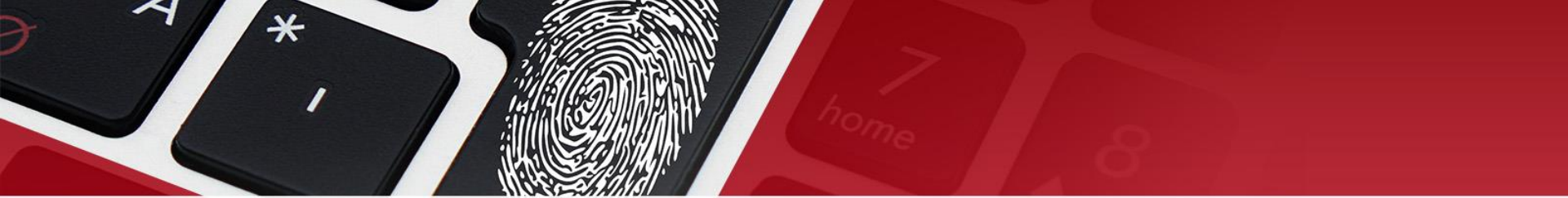
$$P(\text{ClassB} \mid X) = \frac{P(X \mid \text{ClassB}) * P(\text{ClassB})}{P(X)} = 0.25$$

Marginal Probability



$$0.75 < 0.25$$

New Data will be allocated to
ClassA



There are three types of Naive Bayes model under the scikit-learn library:

- **Gaussian**: It is used in classification and it assumes that features follow a normal distribution.
- **Multinomial**: It is used for discrete counts. For example, let's say, we have a text classification problem. Here we can consider Bernoulli trials which is one step further and instead of "word occurring in the document", we have "count how often word occurs in the document", you can think of it as "number of times outcome number x_i is observed over the n trials".
- **Bernoulli**: The binomial model is useful if your feature vectors are binary (i.e. zeros and ones). One application would be text classification with 'bag of words' model where the 1s & 0s are "word occurs in the document" and "word does not occur in the document" respectively.

Decision Tree Classifier

A Classification And Regression Tree (**CART**), is a predictive model, which explains how an outcome variable's values can be predicted based on other values. A **CART** output is a decision tree where each fork is a split in a predictor variable and each end node contains a prediction for the outcome variable





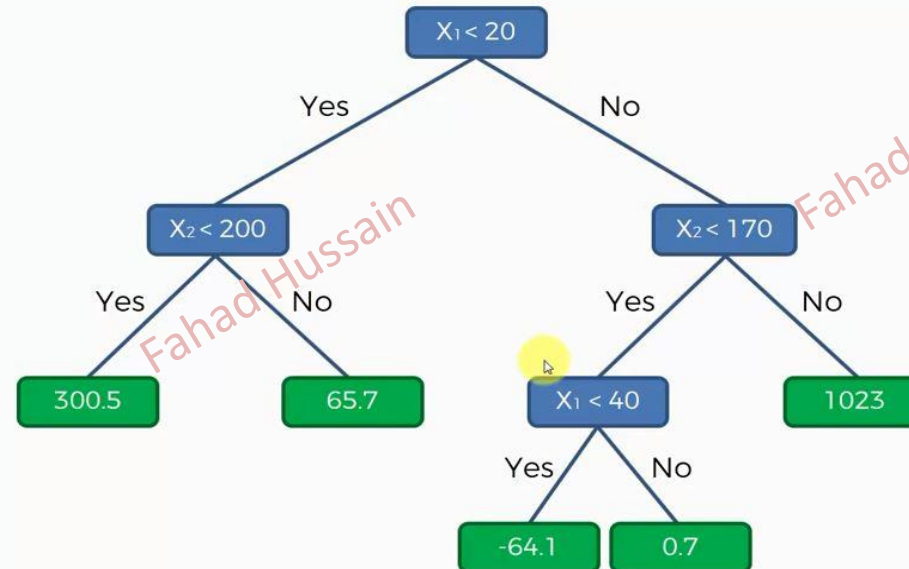
Decision Tree Classifier

Decision Tree Learning Algorithm

- There are many specific decision-tree algorithms. Notable ones include:
- ID3 (Iterative Dichotomiser 3)
- C4.5 (successor of ID3)
- CART (Classification And Regression Tree)
- CHAID (Chi-squared Automatic Interaction Detector). Performs multi-level splits when computing classification trees.
- MARS: extends decision trees to handle numerical data better.

ID3 Decision Tree Classifier

ID3: The core algorithm for building decision trees is called **ID3**. Developed by J. R. Quinlan, this algorithm employs a top-down, greedy search through the space of possible branches with no backtracking. ID3 uses *Entropy* and *Information Gain* to construct a decision tree.





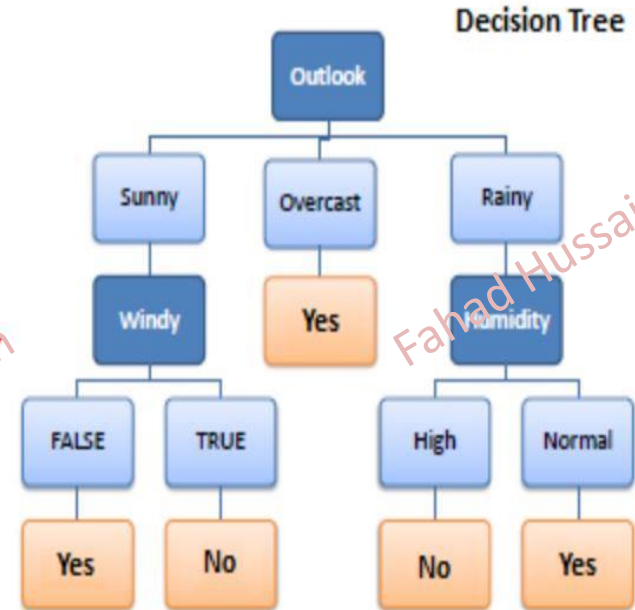
Concept to map Decision Tree

Information Gain: The **information gain** is based on the decrease in entropy after a data-set is split on an attribute. Constructing a **decision tree** is all about finding attribute that returns the highest **information gain** (i.e., the most homogeneous branches) ***OR [a measure of the decrease in disorder achieved by partitioning the original dataset]***

Entropy: Entropy, as it relates to machine learning, is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information. Flipping a coin is an example of an action that provides information that is random. ... This is the essence of entropy.
. Or [is a measure of disorder in a dataset]

ID3 Decision Tree Classifier

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



Concept to map Decision Tree

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

Entropy(PlayGolf) = Entropy(5,9)
= Entropy(0.36, 0.64)
= $-(0.36 \log_2 0.36) - (0.64 \log_2 0.64)$
= 0.94

$$= -p / p + n * \log(p / p + n) - n / p + n * \log(n / p + n)$$

$$= -9 / 9 + 5 * \log(9 / 9 + 5) - 5 / 9 + 5 * \log(5 / 9 + 5)$$

Here $p = 9, n = 5$ also for $\log_2 = \log(?) / \log(2)$

Concept to map Decision Tree

$$E(T, X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

$$E(\text{PlayGolf, Outlook}) = P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3)$$
$$= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971$$
$$= 0.693$$

$$-\frac{2}{5} \log \frac{2}{5} - \frac{3}{5} \log \frac{3}{5} = 0.970$$

$$-\frac{4}{5} \log \frac{4}{5} - \frac{3}{5} \log 0 = 0$$

$$-\frac{3}{5} \log \frac{3}{5} - \frac{2}{5} \log \frac{2}{5} = 0.970$$

Concept to map Decision Tree

$$E(T, X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

$$\begin{aligned} E(\text{PlayGolf}, \text{Outlook}) &= P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3) \\ &= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971 \\ &= 0.693 \end{aligned}$$

$$\text{Entropy } E(A) = \frac{p_i + n_i}{p + n} * (I(p, n))$$

Concept to map Decision Tree

		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			

		Play Golf	
		Yes	No
Temp.	Hot	2	2
	Mild	4	2
	Cool	3	1
Gain = 0.029			

Information Gain

$$Gain(T, X) = Entropy(T) - Entropy(T, X)$$

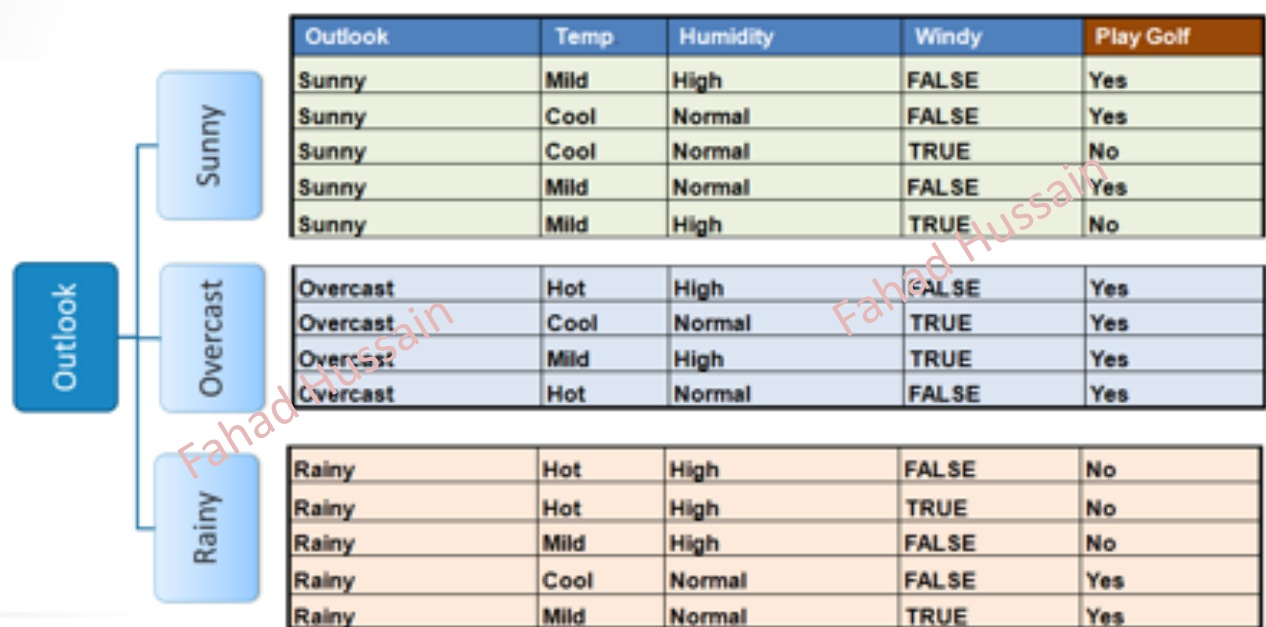
$$\begin{aligned} G(\text{PlayGolf}, \text{Outlook}) &= E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook}) \\ &= 0.940 - 0.693 = 0.247 \end{aligned}$$

		Play Golf	
		Yes	No
Humidity	High	3	4
	Normal	6	1
Gain = 0.152			

		Play Golf	
		Yes	No
Windy	False	6	2
	True	3	3
Gain = 0.048			

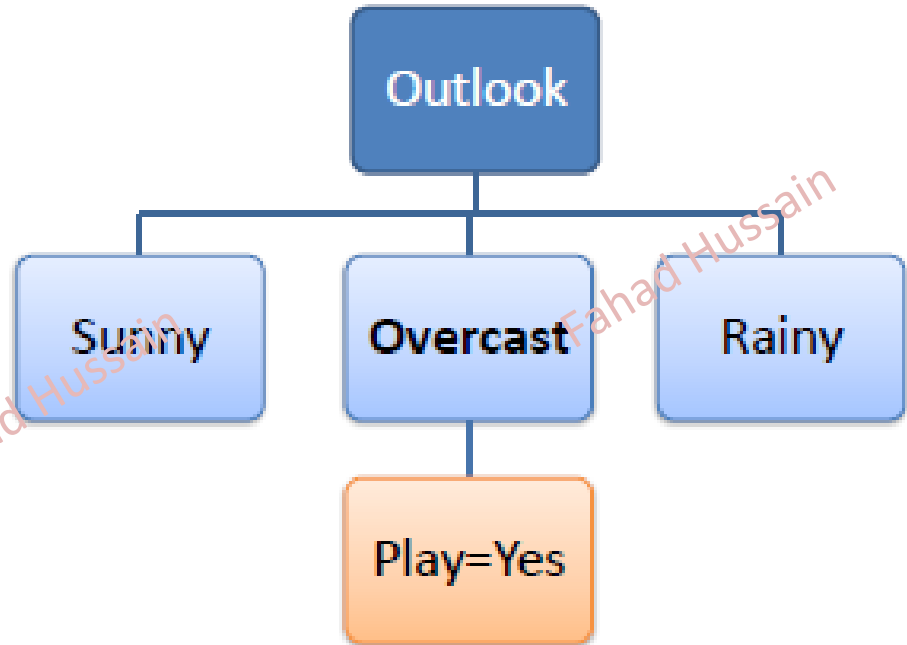
Concept to map Decision Tree

★		Play Golf	
		Yes	No
Outlook	Sunny	3	2
	Overcast	4	0
	Rainy	2	3
Gain = 0.247			



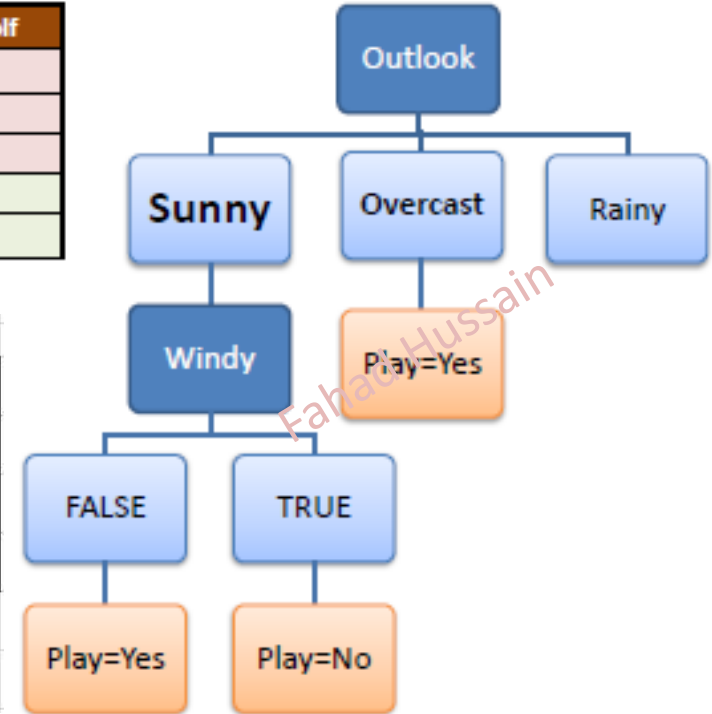
Concept to map Decision Tree

Temp.	Humidity	Windy	Play Golf
Hot	High	FALSE	Yes
Cool	Normal	TRUE	Yes
Mild	High	TRUE	Yes
Hot	Normal	FALSE	Yes



Concept to map Decision Tree

Temp.	Humidity	Windy	Play Golf
Mild	High	FALSE	Yes
Cool	Normal	FALSE	Yes
Mild	Normal	FALSE	Yes
Cool	Normal	TRUE	No
Mild	High	TRUE	No



WIND				Temp			
wind	pi	ni	E	Temp	pi	ni	E
TRUE	0	2	0	hot	0	0	0
FALSE	3	0	0	mild	2	1	0.9
				cold	1	1	1
E	0			E	0.951		
G	0.97			G	0.019		

Concept to map Decision Tree

A decision tree can easily be transformed to a set of rules by mapping from the root node to the leaf nodes one by one.

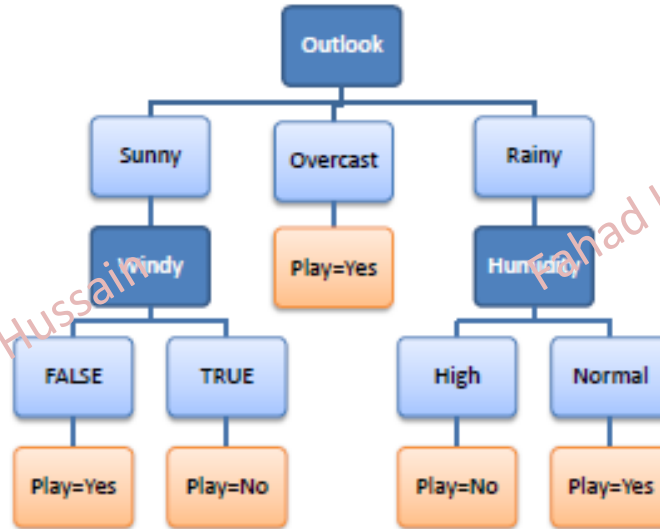
R_1 : IF (Outlook=Sunny) AND (Windy=FALSE) THEN Play=Yes

R_2 : IF (Outlook=Sunny) AND (Windy=TRUE) THEN Play=No

R_3 : IF (Outlook=Overcast) THEN Play=Yes

R_4 : IF (Outlook=Rainy) AND (Humidity=High) THEN Play=No

R_5 : IF (Outlook=Rain) AND (Humidity=Normal) THEN Play=Yes





Concept to map Decision Tree

Gini Index

Gini index says, if we select two items from a population at random then they must be of same class and probability for this is 1 if population is pure.

It can be used only if the target variable is a binary variable Classification and regression tree (CART).

Concept to map Decision Tree

$$Gini(D) = 1 - \sum_{i=1}^m p_i^2,$$

$$Gini(D) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$



RandomForestTree Classifier

In statistics and machine learning, ensemble methods use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone.



Steps to make RandomForestTree Classifier

STEP 1: Pick at random K data points from the Training set.

STEP 2: Build the Decision Tree associated to these K data points.

STEP 3: Choose the number N_{tree} of trees you want to build and repeat STEPS 1 & 2

STEP 4: For a new data point, make each one of your N_{tree} trees predict the category to which the data points belongs, and assign the new data point to the category that wins the majority vote.



Steps to make RandomForestTree Classifier

STEP 1: Pick at random K data points from the Training set.

STEP 2: Build the Decision Tree associated to these K data points.

STEP 3: Choose the number N_{tree} of trees you want to build and repeat STEPS 1 & 2

STEP 4: For a new data point, make each one of your N_{tree} trees predict the category to which the data points belongs, and assign the new data point to the category that wins the majority vote.



Unsupervised Learning

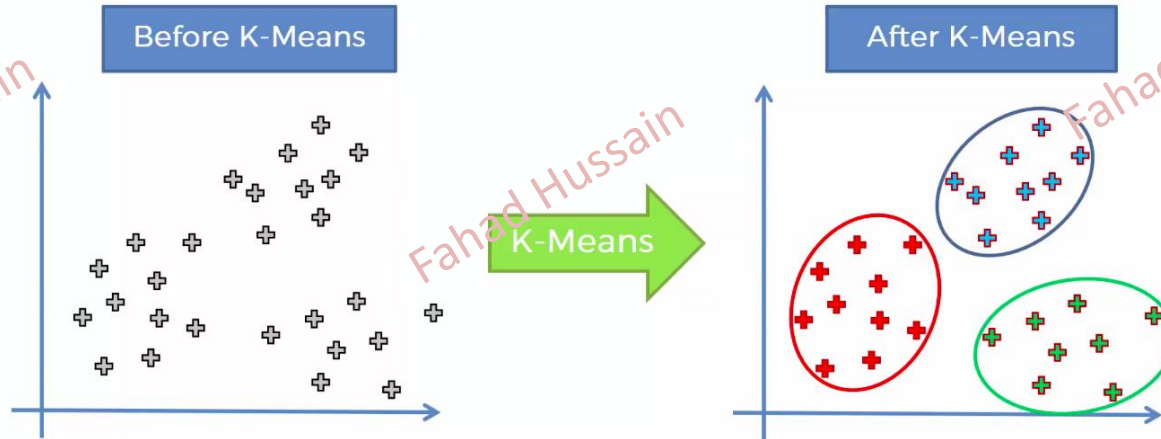
Clustering is similar to classification, but the basis is different.

In Clustering you don't know what you are looking for, and you are trying to identify some segments or clusters in your data. When you use clustering algorithms on your dataset, unexpected things can suddenly pop up like structures, clusters and groupings you would have never thought of otherwise.

K-Means Clustering

What is K Means Clustering

K-means clustering is one of the simplest and popular unsupervised machine learning algorithms. ... In other words, the **K-means algorithm** identifies **k** number of centroids, and then allocates every data point to the nearest **cluster**, while keeping the centroids as small as possible

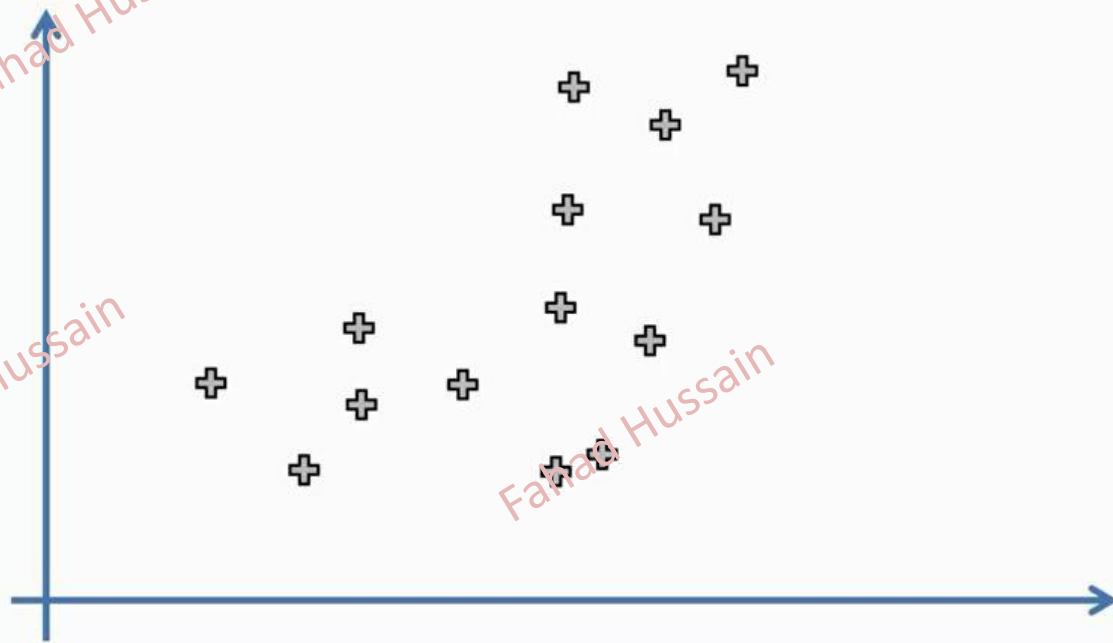




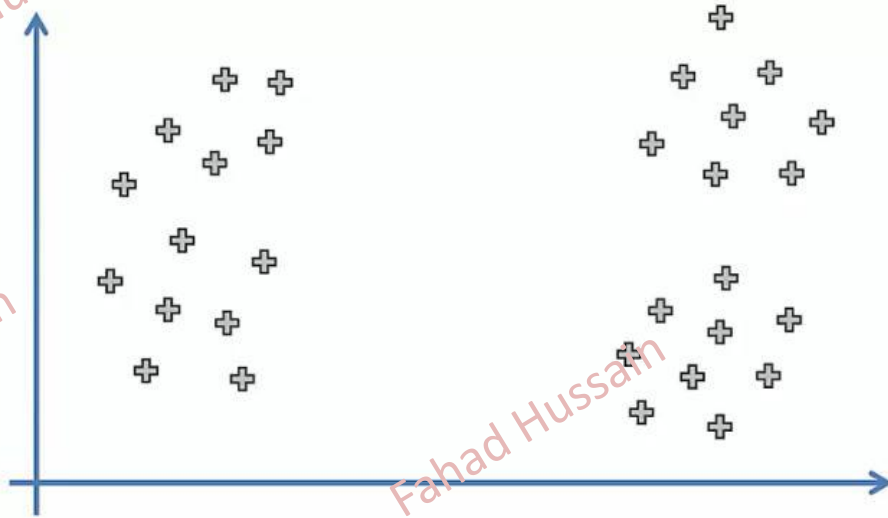
Steps to Follow

- 1. Choose the number of K of clusters**
- 2. Select at random K points, the centroids (not necessarily from your dataset)**
- 3. Assign each data point to the closest centroid => That forms K clusters**
- 4. Compute and place the new centroids of each cluster**
- 5. Reassign each data point to the new closest centroid if any reassignment took place, go to STEPS 4, OTHERWISE GOT TO FINISH!**

Visualization



Randomization initial problem

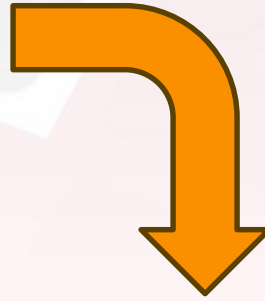


If we choose $K = 3$ clusters...



K value initial problem

k means algorithm



k means plus plus algorithm

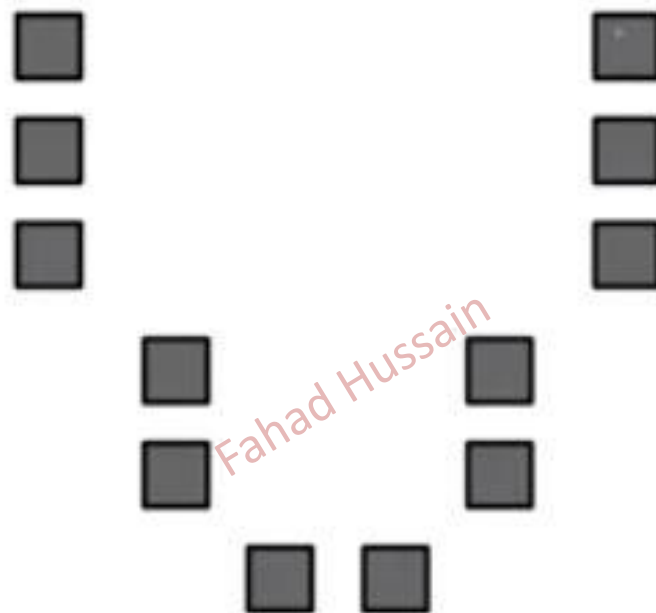
The k -means problem is to find cluster centers that minimize the intra-class variance, i.e. the sum of squared distances from each data point being clustered to its cluster center (the center that is closest to it). Although finding an exact solution to the k -means problem for arbitrary input is NP-hard.



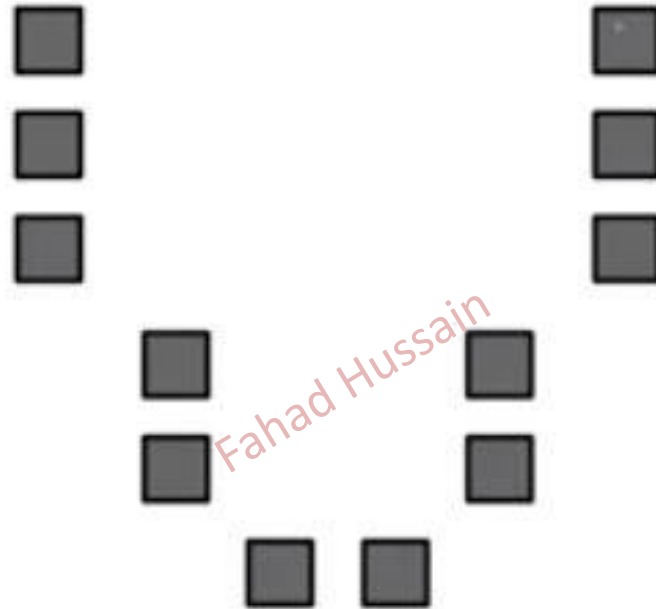
Steps to follow

1. ***Choose first cluster center uniformly at random from data points***
2. ***For each obs x , compute the distance $d(x)$ to nearest cluster center***
3. ***Choose new cluster center from amongst data points, with probability of x being chosen proportional to $d(x)^2$***
4. ***Repeat steps 2 and 3 until k centers have been chosen.***

Point to Visualize



Point to Visualize

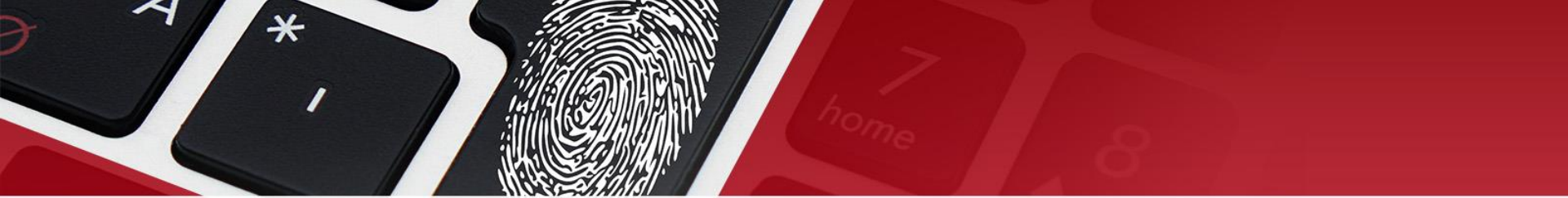




Elbow Method

The KMeans algorithm can cluster observed data. But how many clusters (k) are there?

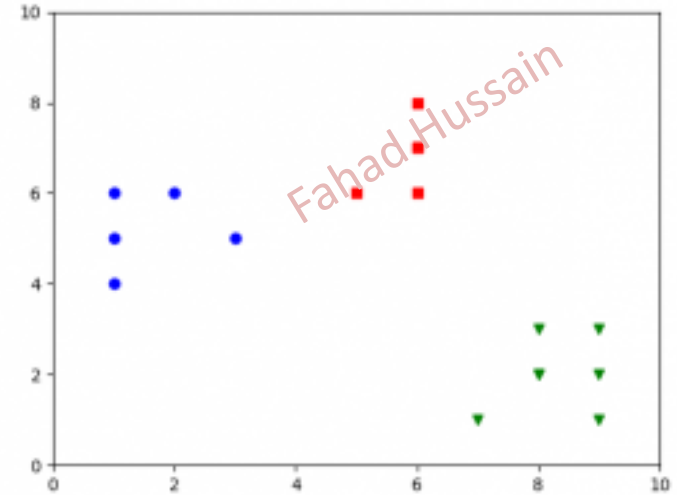
The elbow method finds the optimal value for k (#clusters)



Determine optimal k The technique to determine K , the number of clusters, is called the elbow method.

We'll plot:

- Values for K on the horizontal axis
- The distortion on the Y axis (the values calculated with the cost function).

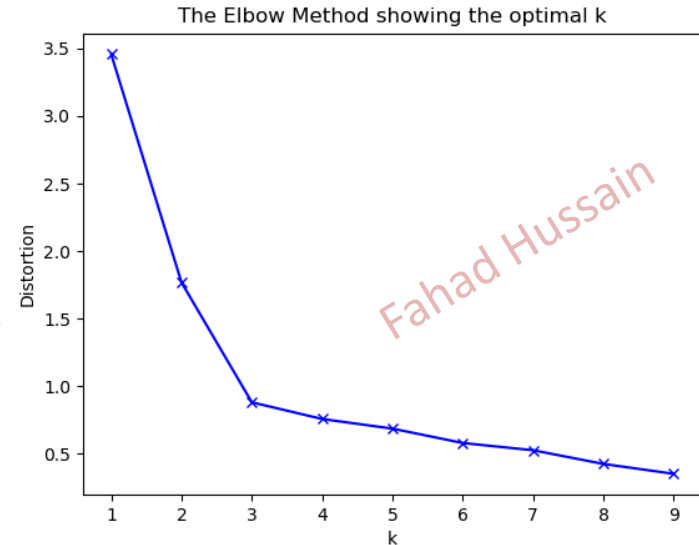


Elbow Method

When K increases, the centroids are closer to the clusters centroids.

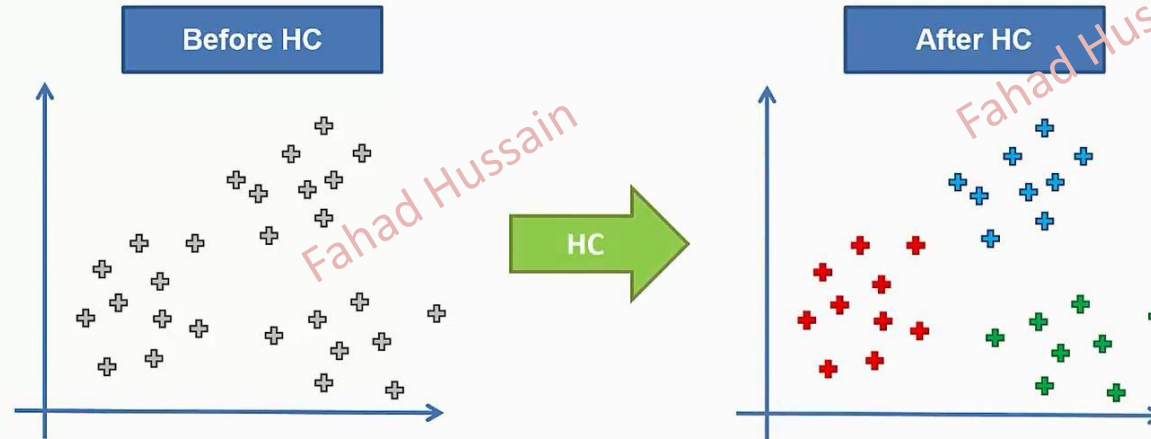
The improvements will decline, at some point rapidly, creating the elbow shape.

That point is the optimal value for K . In the image above, $K=3$.



Hierarchical clustering

Hierarchical clustering, also known as *hierarchical cluster analysis*, is an algorithm that groups similar objects into groups called *clusters*. The endpoint is a set of clusters, where each cluster is distinct from each other cluster, and the objects within each cluster are broadly similar to each other.





Hierarchical clustering Methods

Agglomerative & Divisive

Also known as bottom-up approach or hierarchical agglomerative clustering (HAC). A structure that is more informative than the unstructured set of clusters returned by flat clustering. This clustering algorithm does not require us to pre-specify the number of clusters. Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerates pairs of clusters until all clusters have been merged into a single cluster that contains all data.



Hierarchical clustering Points

1. *Make each data point a single-point cluster, means N cluster*
2. *Take the two closest data points and make them one cluster $N-1$*
3. *Take the two closest cluster and make them one cluster*
4. *Repeat Step 3 until there is only one cluster*



Distance Between Clusters

$$\text{Euclidean Distance between } P_1 \text{ and } P_2 = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

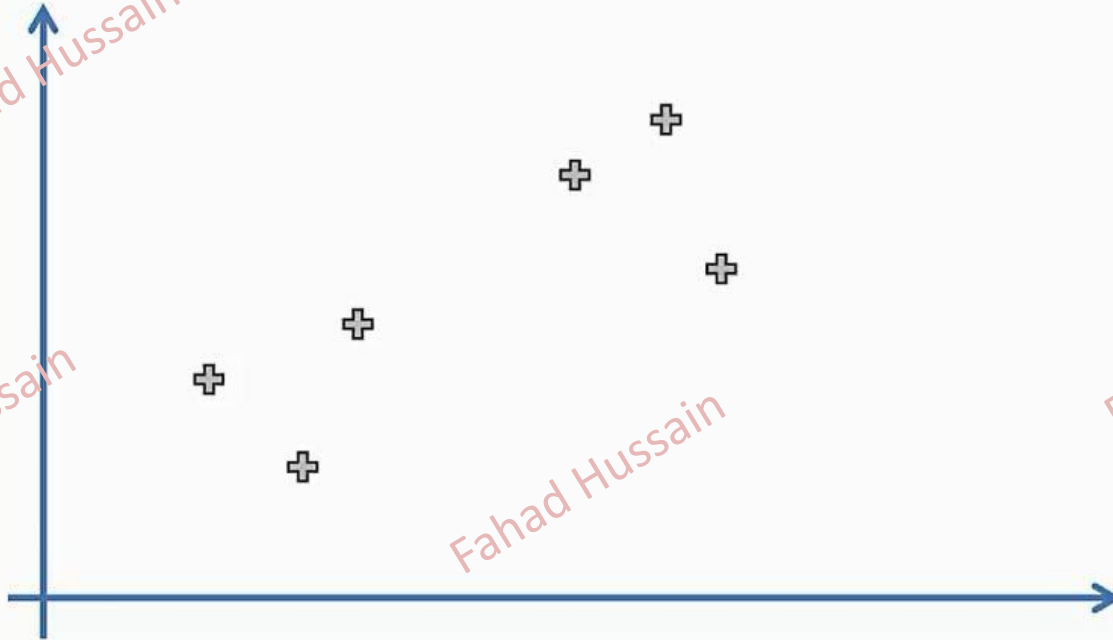
Option 1: closest Point distance

Option 2: furthest points

Option 1: Average Distance

Option 1: Distance Between Centroids

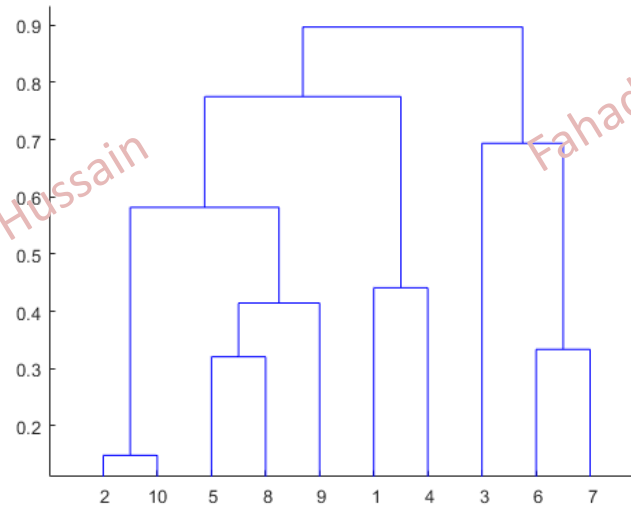
Applying HC on example



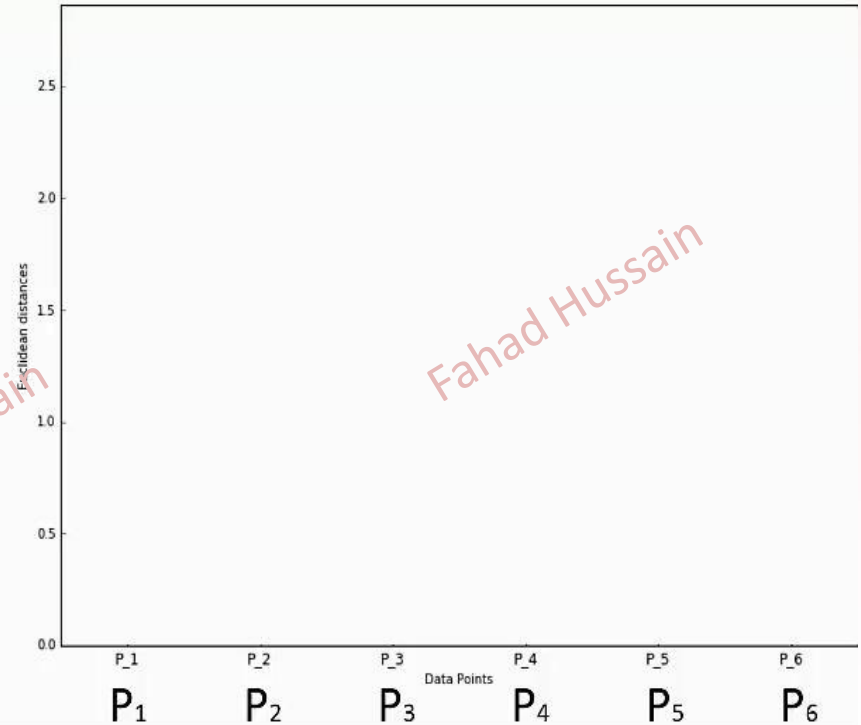
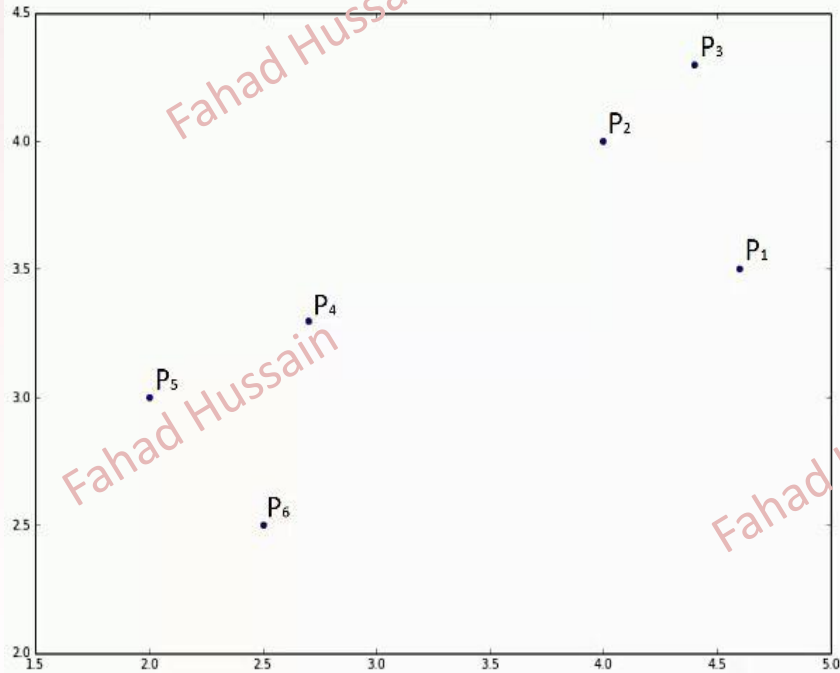
But, how can we find the optimal number of cluster, for solving this problem Dendograms come play its role!

Dendrogram

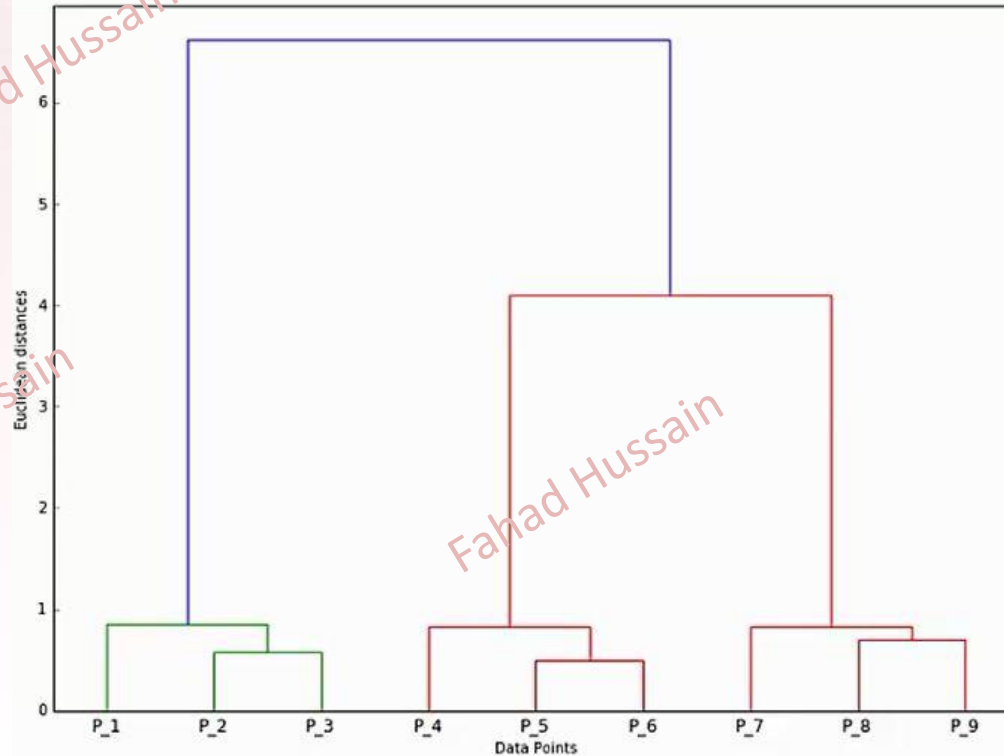
A dendrogram is a diagram representing a tree. This diagrammatic representation is frequently used in different contexts: in hierarchical clustering, it illustrates the arrangement of the clusters produced by the corresponding analyses.



Dendrogram Solving for *Data Example*



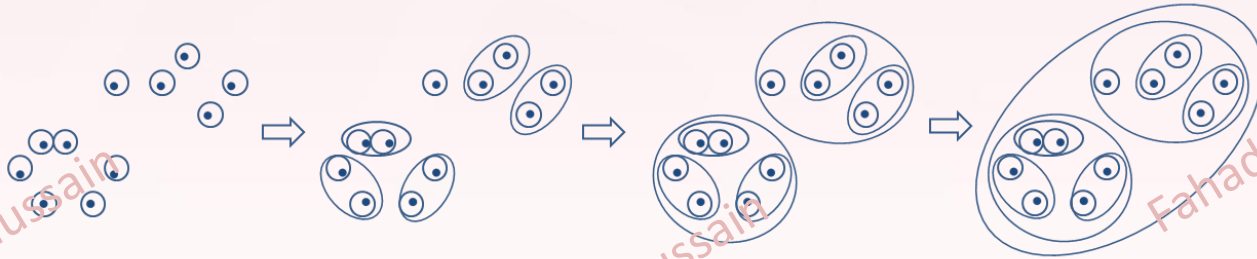
Dendrogram Quiz



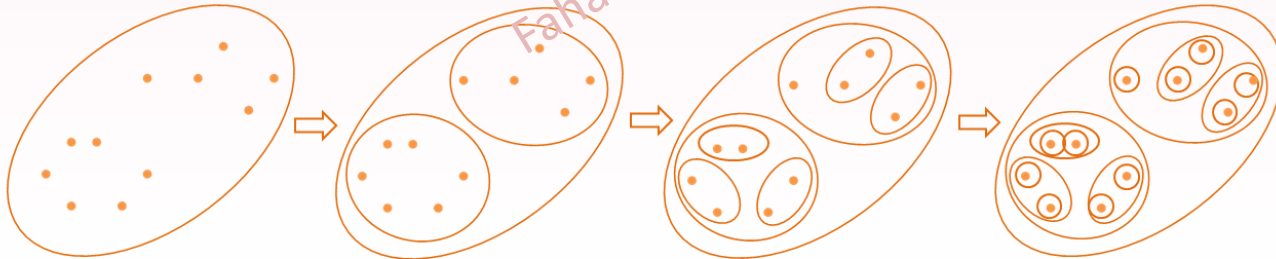
Divisive Hierarchical Clustering

The **divisive** hierarchical **clustering**, also known as DIANA (**D**ivisive **A**NALysis) is the inverse of agglomerative **clustering**.

Agglomerative Hierarchical Clustering



Divisive Hierarchical Clustering

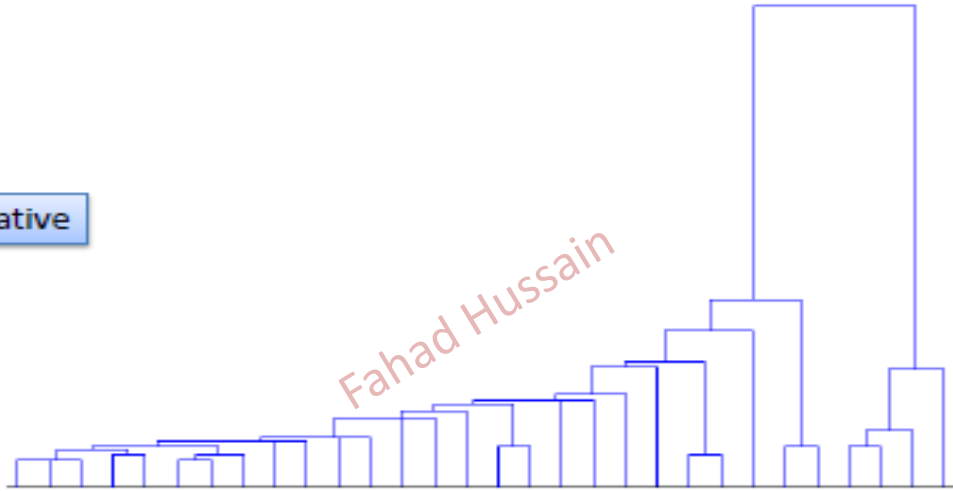


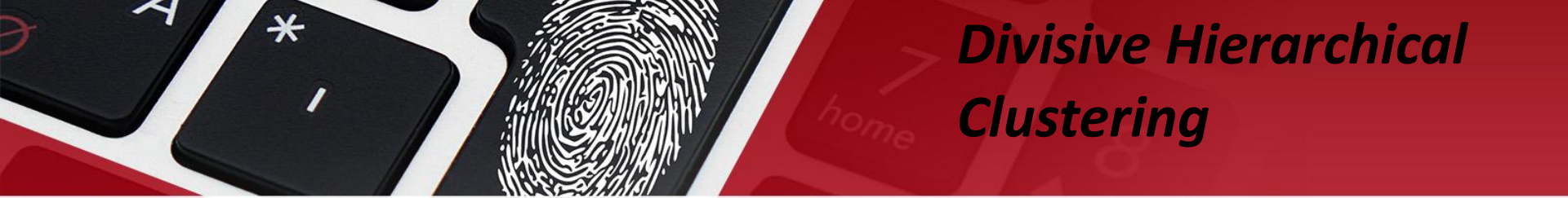
Divisive Hierarchical Clustering

Hierarchical Clustering

Agglomerative

Divisive





Divisive Hierarchical Clustering

Agglomerative Hierarchical Clustering

Bottom-up strategy

Each cluster starts with only one object

Clusters are merged into larger and larger clusters until:

All the objects are in a single cluster

Certain termination conditions are satisfied

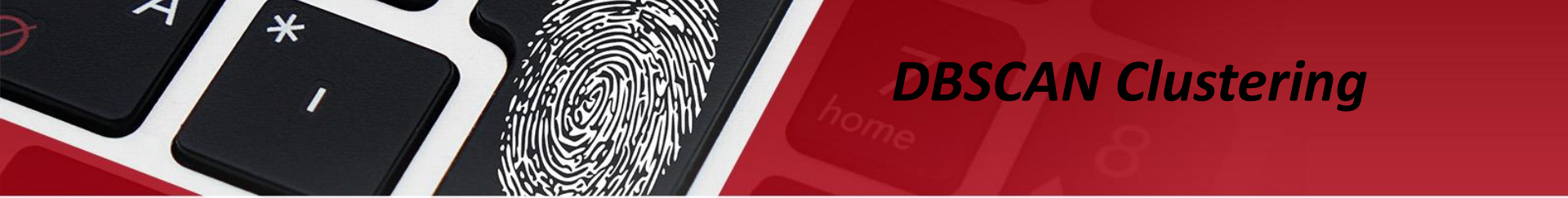
Divisive Hierarchical Clustering

Top-down strategy

Start with all objects in one cluster

Clusters are subdivided into smaller and smaller clusters until:

Each object forms a cluster on its own



DBSCAN Clustering

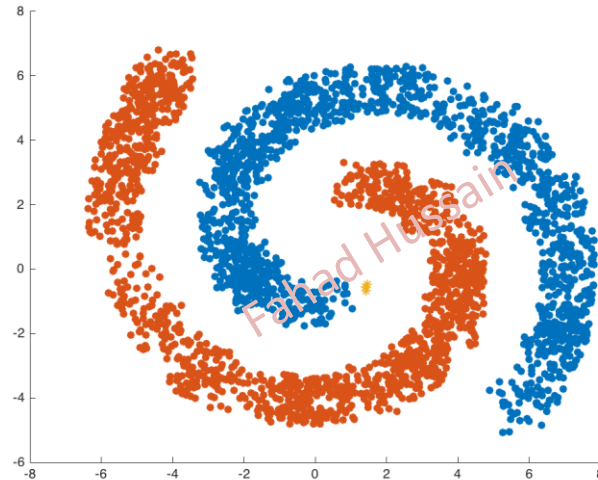
DBSCAN stands for Density-based spatial clustering of applications with noise, DBSCAN is a **clustering** method that is used in machine learning to separate **clusters** of high density from **clusters** of low density.

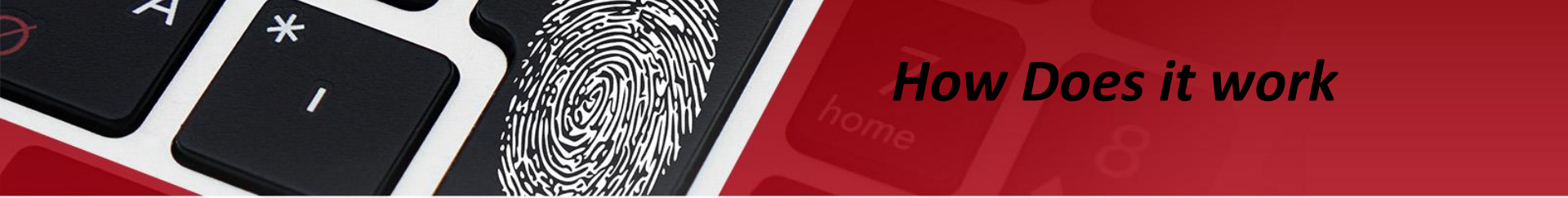
eps : It defines the neighborhood around a data point i.e. if the distance between two points is lower or equal to 'eps' then they are considered as neighbors. If the eps value is chosen too small then large part of the data will be considered as outliers. If it is chosen very large then the clusters will merge and majority of the data points will be in the same clusters. One way to find the eps value is based on the k-distance graph.

MinPts: Minimum number of neighbors (data points) within eps radius. Larger the dataset, the larger value of MinPts must be chosen. As a general rule, the minimum MinPts can be derived from the number of dimensions D in the dataset as, $\text{MinPts} \geq D+1$. The minimum value of MinPts must be chosen at least 3.

DBSCAN Clustering

DBSCAN stands for Density-based spatial clustering of applications with noise. DBSCAN is a **clustering** method that is used in machine learning to separate **clusters** of high density from **clusters** of low density.





How Does it work

eps : It defines the neighborhood around a data point i.e. if the distance between two points is lower or equal to 'eps' then they are considered as neighbors. If the eps value is chosen too small then large part of the data will be considered as outliers. If it is chosen very large then the clusters will merge and majority of the data points will be in the same clusters. One way to find the eps value is based on the k-distance graph.

MinPts: Minimum number of neighbors (data points) within eps radius. Larger the dataset, the larger value of MinPts must be chosen. As a general rule, the minimum MinPts can be derived from the number of dimensions D in the dataset as, $\text{MinPts} \geq D+1$. The minimum value of MinPts must be chosen at least 3.

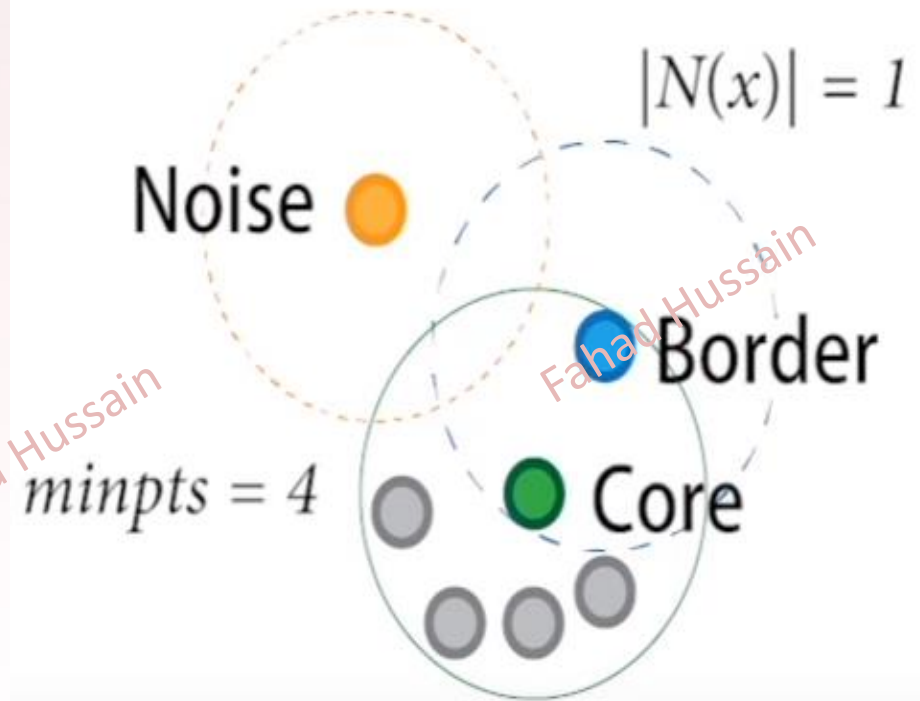
How Does it work

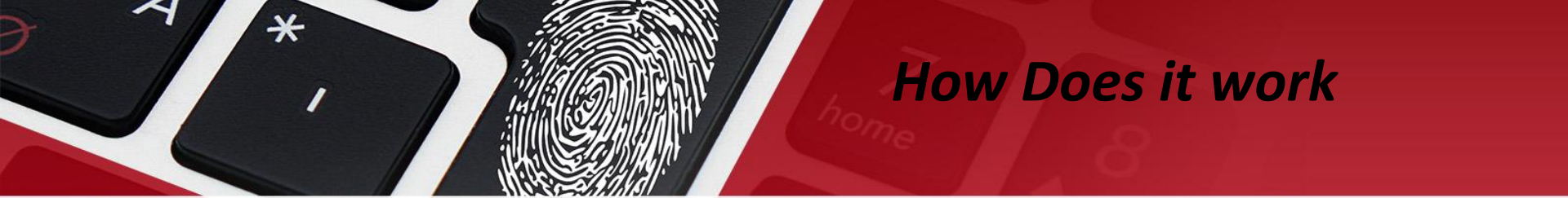
we have 3 types of data points.

Core Point: A point is a core point if it has more than MinPts points within eps.

Border Point: A point which has fewer than MinPts within eps but it is in the neighborhood of a core point.

Noise or outlier: A point which is not a core point or border point.

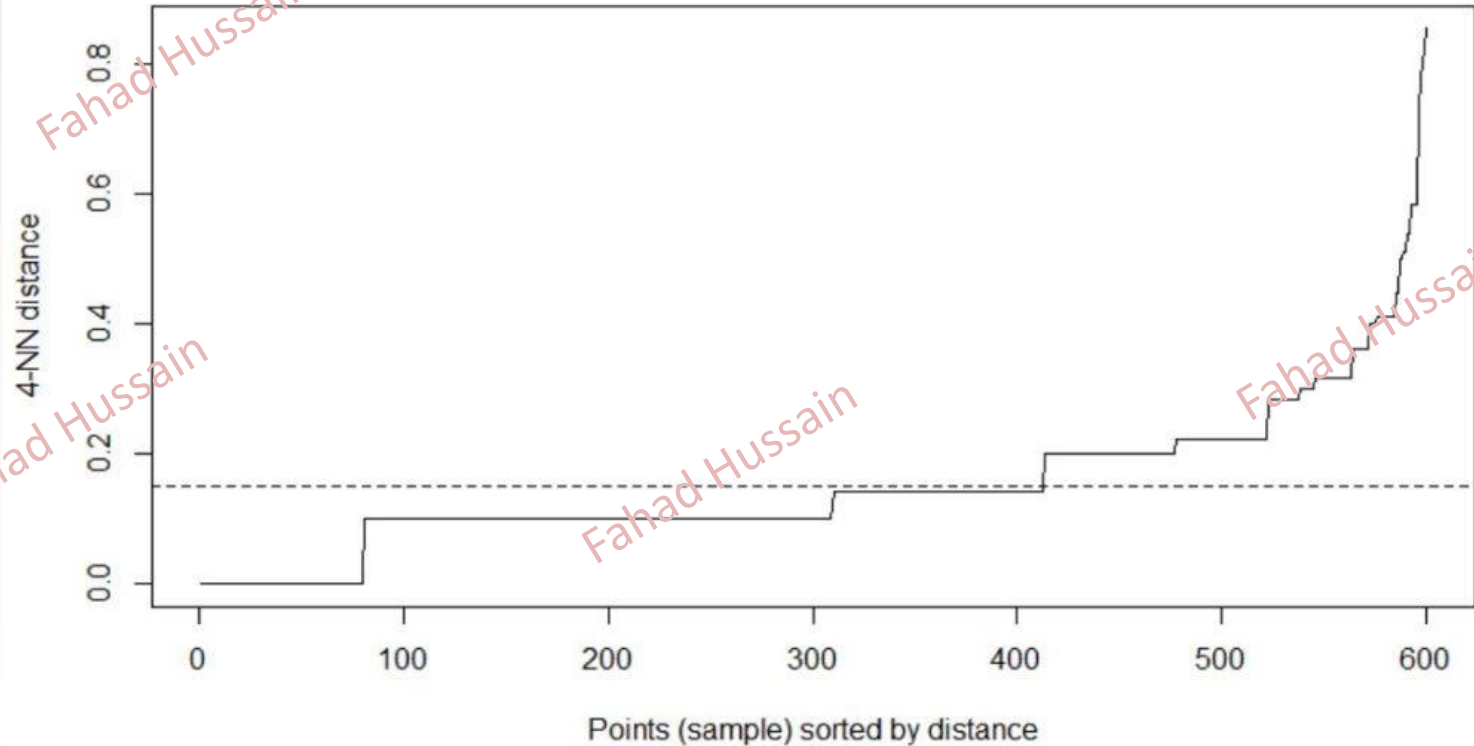




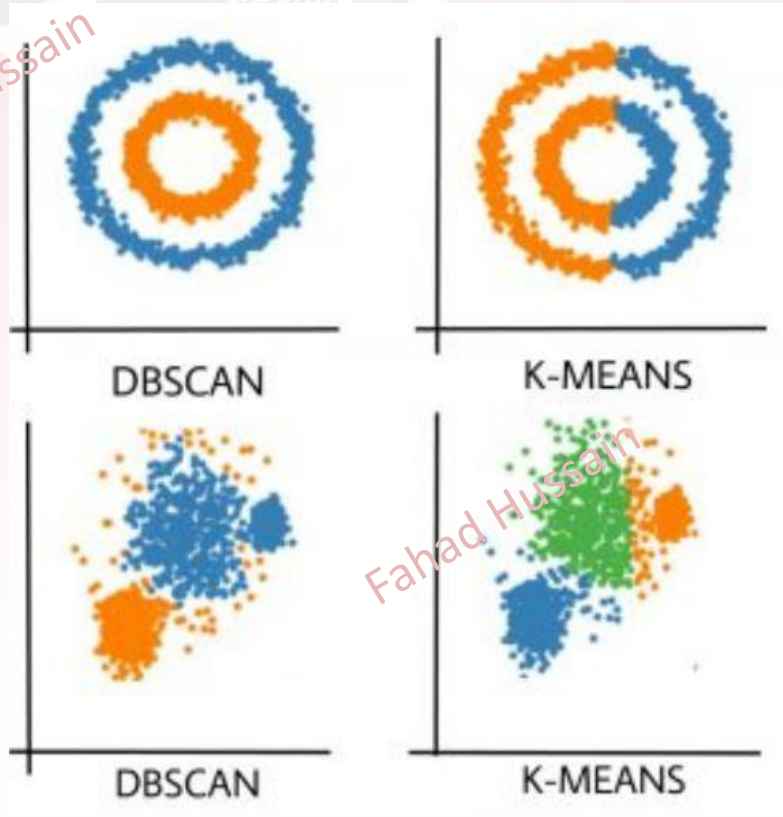
How Does it work

DBSCAN is of the clustering based method which is used mostly to identify outliers. **eps** is the maximum distance between two points. It is this distance that the algorithm uses to decide on whether to club the two points together. We will make use of the average distances of every point to its k nearest neighbors. These k distances are then plotted in ascending order. The point where you see an elbow like bend corresponds to the optimal ***eps*** value. At this point, a sharp change in the distance occurs, and thus this point serves as a threshold.

How Does it work



How Does it work





Queries as Comments

Thank You Very Much

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



Clusters' Summary

Clustering is similar to classification, but the basis is different.

In Clustering you don't know what you are looking for, and you are trying to identify some segments or clusters in your data. When you use clustering algorithms on your dataset, unexpected things can suddenly pop up like structures, clusters and groupings you would have never thought of otherwise.

It may be classified into Two form....

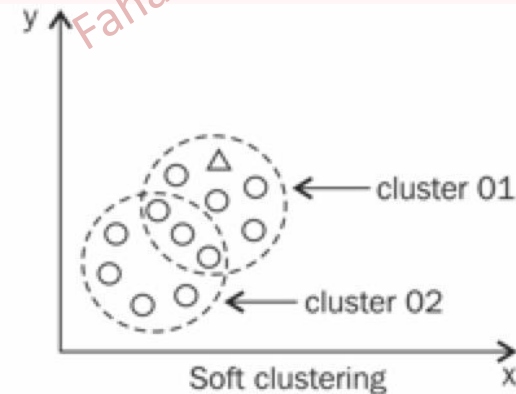
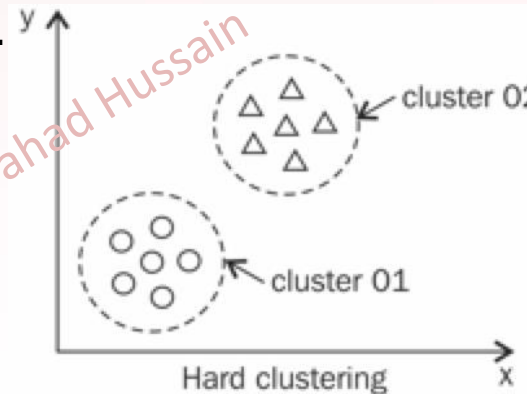
Hard Clustering

Soft Clustering

Clusters' Summary

•**Hard Clustering:** In hard clustering, each data point either belongs to a cluster completely or not. For example, in the above example each customer is put into one group out of the 10 groups.

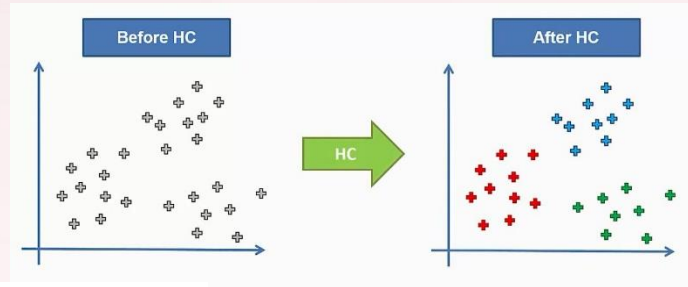
•**Soft Clustering:** In soft clustering, instead of putting each data point into a separate cluster, a probability or likelihood of that data point to be in those clusters is assigned. For example, from the above scenario each customer is assigned a probability to be in either of 10 clusters of the retail store.



Clusters' Summary

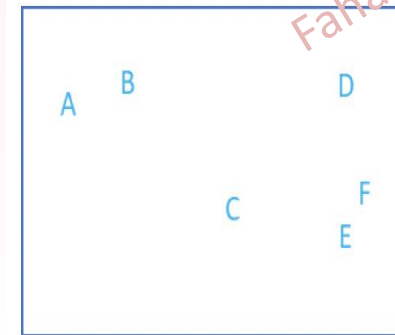
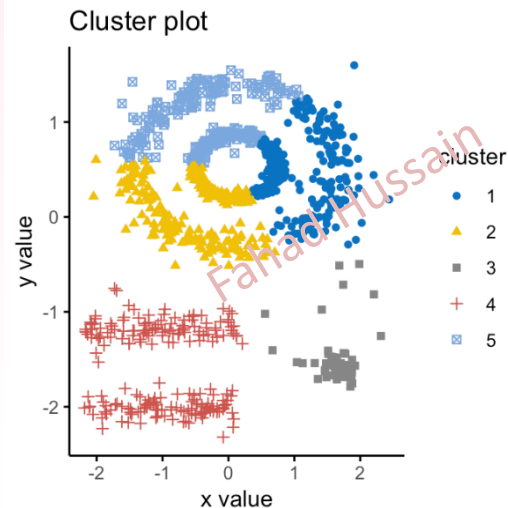
Discussed Cluster...

K means Clustering



Hierarchical clustering (including Agglomerative & Divisive Approach)

DBSCAN Clustering



Dendrogram





Clusters' Summary

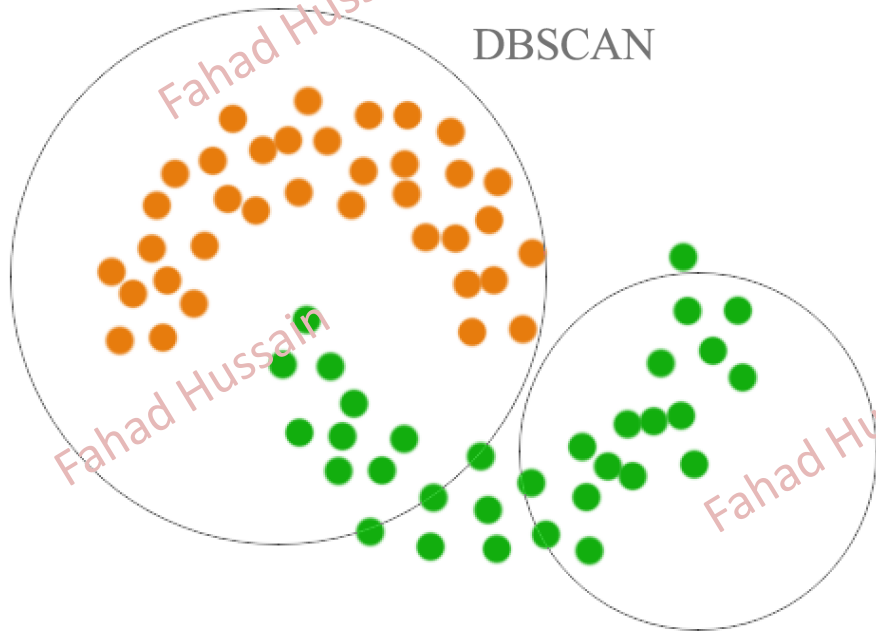
Hierarchical Clustering vs K-Means

	Hierarchical Clustering	K-Means
Running Time	Slower	Faster
Assumptions	Requires distance metric	Requires distance metric
Parameters	None	K (number of clusters)
Clusters	Subjective (only a tree is returned)	Exactly K clusters

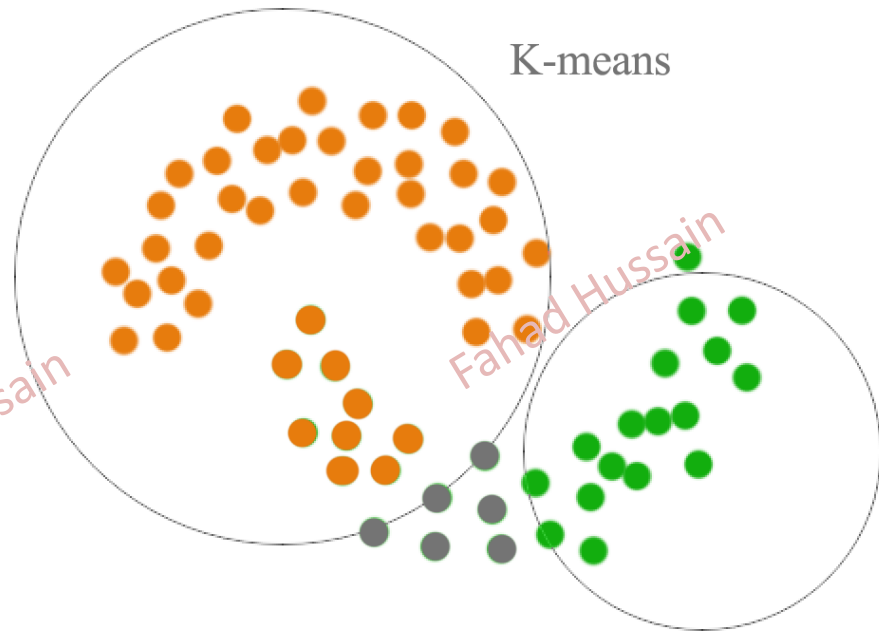
***Whereas
DBSCAN own its
Strength!!!***

Clusters' Summary

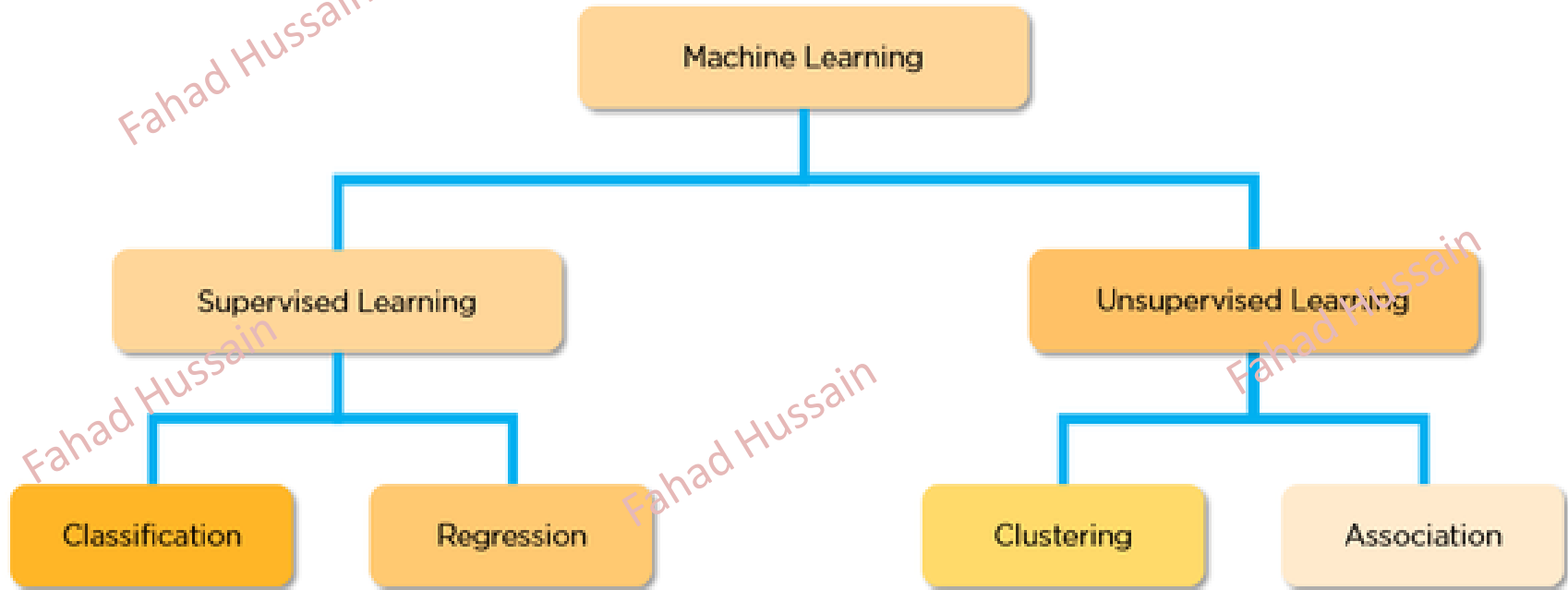
DBSCAN

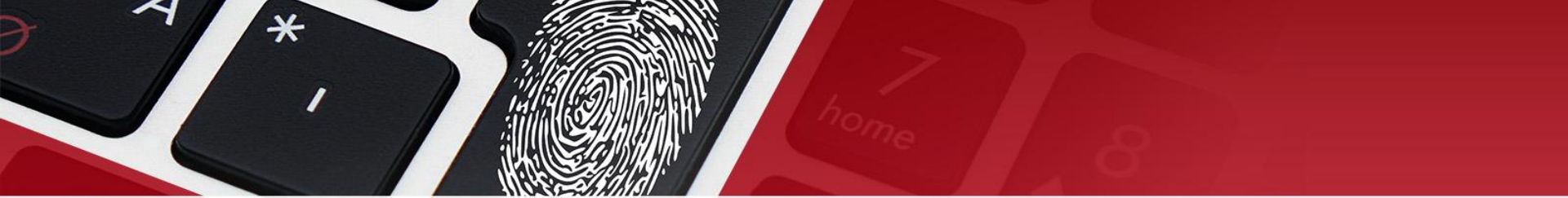


K-means



Supervised Learning Of Machine Learning





Association rule learning is a rule-based machine learning method for discovering interesting relations between variables in large databases. It is intended to identify strong rules discovered in databases using some measures of interestingness.

[wikipedia.org]

Way of Association





Association Rule

Association analysis identifies relationships between observations and variables from a dataset. These relationships are expressed by a set of rules that indicate groups of items that tend to be associated with others. By using this rule of association we can suggest / predict the interest of user in any Mall/ Market. Therefore it is used in market product normally...

Example database with 5 transactions and 5 items

transaction ID	milk	bread	butter	beer	diapers
1	1	1	0	0	0
2	0	0	1	0	0
3	0	0	0	1	1
4	1	1	1	0	0
5	0	1	0	0	0



Algorithm of Association

Apriori algorithm

Apriori algorithm, a classic algorithm, is useful in mining frequent itemsets and relevant association rules. Usually, you operate this algorithm on a database containing a large number of transactions. One such example is the items customers buy at a supermarket. It helps the customers buy their items with ease, and enhances the sales performance of the departmental store.



Algorithm of Association

Three significant components comprise the apriori algorithm.

Support

Confidence

Lift

Problem:

Out of the 2000 transactions, 200 contain jam whereas 300 contain bread. These 300 transactions include a 100 that includes bread as well as jam. Using this data, we shall find out the support, confidence, and lift.



Algorithm of Association

$$\text{support}(I) = \frac{\# \text{ transactions containing } I}{\# \text{ transactions}}$$

$$\text{confidence}(I_1 \rightarrow I_2) = \frac{\# \text{ transactions containing } I_1 \text{ and } I_2}{\# \text{ transactions containing } I_1}$$

$$\text{lift}(I_1 \rightarrow I_2) = \frac{\text{confidence}(I_1 \rightarrow I_2)}{\text{support}(I_2)}$$



Algorithm of Association

Support

Support is the default popularity of any item. You calculate the Support as a quotient of the division of the number of transactions containing that item by the total number of transactions.

$$\text{Support (Jam)} = (\text{Transactions involving jam}) / (\text{Total Transactions})$$

$$= 200 / 2000 = 10\%$$



Algorithm of Association

Confidence

Confidence is the likelihood that customer bought both bread and jam. Dividing the number of transactions that include both bread and jam by the total number of transactions will give the Confidence figure.

$$\text{Confidence} = (\text{Transactions involving both bread and jam}) / (\text{Total Transactions involving jam})$$
$$= 100 / 200 = 50\%$$

It implies that 50% of customers who bought jam bought bread as well.



Algorithm of Association

Lift

According to our example, Lift is the increase in the ratio of the sale of bread when you sell jam. The mathematical formula of Lift is as follows.

$$\begin{aligned}\text{Lift} &= (\text{Confidence (Jam} \rightarrow \text{Bread)}) / (\text{Support (Jam)}) \\ &= 50 / 10 = 5\end{aligned}$$

It says that the likelihood of a customer buying both jam and bread together is 5 times more than the chance of purchasing jam alone. If the Lift value is less than 1, it entails that the customers are unlikely to buy both the items together. Greater the value, the better is the combination.

APRIORI ALGORITHM EXAMPLE

Database D

Minsup = 0.5

TID	Items
100	1 3 4
200	2 3 5
300	1 2 3 5
400	2 5

Scan D

C_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{4}	1
{5}	3

L_1

itemset	sup.
{1}	2
{2}	3
{3}	3
{5}	3

L_2

itemset	sup
{1 3}	2
{2 3}	2
{2 5}	3
{3 5}	2

C_2

itemset	sup
{1 2}	1
{1 3}	2
{1 5}	1
{2 3}	2
{2 5}	3
{3 5}	2

Scan D

C_2

itemset
{1 2}
{1 3}
{1 5}
{2 3}
{2 5}
{3 5}

C_3

itemset
{2 3 5}

Scan D

L_3

itemset	sup
{2 3 5}	2

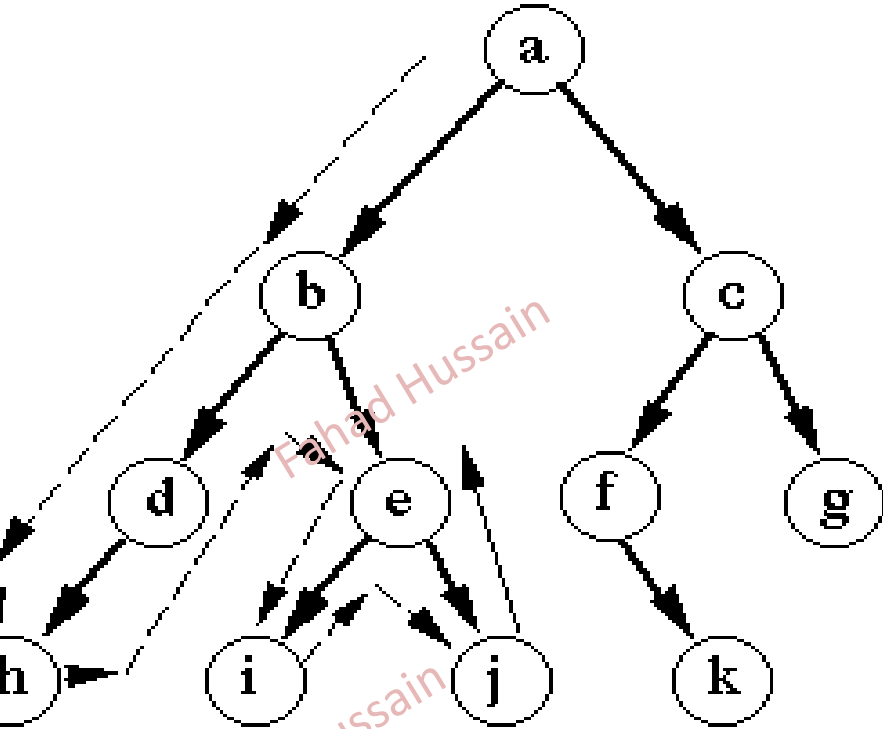


Algorithm of Association

Eclat Algorithm

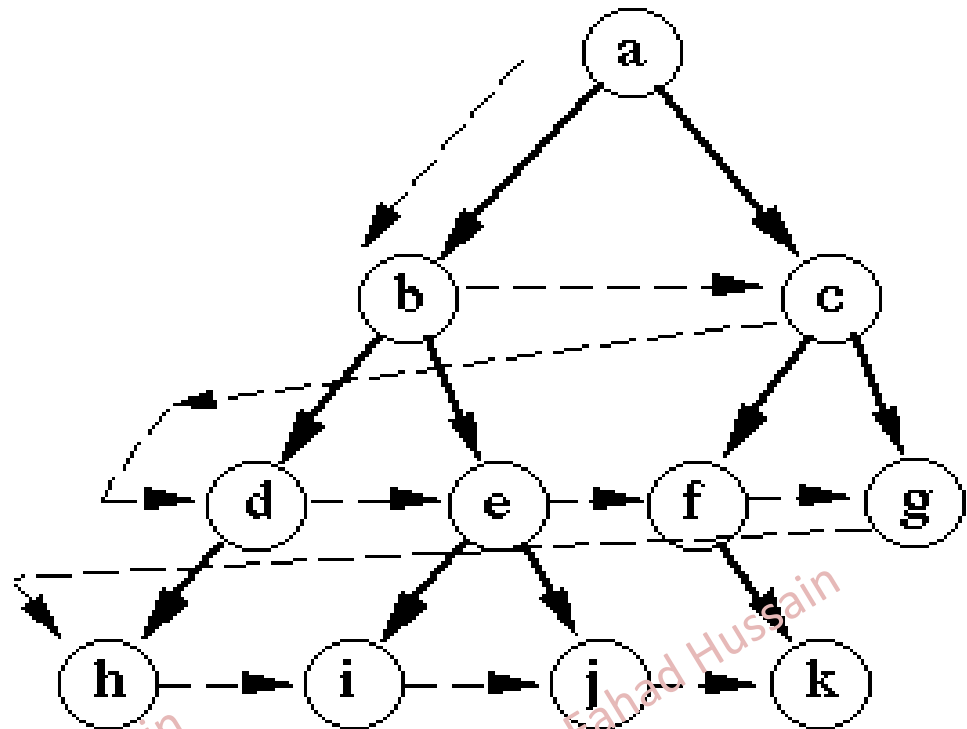
The ECLAT algorithm stands for **Equivalence Class Clustering and bottom-up Lattice Traversal**. It is one of the popular methods of Association Rule mining. It is a more efficient and scalable version of the Apriori algorithm.

While the Apriori algorithm works in a horizontal sense imitating the **Breadth-First Search** of a graph, the ECLAT algorithm works in a vertical manner just like the **Depth-First Search** of a graph. This vertical approach of the ECLAT algorithm makes it a faster algorithm than the Apriori algorithm.



Depth-first search

Eclat Algorithm work on vertical manner
like the **Depth-First Search**



Breadth-first search

Apriori Algorithm work horizontal sense
like **Breadth-First Search**



Working Strategy

The basic idea is to use Transaction Id Sets(tidsets) intersections to compute the support value of a candidate and avoiding the generation of subsets which do not exist in the prefix tree. In the first call of the function, all single items are used along with their tidsets. Then the function is called recursively and in each recursive call, each item-tidset pair is verified and combined with other item-tidset pairs. This process is continued until no candidate item-tidset pairs can be combined.

Eclat Model based on Support, to follow below rule:

- 1. Set a minimum support**
- 2. Take all the subsets in transactions having higher support than minimum support**
- 3. Sort these subsets by decreasing support**



Algorithm of Association

Support

Support is the default popularity of any item. You calculate the Support as a quotient of the division of the number of transactions containing that item by the total number of transactions.

Support (Jam) = (Transactions involving jam) / (Total Transactions)

$$= 200 / 2000 = 10\%$$

Transaction Id	Bread	Butter	Milk	Coke	Jam
T1	1	1	0	0	1
T2	0	1	0	1	0
T3	0	1	1	0	0
T4	1	1	0	1	0
T5	1	0	1	0	0
T6	0	1	1	0	0
T7	1	0	1	0	0
T8	1	1	1	0	1
T9	1	1	1	0	0

Working Strategy

k = 1, minimum support = 2

ITEM	TIDSET
Bread	{T1, T4, T5, T7, T8, T9}
Butter	{T1, T2, T3, T4, T6, T8, T9}
Milk	{T3, T5, T6, T7, T8, T9}
Coke	{T2, T4}
Jam	{T1, T8}

Working Strategy

$k = 2$

ITEM	TIDSET
{Bread, Butter}	{T1, T4, T8, T9}
{Bread, Milk}	{T5, T7, T8, T9}
{Bread, Coke}	{T4}
{Bread, Jam}	{T1, T8}
{Butter, Milk}	{T3, T6, T8, T9}
{Butter, Coke}	{T2, T4}
{Butter, Jam}	{T1, T8}
{Milk, Jam}	{T8}

Working Strategy

k = 3

ITEM	TIDSET
{Bread, Butter, Milk}	{T8, T9}
{Bread, Butter, Jam}	{T1, T8}

k = 4

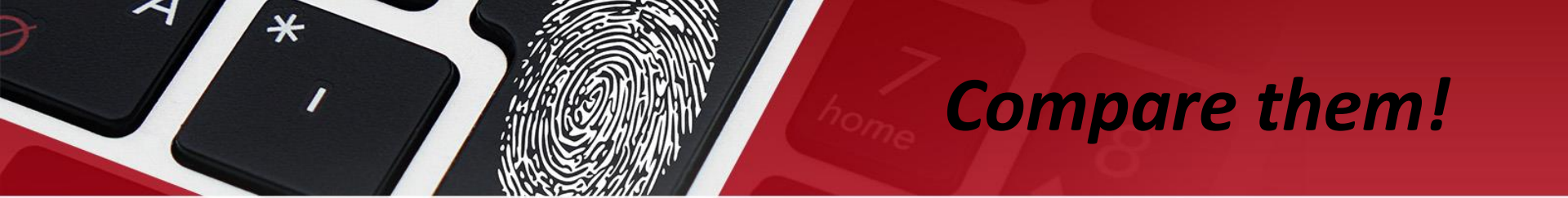
ITEM	TIDSET
{Bread, Butter, Milk, Jam}	{T8}

Working Strategy

We stop at $k = 4$ because there are no more item-tidset pairs to combine.

Since minimum **support=2**, we conclude the following rules from the given dataset!

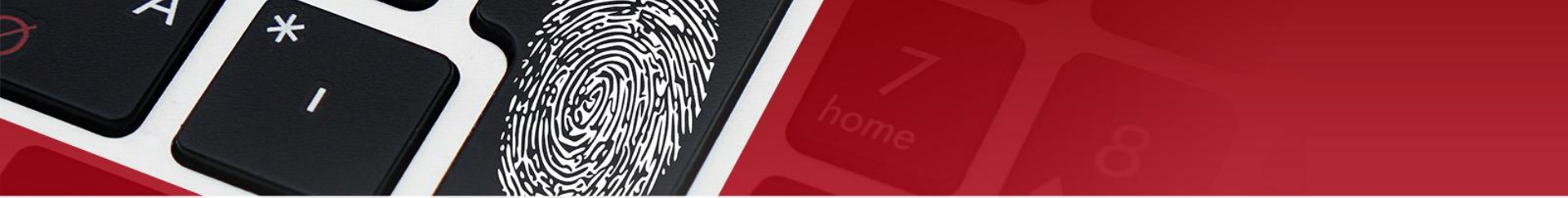
ITEMS BOUGHT	RECOMMENDED PRODUCTS
Bread	Butter
Bread	Milk
Bread	Jam
Butter	Milk
Butter	Coke
Butter	Jam
Bread and Butter	Milk
Bread and Butter	Jam



Compare them!

ITEM	TIDSET
{Bread, Butter}	{T1, T4, T8, T9}
{Bread, Milk}	{T5, T7, T8, T9}
{Bread, Coke}	{T4}
{Bread, Jam}	{T1, T8}
{Butter, Milk}	{T3, T6, T8, T9}
{Butter, Coke}	{T2, T4}
{Butter, Jam}	{T1, T8}
{Milk, Jam}	{T8}

ITEMS BOUGHT	RECOMMENDED PRODUCTS
Bread	Butter
Bread	Milk
Bread	Jam
Butter	Milk
Butter	Coke
Butter	Jam
Bread and Butter	Milk
Bread and Butter	Jam



Advantages over Apriori algorithm

1.Memory Requirements: Since the ECLAT algorithm uses a Depth-First Search approach, it uses less memory than Apriori algorithm.

2.Speed: The ECLAT algorithm is typically faster than the Apriori algorithm.

3.Number of Computations: The ECLAT algorithm does not involve the repeated scanning of the data to compute the individual support values

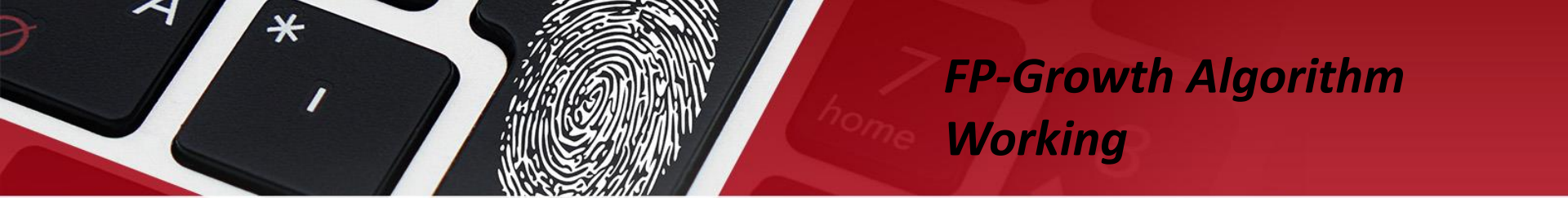


FP-Growth Algorithm

In Data Mining the task of finding frequent pattern in large databases is very important and has been studied in large scale in the past few years. Unfortunately, this task is computationally expensive, especially when a large number of patterns exist.

The FP-Growth Algorithm is an alternative way to find frequent itemsets without using candidate generations, thus improving performance. For so much it uses a divide-and-conquer strategy.

The core of this method is the usage of a special data structure named frequent-pattern tree (FP-tree), which retains the itemset association information.



FP-Growth Algorithm Working

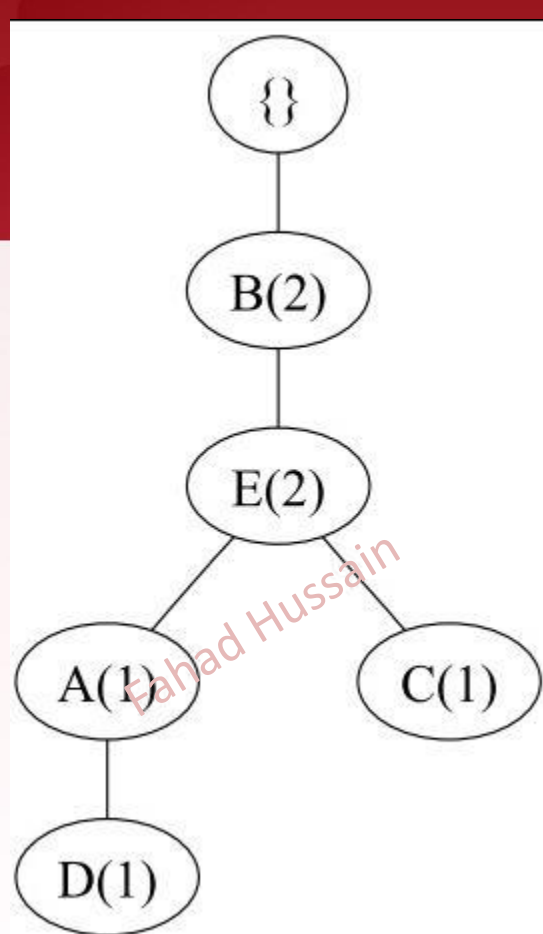
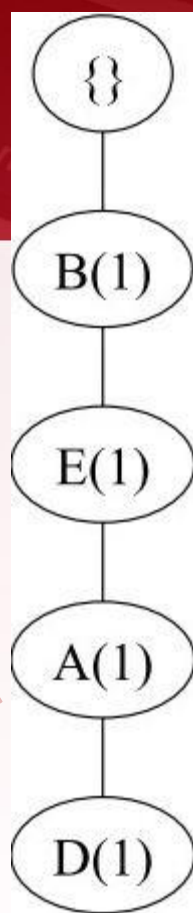
In simple words, this algorithm works as follows:

First it compresses the input database creating an FP-tree instance to represent frequent items. After this first step it divides the compressed database into a set of conditional databases, each one associated with one frequent pattern. Finally, each such database is mined separately. Using this strategy, the FP-Growth reduces the search costs looking for short patterns recursively and then concatenating them in the long frequent patterns, offering good selectivity.

In large databases, it's not possible to hold the FP-tree in the main memory. A strategy to cope with this problem is to firstly partition the database into a set of smaller databases (called projected databases), and then construct an FP-tree from each of these smaller databases.

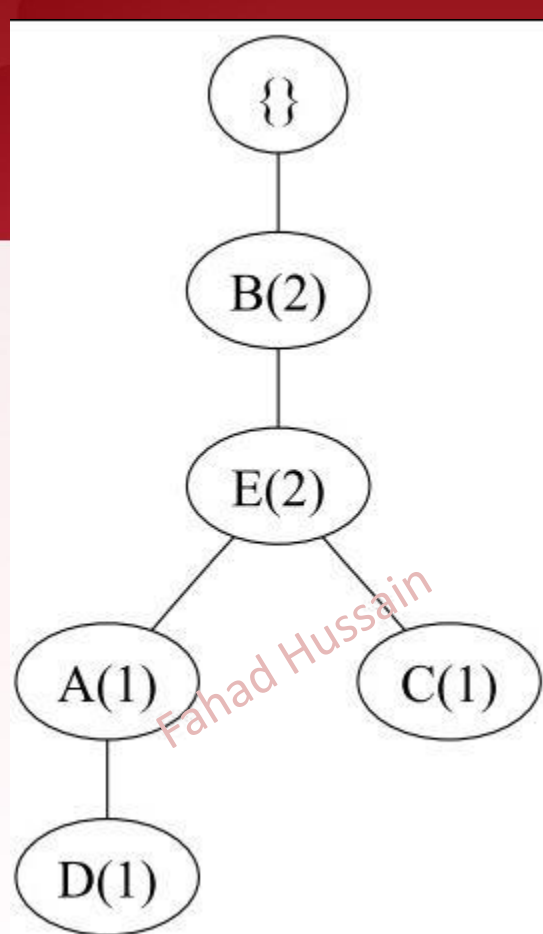
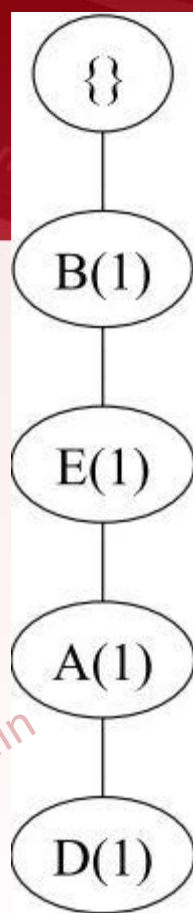
<i>Id</i>	<i>items</i>	ITEMS	SUPPORT
1	ABDE	A	4
2	BCE	B	6
3	ABDE	C	4
4	ABCE	D	4
5	ABCDE	E	5
6	BCD		

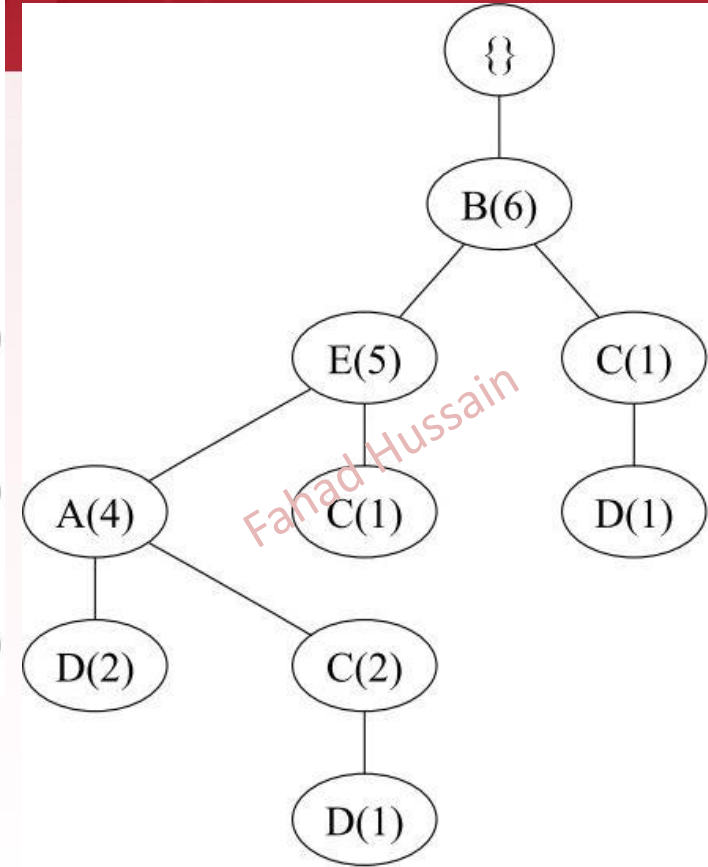
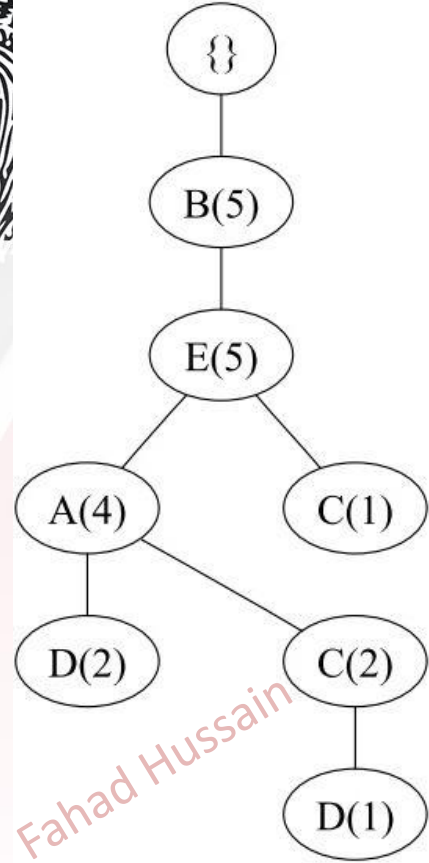
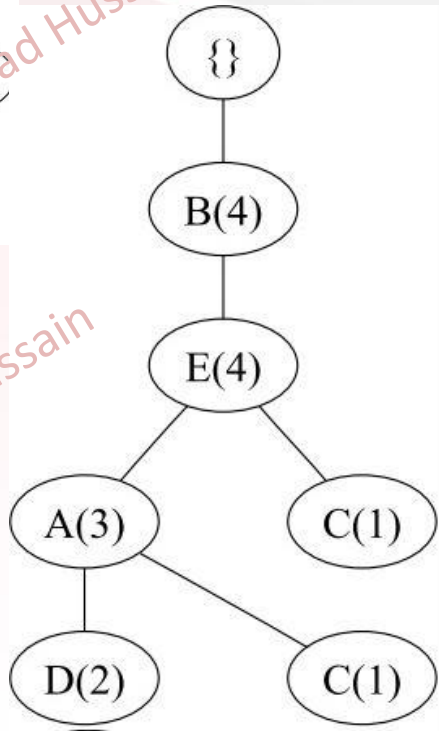
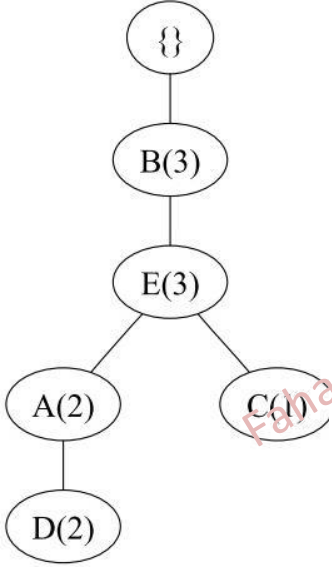
<i>Id</i>	<i>items</i>	<i>Id</i>	<i>items</i>
B	6	1	BEAD
E	5	2	BEC
A	4	3	BEAD
C	4	4	BEAC
D	4	5	BEACD
		6	BCD

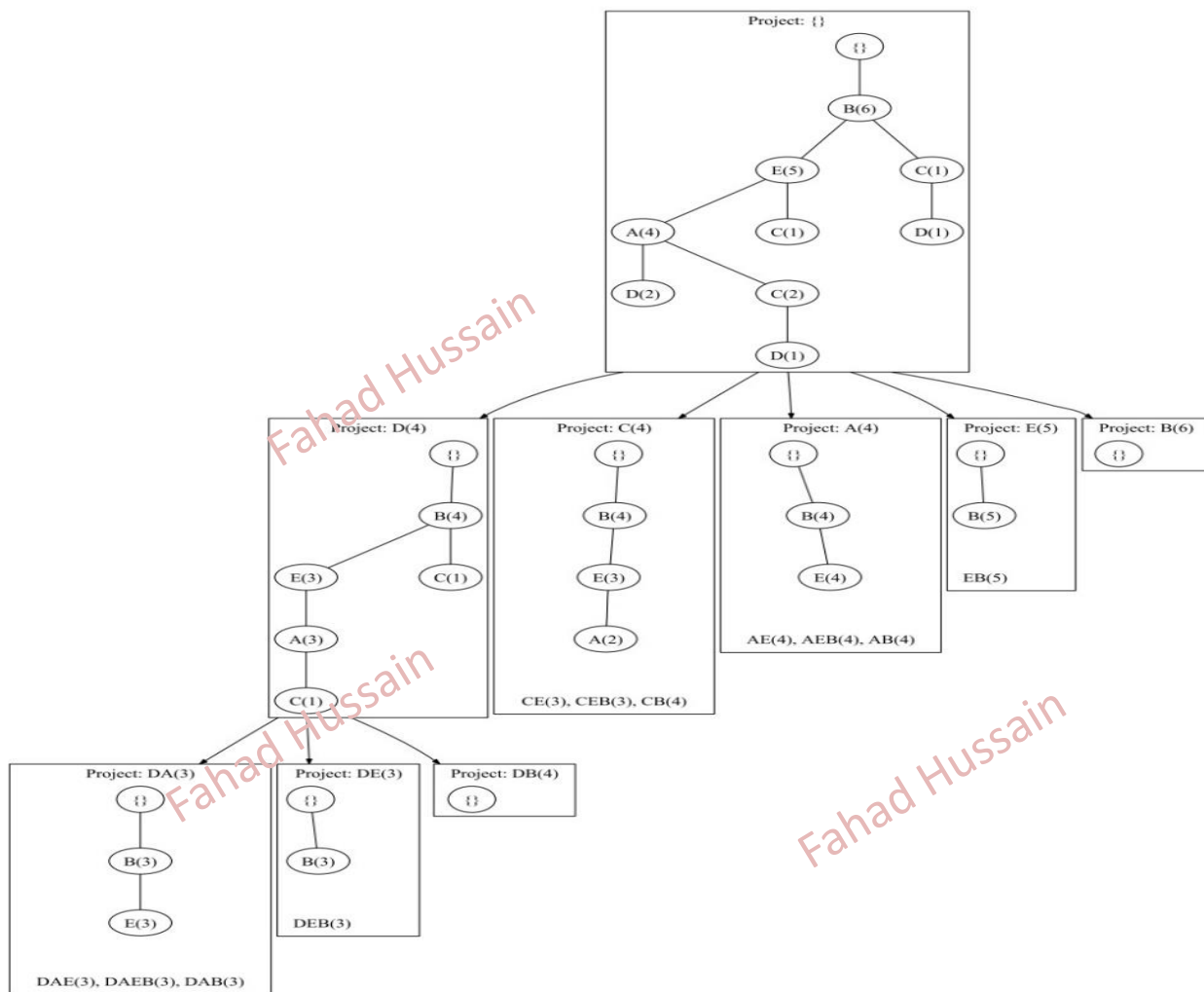


<i>Id</i>	<i>items</i>	ITEMS	SUPPORT
1	ABDE	A	4
2	BCE	B	6
3	ABDE	C	4
4	ABCE	D	4
5	ABCDE	E	5
6	BCD		

<i>Id</i>	<i>items</i>	<i>Id</i>	<i>items</i>
B	6	1	BEAD
E	5	2	BEC
A	4	3	BEAD
C	4	4	BEAC
D	4	5	BEACD
		6	BCD



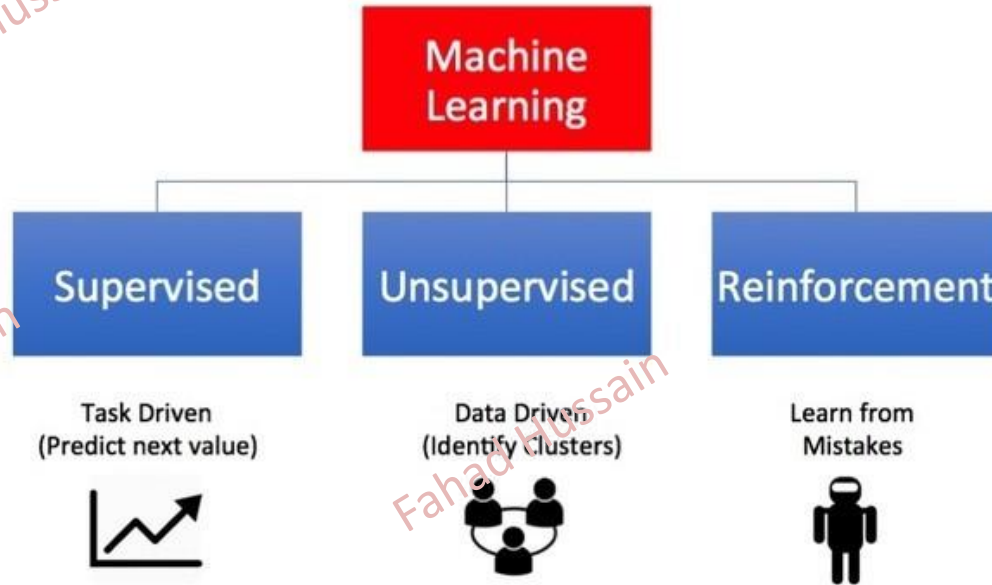




frequent patterns
founded by the
recursively calls to
FP-Growth Algorithm.

Reinforcement learning

Types of Machine Learning



Learn to make a good sequences of decision



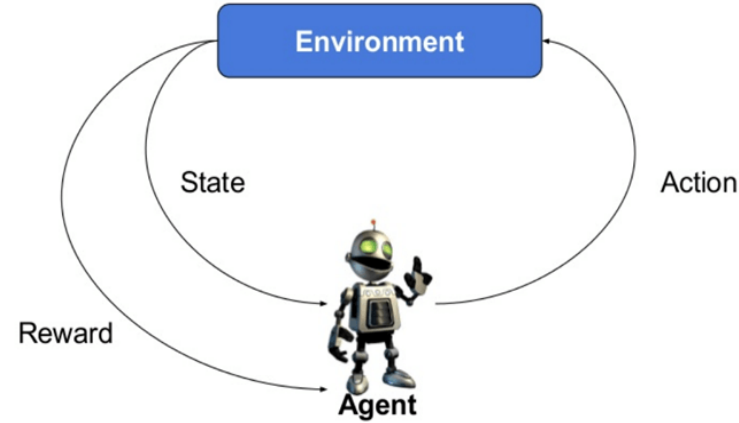
Reinforcement learning

Uncertainty is one of the big challenge in AI,
ML to make machine capable to take **good**
decision!!!



Reinforcement learning

Reinforcement learning is an area of machine learning concerned with how software **agents** ought to take actions in an **environment** so as to maximize some notion of cumulative **reward**. Reinforcement learning is one of three basic machine learning paradigms, alongside supervised learning and unsupervised learning.



Reinforcement learning

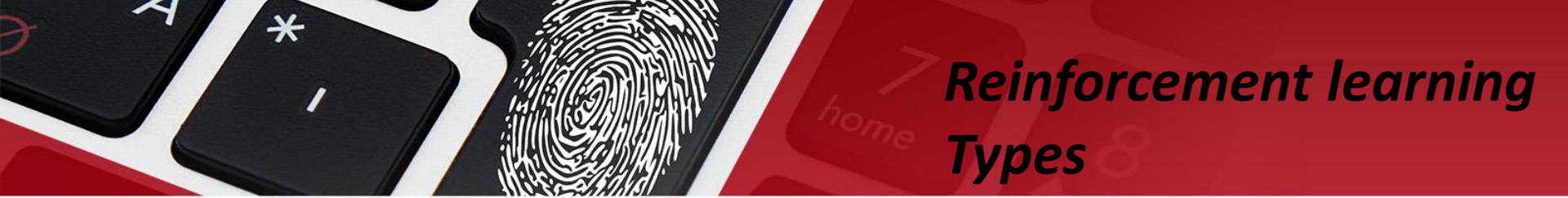
Environment: Physical world in which the agent operates

State: Current situation of the agent

Reward: Feedback from the environment

Policy: Method to map agent's state to actions

Value: Future reward that an agent would receive by taking an action in a particular state



Reinforcement learning Types

Positive –

Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words it has a positive effect on the behavior. Advantages of reinforcement learning are:

1. Maximizes Performance
2. Sustain Change for a long period of time

1. Disadvantages of reinforcement learning:

1. Too much Reinforcement can lead to overload of states which can diminish the results



Reinforcement learning Types with Usage

Negative –

Negative Reinforcement is defined as strengthening of a behavior because a negative condition is stopped or avoided. Advantages of reinforcement learning:

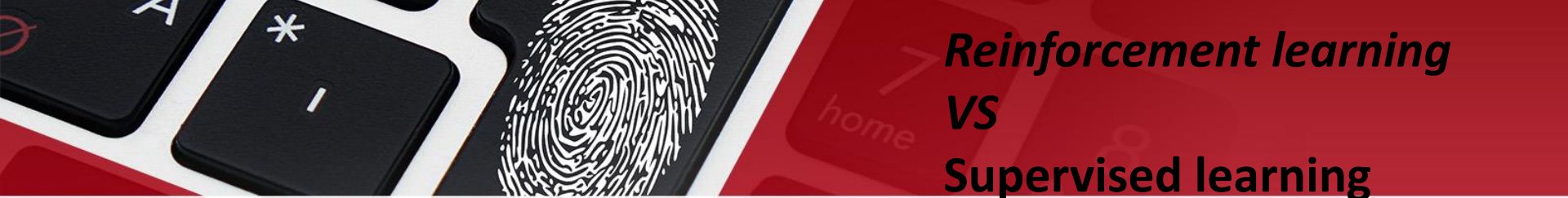
1. Increases Behavior
2. Provide defiance to minimum standard of performance

1. Disadvantages of reinforcement learning:

1. It Only provides enough to meet up the minimum behavior

Applications of Reinforcement Learning

- Robotics for industrial automation.
- Machine learning and data processing
- Create training systems that provide custom instruction and materials according to the requirement of students.



Reinforcement learning VS Supervised learning

Reinforcement learning is all about making decisions sequentially. In simple words we can say that the output depends on the state of the current input and the next input depends on the output of the previous input

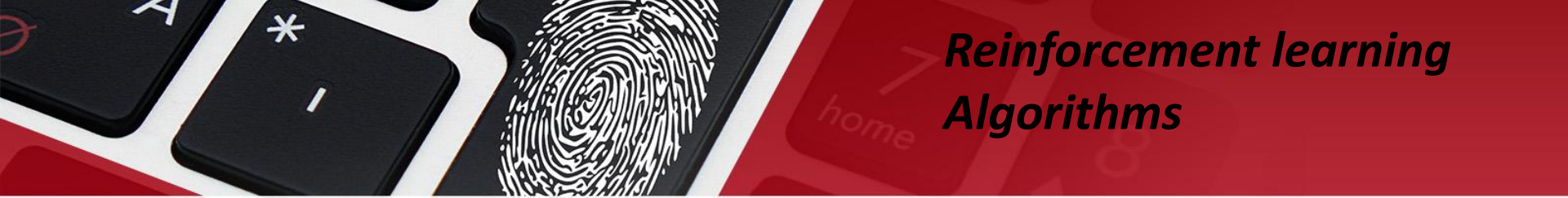
In Supervised learning the decision is made on the initial input or the input given at the start

In Reinforcement learning decision is dependent, So we give labels to sequences of dependent decisions

In Supervised learning the decisions are independent of each other so labels are given to each decision.

Example: Chess game

Example: Object recognition



Reinforcement learning Algorithms

- ❖ Markov Decision Process
- ❖ Q Learning Algorithm
- ❖ SARSA Reinforcement Learning
- ❖ Multi Armed Bandit Problem
- ❖ Thompson Sampling

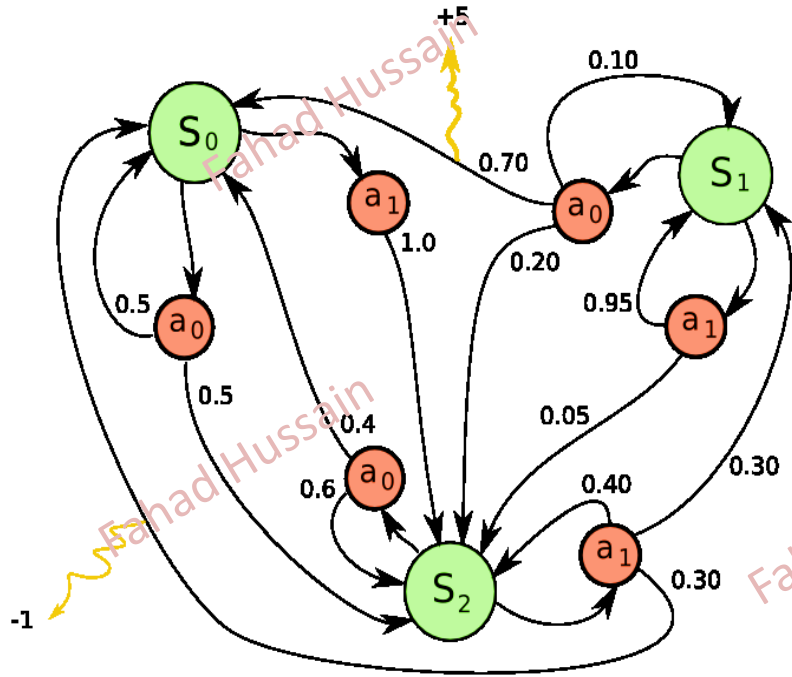


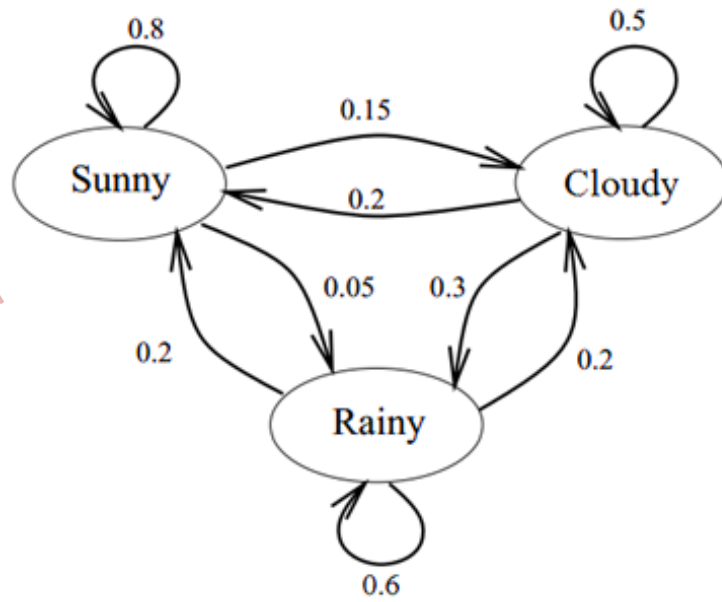
Markov Decision Process

Markov model is a stochastic model that assumes the Markov property. A stochastic model models a process where the state depends on previous states in a non-deterministic way. A stochastic process has the Markov property if the conditional probability distribution of future states of the process.

	System state is fully observable	System state is partially observable
System is autonomous	Markov chain	Hidden Markov model
System is controlled	Markov decision process	Partially observable Markov decision process

Working Example by Example





		Tomorrow's Weather		
Today's Weather		Sunny	Rainy	Cloudy
	Sunny	0.8	0.05	0.15
	Rainy	0.2	0.6	0.2
	Foggy	0.2	0.3	0.5

		Tomorrow's Weather		
Today's Weather		Sunny	Rainy	Cloudy
	Sunny	0.8	0.05	0.15
	Rainy	0.2	0.6	0.2
	Foggy	0.2	0.3	0.5

Today is sunny what's the probability that tomorrow is sunny and the day after is rainy
First we translates into

$$P(w_2 = \text{sunny}, w_3 = \text{rainy} | w_1 = \text{sunny})$$

$$\begin{aligned}
 P(w_2 = \text{sunny}, w_3 = \text{rainy} | w_1 = \text{sunny}) &= P(w_2 = \text{sunny} | w_1 = \text{sunny}) * \\
 &\quad P(w_3 = \text{rainy} | w_2 = \text{sunny}, w_3 = \text{sunny}) \\
 &= P(w_2 = \text{sunny} | w_1 = \text{sunny}) * \\
 &\quad P(w_3 = \text{rainy} | w_2 = \text{sunny}) \\
 &= 0.8 * 0.05 \\
 &= \mathbf{0.04}
 \end{aligned}$$

		Tomorrow's Weather		
Today's Weather		Sunny	Rainy	Cloudy
	Sunny	0.8	0.05	0.15
	Rainy	0.2	0.6	0.2
	Foggy	0.2	0.3	0.5

Today is sunny what's the probability that tomorrow is sunny and the day after is rainy
First we translates into

$$P(w_2 = \text{sunny}, w_3 = \text{rainy} | w_1 = \text{sunny})$$

$$\begin{aligned}
 P(w_2 = \text{sunny}, w_3 = \text{rainy} | w_1 = \text{sunny}) &= P(w_2 = \text{sunny} | w_1 = \text{sunny}) * \\
 &\quad P(w_3 = \text{rainy} | w_2 = \text{sunny}, w_3 = \text{sunny}) \\
 &= P(w_2 = \text{sunny} | w_1 = \text{sunny}) * \\
 &\quad P(w_3 = \text{rainy} | w_2 = \text{sunny}) \\
 &= 0.8 * 0.05 \\
 &= \mathbf{0.04}
 \end{aligned}$$



Hidden Markov model (HMM)

Hidden Markov Models (HMMs) are a class of probabilistic graphical **model** that allow us to predict a sequence of unknown (**hidden**) variables from a set of observed variables. A simple **example** of an **HMM** is predicting the weather (**hidden** variable) based on the type of clothes that someone wears (observed)

A hidden Markov model (HMM) is a statistical Markov model in which the system being modeled is assumed to be a Markov process with unobserved (hidden) states. A HMM can be considered the simplest dynamic Bayesian network. Hidden Markov models are especially known for their application in temporal pattern recognition such as speech, handwriting, gesture recognition, part-of-speech tagging, musical score following, partial discharges and bioinformatics.

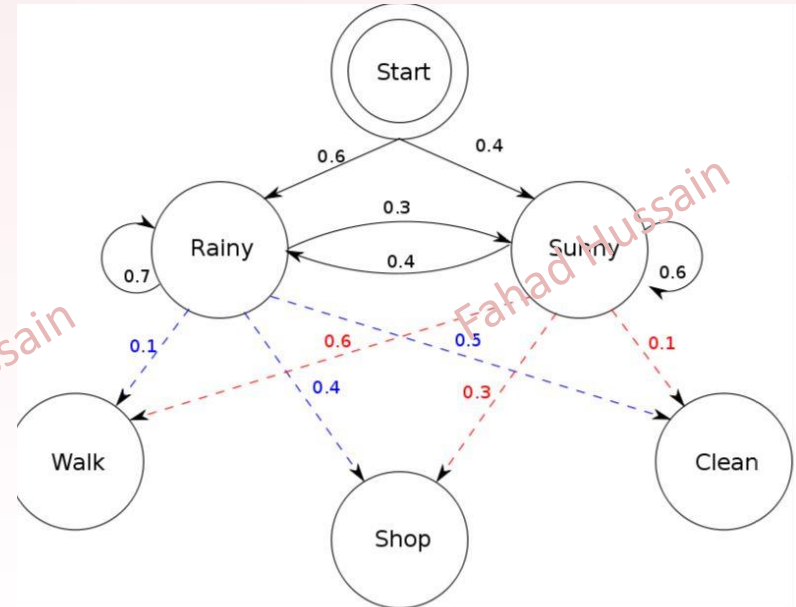
Hidden Markov model (HMM)

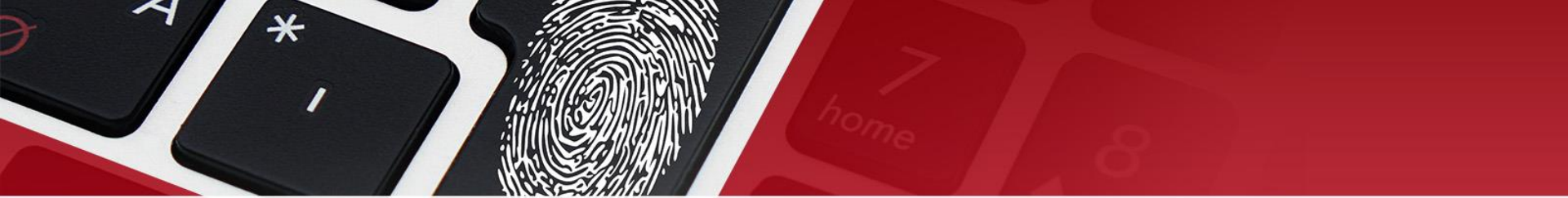
The term hidden refers to the first order Markov process behind the observation. Observation refers to the data we know and can observe. Markov process is shown by the interaction between “Rainy” and “Sunny” in the below diagram and each of these are **HIDDEN STATES**.

Initial Probabilities

Transition Probabilities

Emission Probabilities





Fahad Hussain

Markov Decision Process ***Bellman Equation*** ***Q Learning***

Fahad Hussain

Fahad Hussain

Fahad Hussain

Bellman Equation Q learning



Q Learning



Q Learning



Environment: Physical world in which the agent operates

State: Current situation of the agent

Reward: Feedback from the environment

Policy: Method to map agent's state to actions

Value: Future reward that an agent would receive by taking an action in a particular state

Markov Decision process

MDP provide a mathematical framework for modeling decision making in situation where outcomes are partly random and partly under the control of a decision maker.





Markov Decision process

Bellman Equation

A Bellman equation, named after Richard E. Bellman, is a necessary condition for optimality associated with the mathematical optimization method known as dynamic programming.

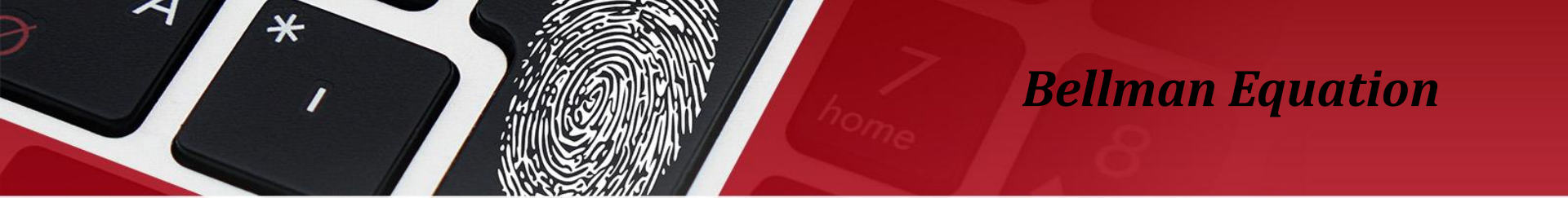
It writes the "value" of a decision problem at a certain point in time in terms of the payoff from some initial choices and the "value" of the remaining decision problem that results from those initial choices.[citation needed] This breaks a dynamic optimization problem into a sequence of simpler subproblems, as Bellman's "principle of optimality" prescribes!

$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$

Markov Decision Process Follow the Bellman

$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$





Bellman Equation

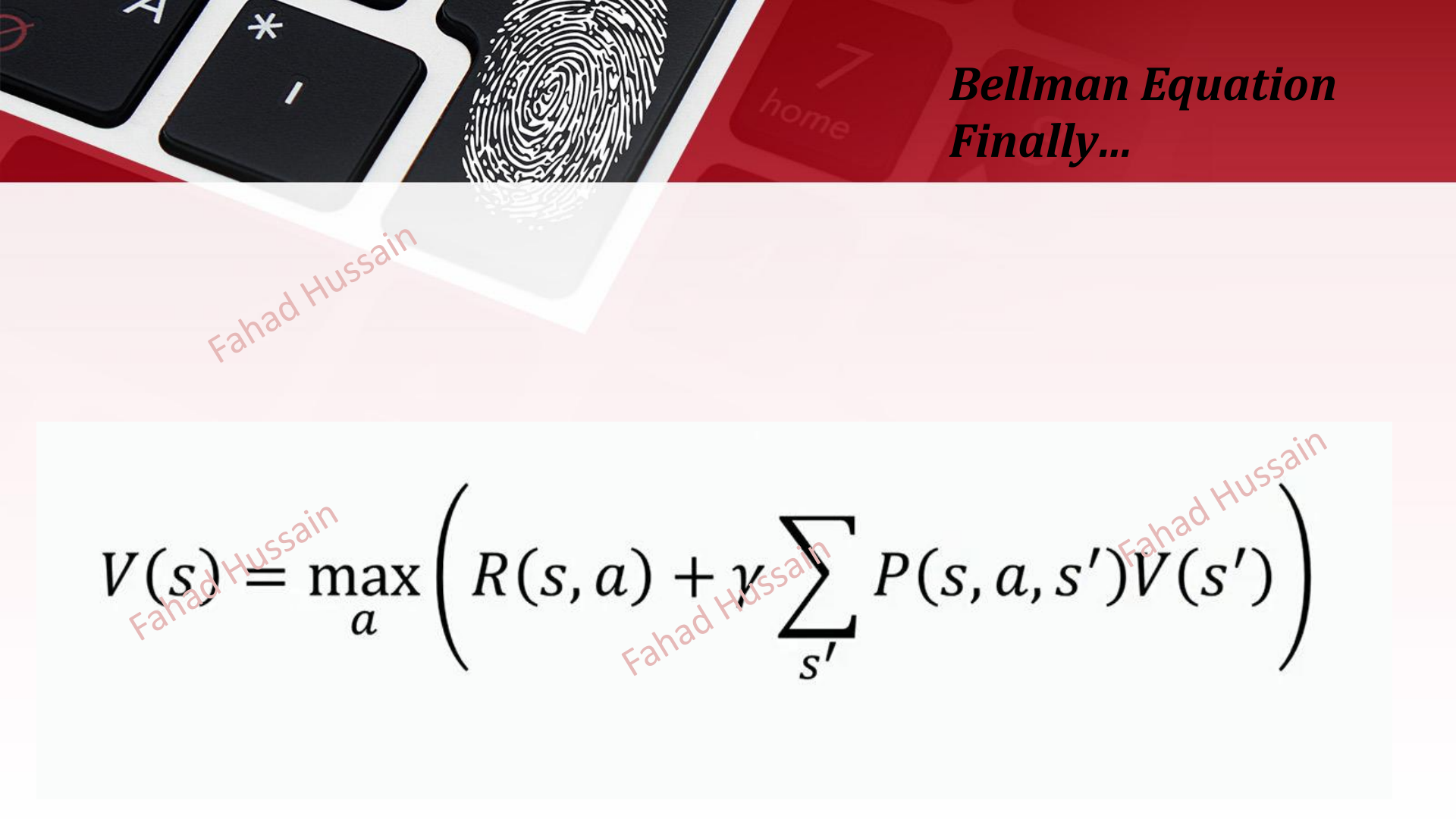
$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

Maximum of all possible action Reward acc. To state and action (gamma) Discount value to the next state

$$0.8 * V(s'_1) + 0.1 * V(s'_2) + 0.1 * V(s'_3)$$

$$V(s) = \max_a (R(s, a) + \underbrace{\gamma V(s')})$$

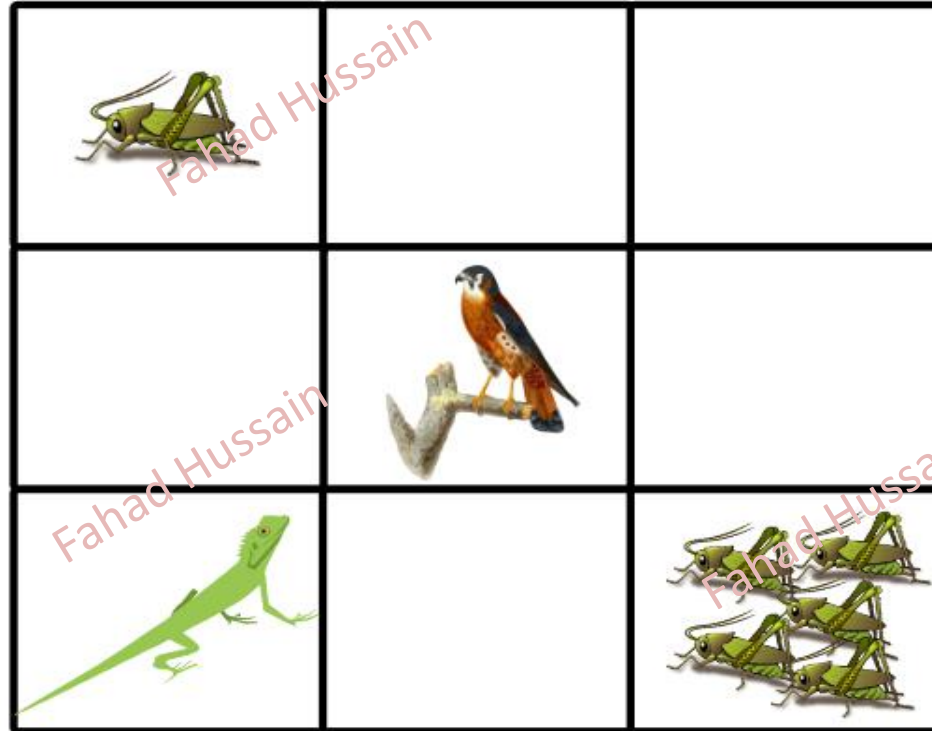
$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$



Bellman Equation Finally...

$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$

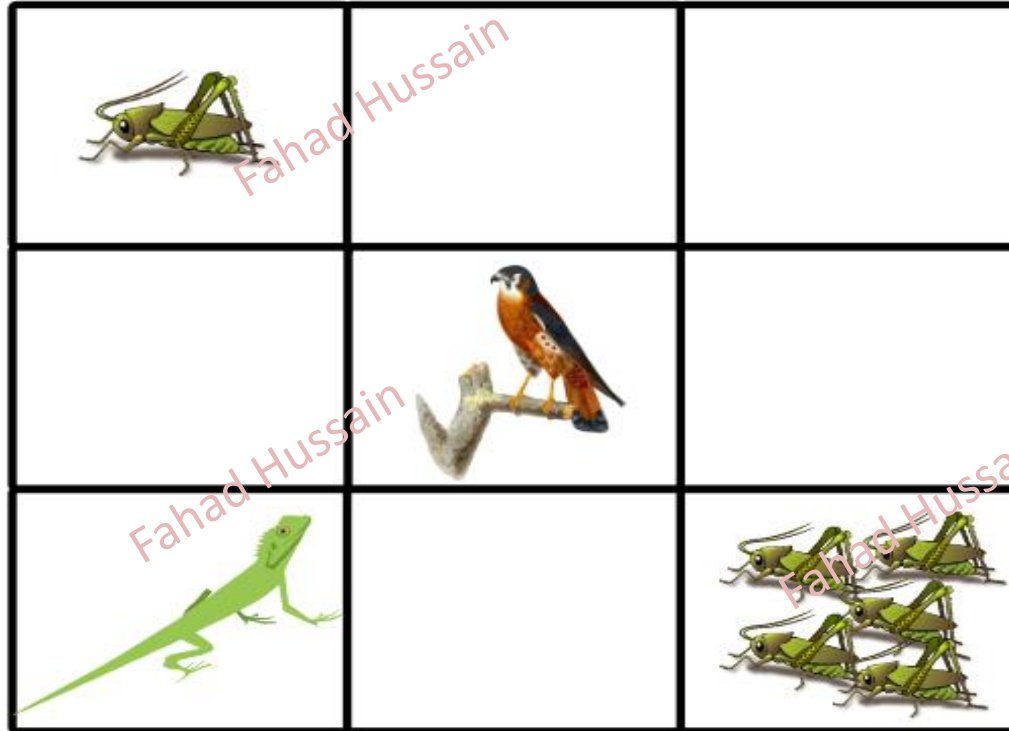
Bellman Equation Practical work



$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

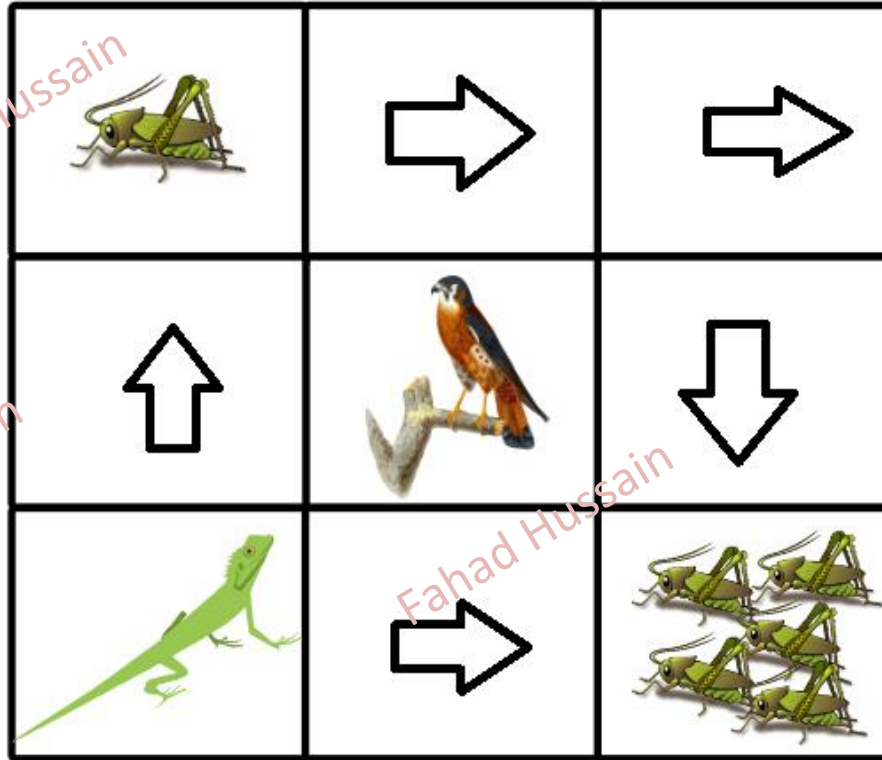
Bellman Equation

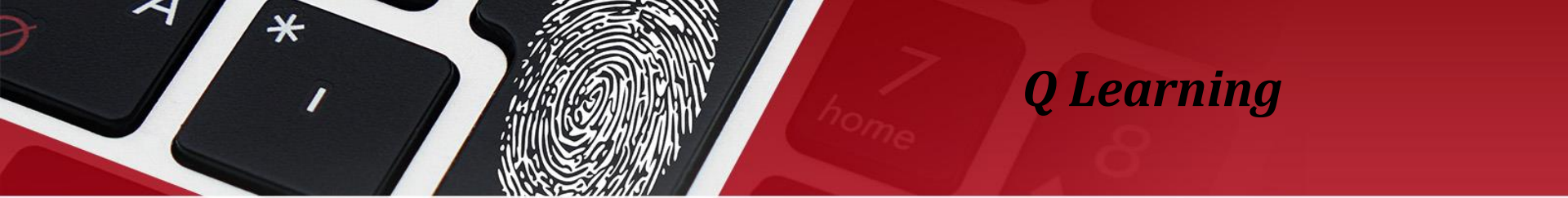
Practical work



$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

Bellman Equation Practical work





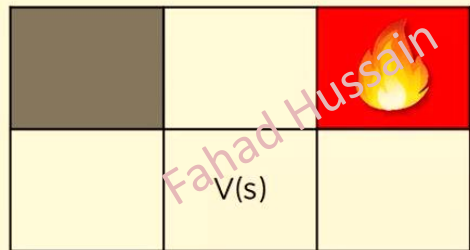
Q Learning

Markov decision processes give us a way to formalize sequential decision making.

It is the agent's goal to maximize the cumulative rewards.

Q-learning is one of the technique to find the optimal policy in an MDP. The objective of Q-learning is to find a policy that is optimal in the sense that the expected value of the total reward over all successive steps is the maximum achievable. So, in other words, the goal of Q-learning is to find the optimal policy by learning the optimal Q-values for each state-action pair.

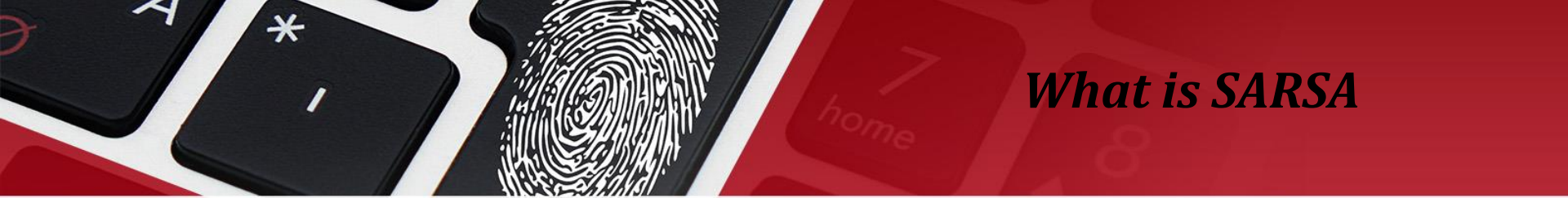
Q Learning



$$V(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s') \right)$$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} (P(s, a, s') V(s'))$$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} \left(P(s, a, s') \max_{a'} Q(s', a') \right)$$

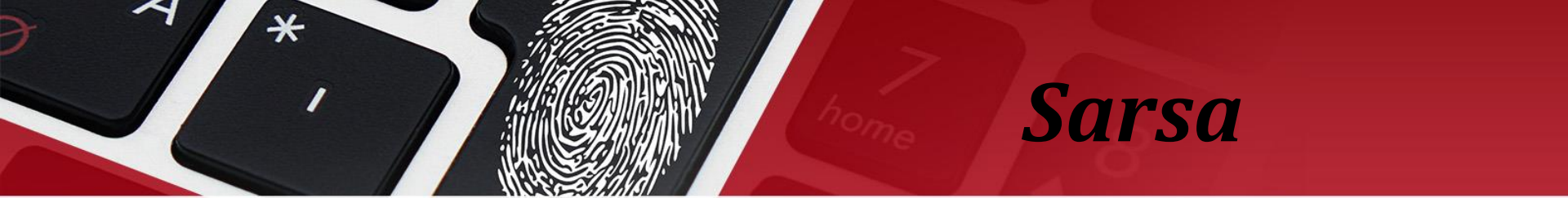


What is SARSA

State–action–reward–state–action (SARSA) is an algorithm for learning a Markov decision process policy, used in the reinforcement learning area of machine learning. It is a technical note with the name "Modified Connectionist Q-Learning" (MCQ-L). The alternative name SARSA, proposed by Rich Sutton, was only mentioned as a footnote.

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



Sarsa

SARSA algorithm is a slight variation of the popular Q-Learning algorithm. For a learning agent in any Reinforcement Learning algorithm it's policy can be of two types

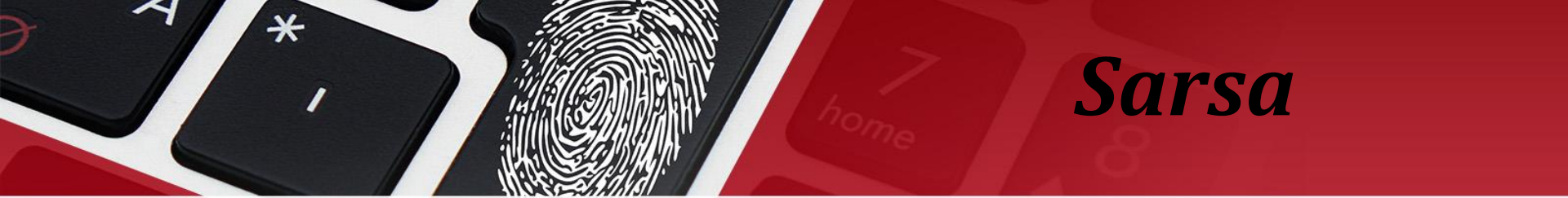
On Policy: In this, the learning agent learns the value function according to the current action derived from the policy currently being used.

Off Policy: In this, the learning agent learns the value function according to the action derived from another policy.

Q-Learning technique is an **Off Policy** technique and uses the greedy approach to learn the Q-value. SARSA technique, on the other hand, is an **On Policy** and uses the action performed by the current policy to learn the Q-value.

For Notes and Code:

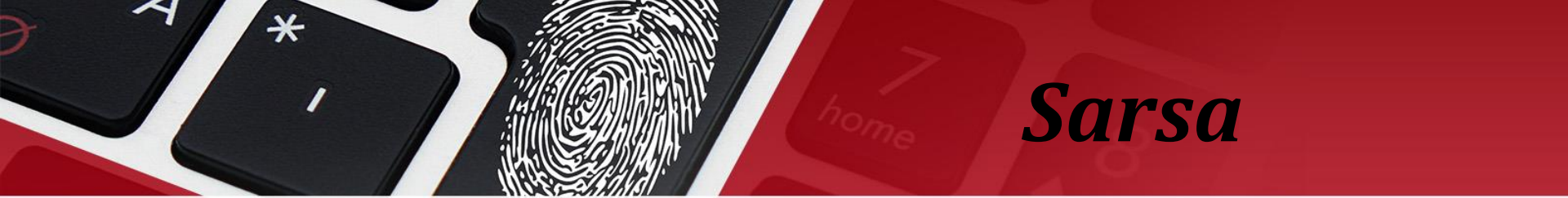
Visit: <http://fahadhussaincs.blogspot.com/>



Sarsa

A SARSA agent interacts with the environment and updates the policy based on actions taken, hence this is known as an on-policy learning algorithm. The Q value for a state-action is updated by an error, adjusted by the learning rate α . Q values represent the possible reward received in the next time step for taking action a in state s , plus the discounted future reward received from the next state-action observation.

Watkin's Q-learning updates an estimate of the optimal state-action value function Q^* based on the maximum reward of available actions. While SARSA learns the Q values associated with taking the policy it follows itself, Watkin's Q-learning learns the Q values associated with taking the optimal policy while following an exploration/exploitation policy.



Sarsa

Q-Learning:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

SARSA:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

The action $a_{(t+1)}$ is the action performed in the next state $s_{(t+1)}$ under current policy.

Here, the update equation for SARSA depends on the current state, current action, reward obtained, next state and next action. This observation lead to the naming of the learning technique as SARSA stands for **State Action Reward State Action** which symbolizes the tuple (s, a, r, s', a') .

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



Q Learning VS Sarsa

In Q-Learning, the agent learns optimal policy using absolute greedy policy and behaves using other policies such as ϵ -greedy policy. Because the update policy is different to the behavior policy, so Q-Learning is off-policy.

In SARSA, the agent learns optimal policy and behaves using the same policy such as ϵ -greedy policy. Because the update policy is the same to the behavior policy, so SARSA is on-policy.

on-policy	off-policy
Agent can pick actions	Agent can't pick actions
Most obvious setup :)	Learning with exploration, playing without exploration
Agent always follows his own policy	Learning from expert (expert is imperfect)
	Learning from sessions (recorded data)

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



Queries as Comments

Thank You Very Much

For Notes and Code:

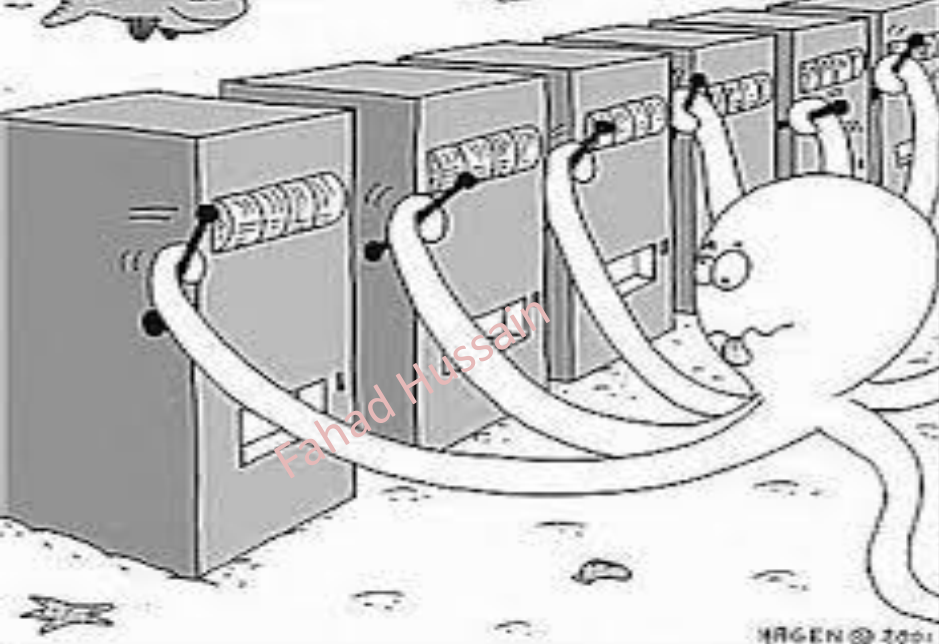
Visit: <http://fahadhussaincs.blogspot.com/>

Single Arm Bandit Problem



For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



Multi Arm Bandit Problem



1



2



3



4



5



Multi Arm Bandit Problem

In probability theory, the multi-armed bandit problem (sometimes called the K-[1] or N-armed bandit problem) is a problem in which a fixed limited set of resources must be allocated between competing (alternative) choices in a way that maximizes their expected gain, when each choice's properties are only partially known at the time of allocation, and may become better understood as time passes or by allocating resources to the choice.

This is a classic reinforcement learning problem that exemplifies the exploration-exploitation tradeoff dilemma. The name comes from imagining a gambler at a row of slot machines (sometimes known as "one-armed bandits"), who has to decide which machines to play, how many times to play each machine and in which order to play them, and whether to continue with the current machine or try a different machine. The multi-armed bandit problem also falls into the broad category of stochastic scheduling.

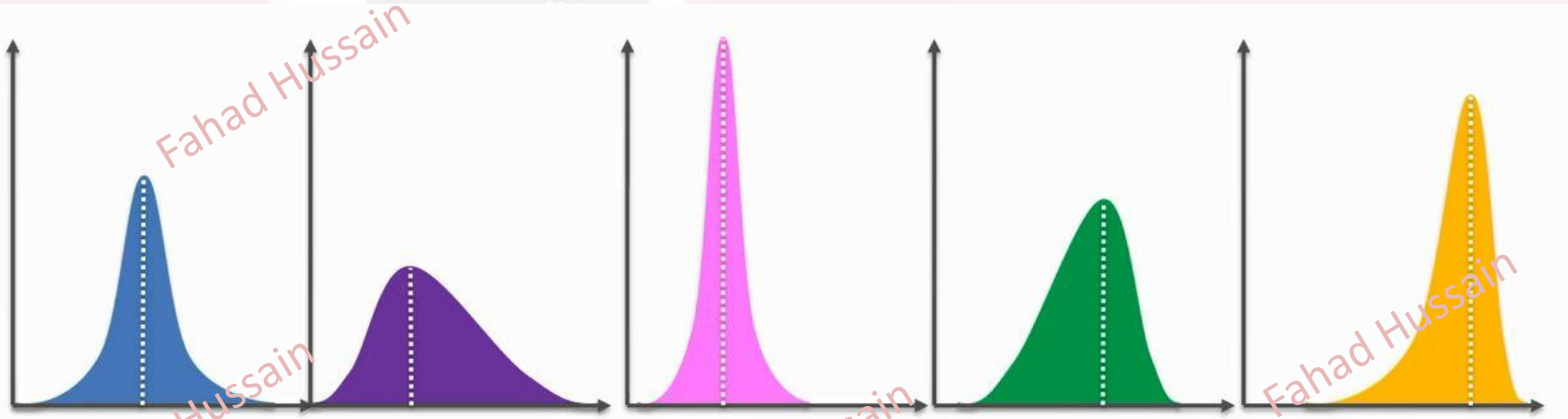


Multi Arm Bandit Problem

Exploration versus Exploitation

- **Exploration** - improve knowledge for long-term benefit
- **Exploitation** - exploit knowledge for short-term benefit

Multi Arm Bandit Problem



Application of Multi Arm Bandit Problem

There are many practical applications of the bandit model, for example:

1. Clinical trials investigating the effects of different experimental treatments while minimizing patient losses
2. Adaptive routing efforts for minimizing delays in a network,
3. Financial portfolio design
4. Ads



Ad1



Ad2



Ad3



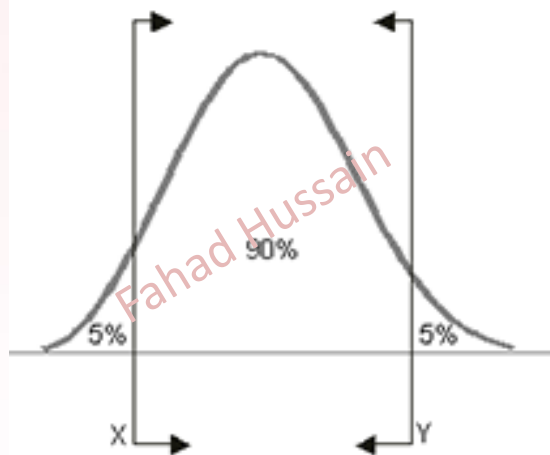
Ad4



Ad5

Upper Confidence Bond

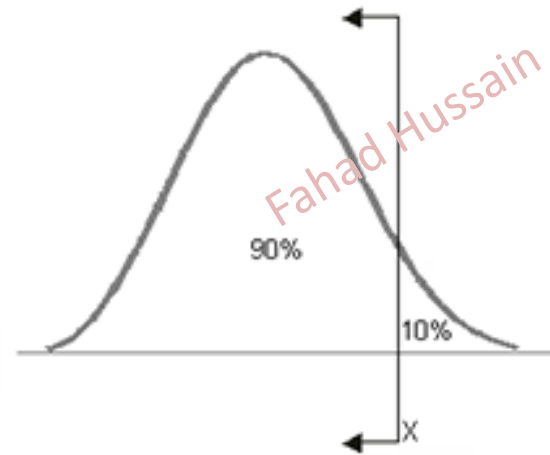
A one-sided **bound** defines the point where a certain percentage of the population is either higher or lower than the defined point. This means that there are two types of one-sided **bounds**: **upper** and lower. ... For example, if X is a **95% upper** one-sided **bound**, this would indicate that **95%** of the population is less than X



2-sided confidence bounds



Lower 1-sided confidence bounds



Upper 1-sided confidence bounds



Upper Confidence Bond

- We have d arms. For example, arms are ads that we display to users each time they connect to a web page.
- Each time a user connects to this web page, that makes a round.
- At each round n , we choose one ad to display to the user.
- At each round n , ad i gives reward $r_i(n) \in \{0, 1\}$: $r_i(n) = 1$ if the user clicked on the ad i , 0 if the user didn't.
- Our goal is to maximize the total reward we get over many rounds.



Upper Confidence Bond

Step 1. At each round n , we consider two numbers for each ad i :

- $N_i(n)$ - the number of times the ad i was selected up to round n ,
- $R_i(n)$ - the sum of rewards of the ad i up to round n .

Step 2. From these two numbers we compute:

- the average reward of ad i up to round n

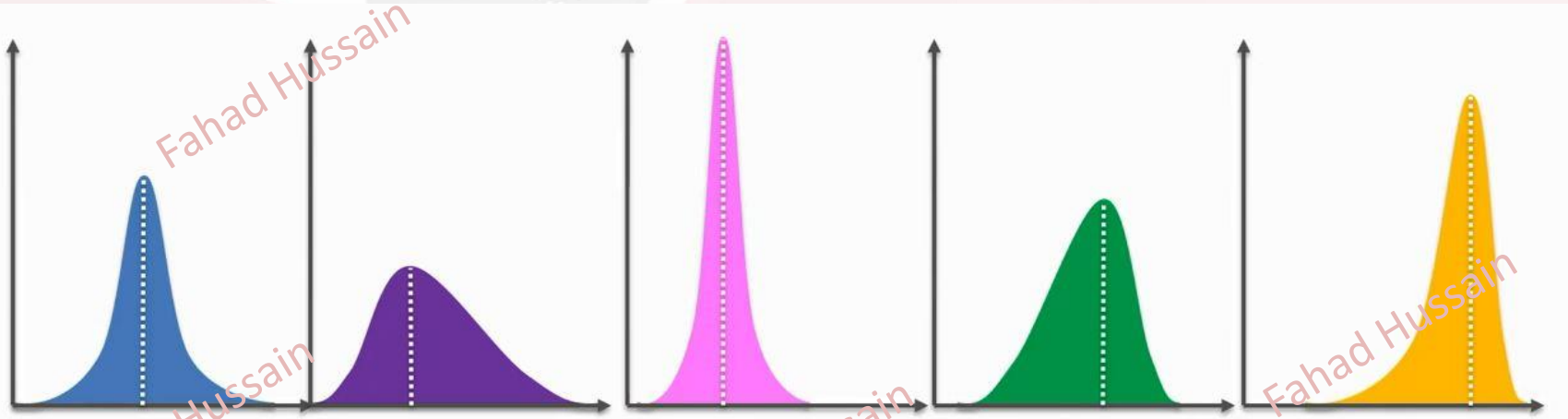
$$\bar{r}_i(n) = \frac{R_i(n)}{N_i(n)}$$

- the confidence interval $[\bar{r}_i(n) - \Delta_i(n), \bar{r}_i(n) + \Delta_i(n)]$ at round n with

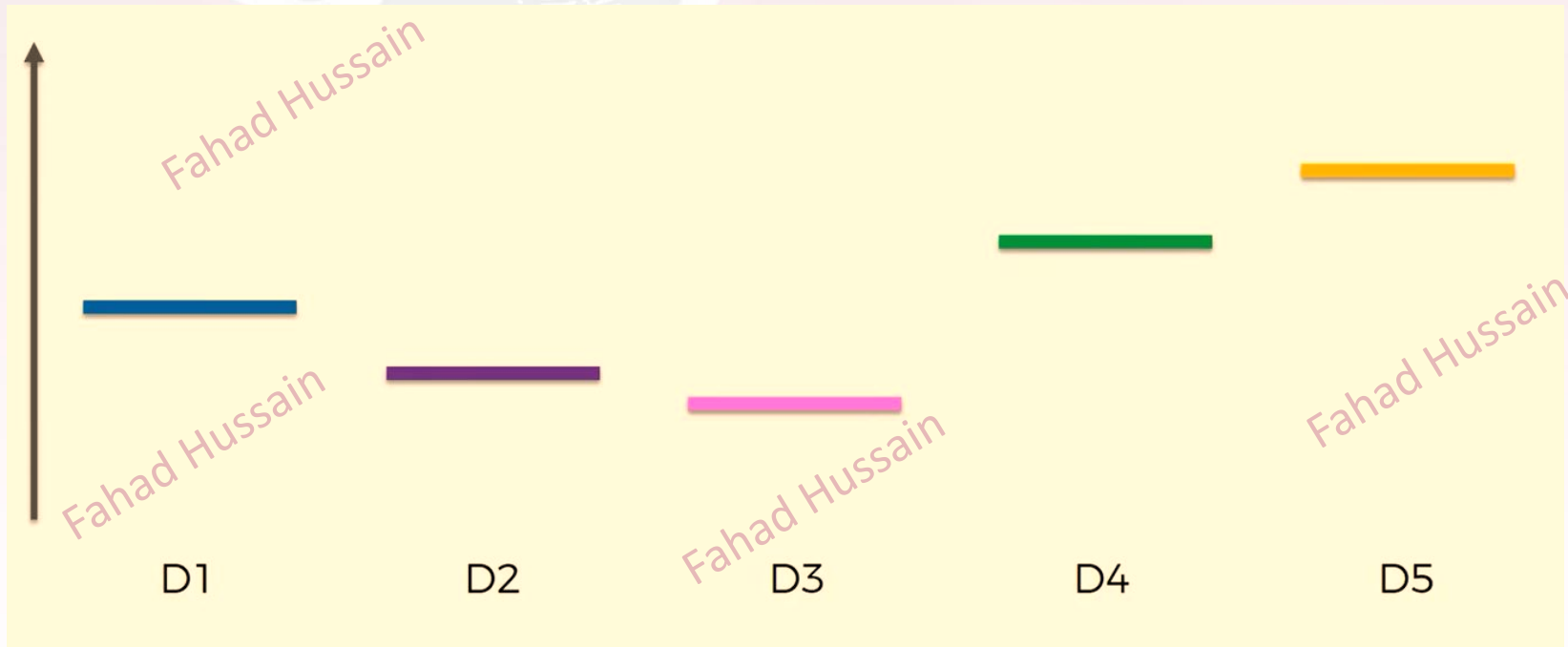
$$\Delta_i(n) = \sqrt{\frac{3 \log(n)}{2 N_i(n)}}$$

Step 3. We select the ad i that has the maximum UCB $\bar{r}_i(n) + \Delta_i(n)$.

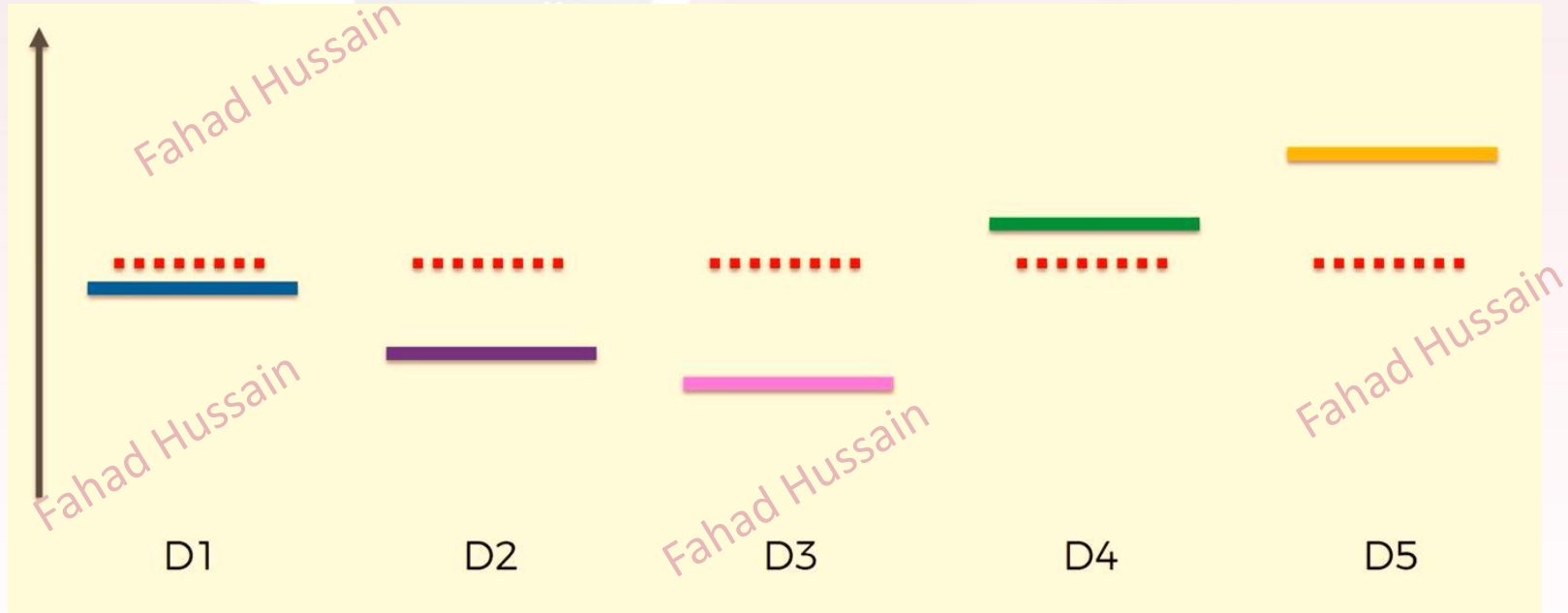
Multi Arm Bandit Problem



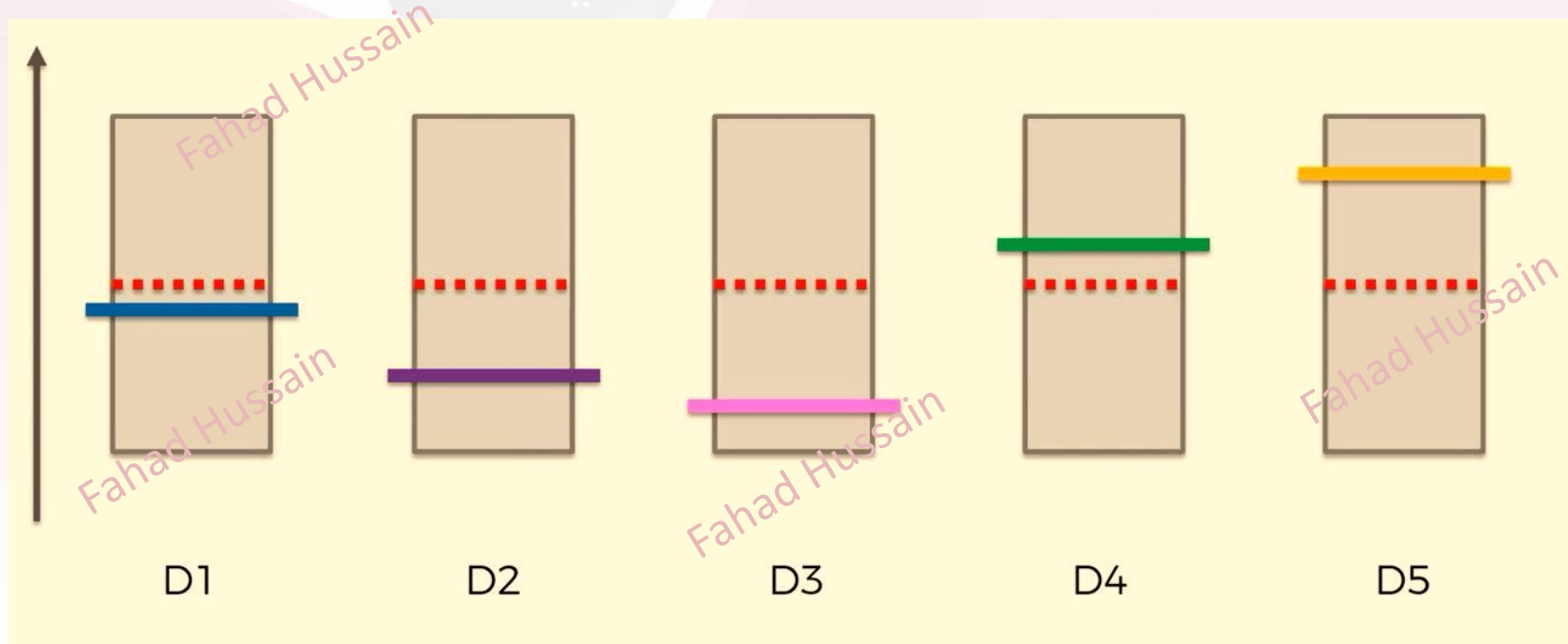
Upper Confidence Bond



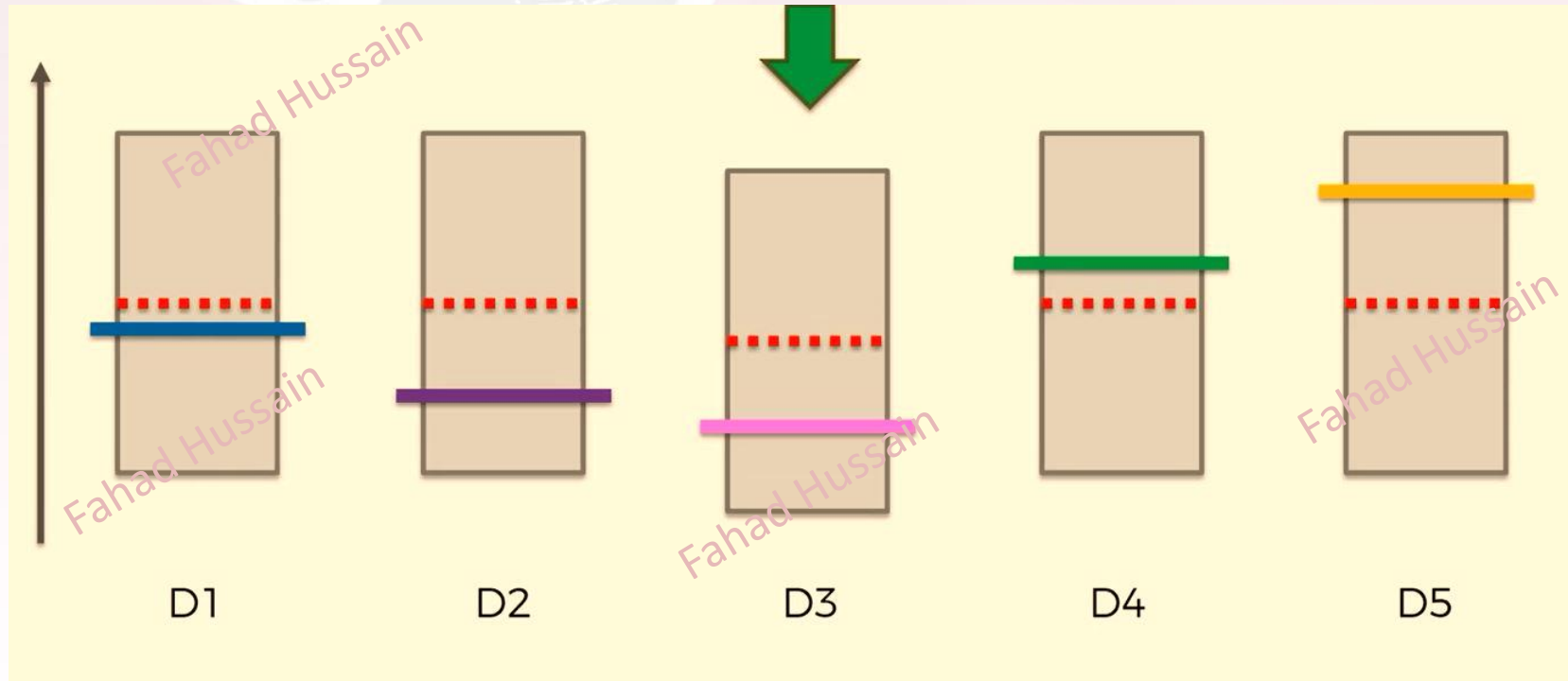
Upper Confidence Bond



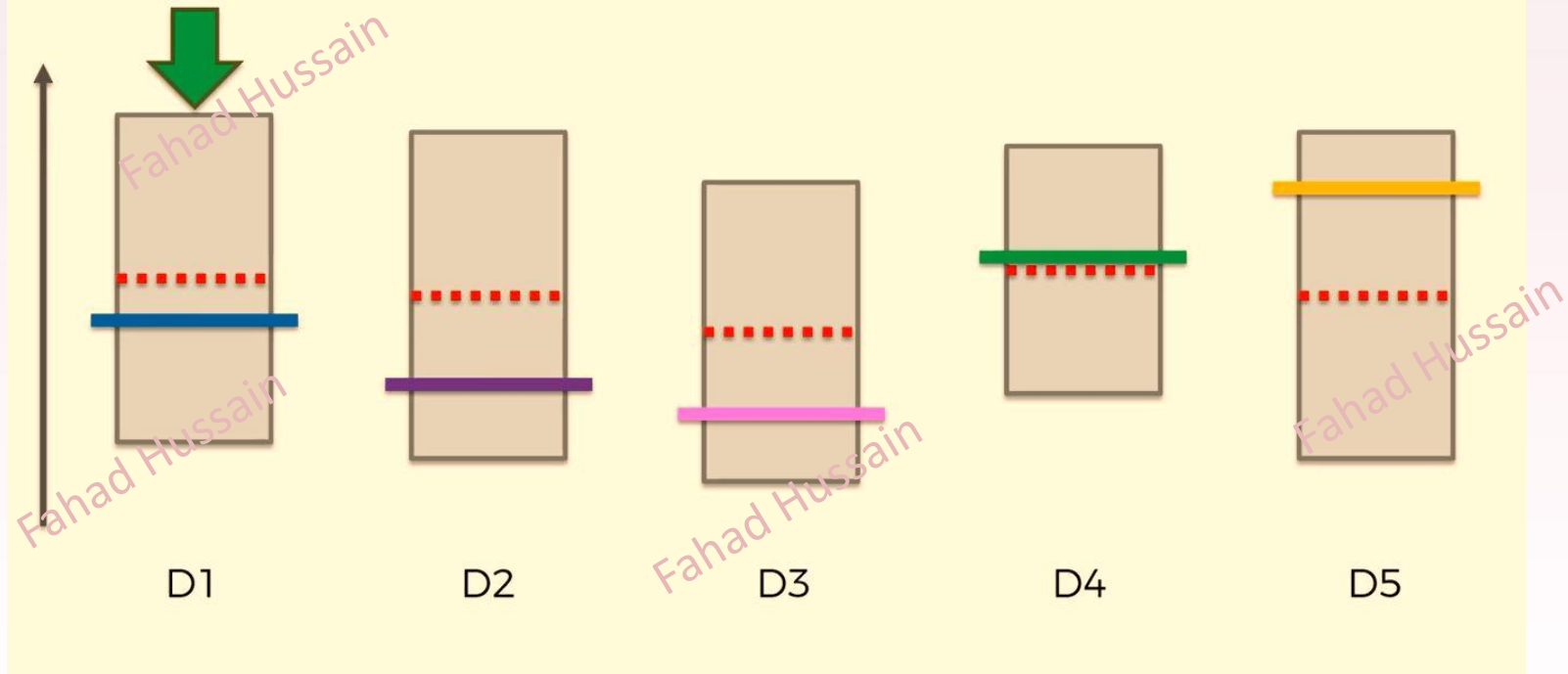
Upper Confidence Bond



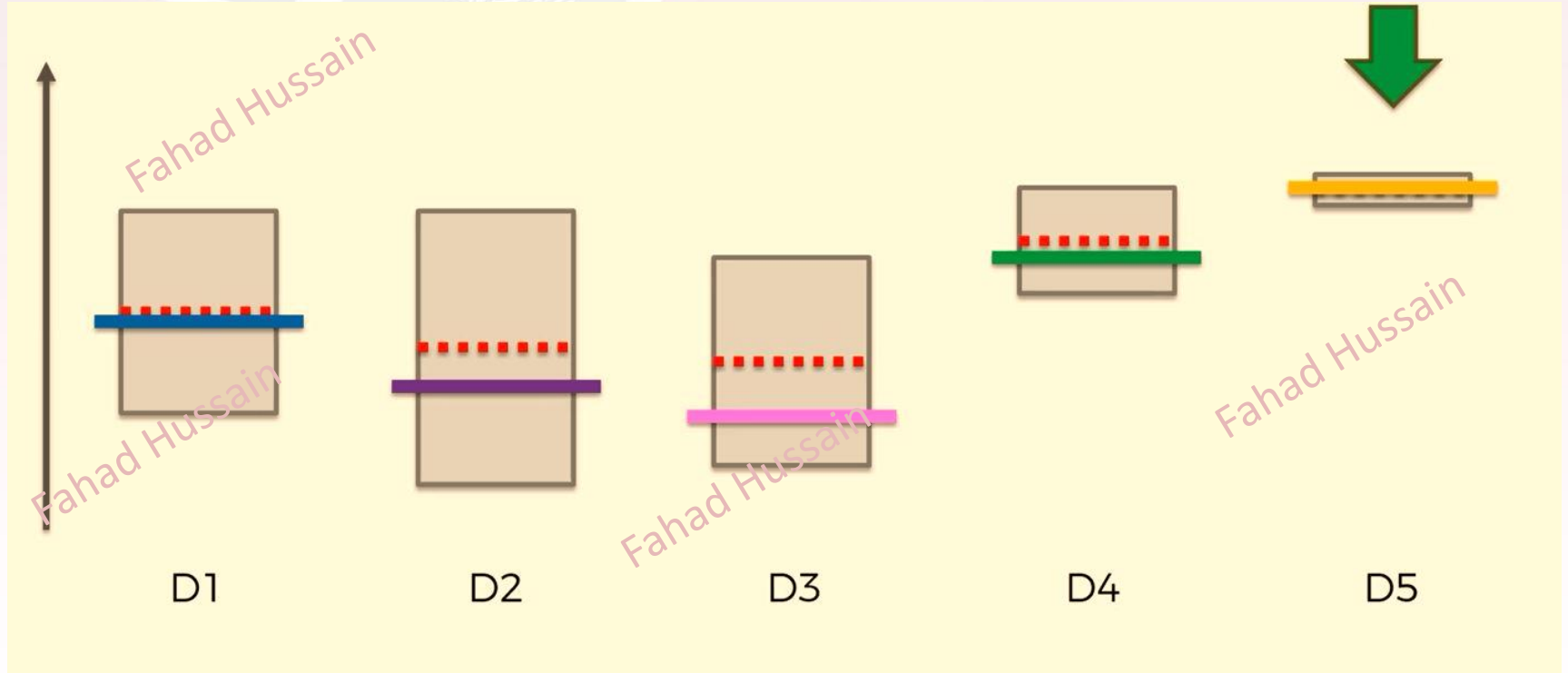
Upper Confidence Bond



Upper Confidence Bond



Upper Confidence Bond



Multi armed bandit Thompson sampling

Thompson sampling is a Bayesian approach to the **Multi-Armed Bandit** problem that dynamically balances incorporating more information to produce more certain predicted probabilities of each lever with the need to maximize current wins!



1



2



3



4



5



Multi armed bandit

Thompson sampling

- We have d arms. For example, arms are ads that we display to users each time they connect to a web page.
- Each time a user connects to this web page, that makes a round.
- At each round n , we choose one ad to display to the user.
- At each round n , ad i gives reward $r_i(n) \in \{0, 1\}$: $r_i(n) = 1$ if the user clicked on the ad i , 0 if the user didn't.
- Our goal is to maximize the total reward we get over many rounds.



Multi armed bandit Thompson sampling

- Ad i gets rewards $\mathbf{y}$ from Bernoulli distribution $p(\mathbf{y}|\theta_i) \sim \mathcal{B}(\theta_i)$.
- θ_i is unknown but we set its uncertainty by assuming it has a uniform distribution $p(\theta_i) \sim \mathcal{U}([0, 1])$, which is the prior distribution.
- Bayes Rule: we approach θ_i by the posterior distribution

$$\underbrace{p(\theta_i|\mathbf{y})}_{\text{posterior distribution}} = \frac{p(\mathbf{y}|\theta_i)p(\theta_i)}{\int p(\mathbf{y}|\theta_i)p(\theta_i)d\theta_i} \propto \underbrace{p(\mathbf{y}|\theta_i)}_{\text{likelihood function}} \times \underbrace{p(\theta_i)}_{\text{prior distribution}}$$

- We get $p(\theta_i|\mathbf{y}) \sim \beta(\text{number of successes} + 1, \text{number of failures} + 1)$
- At each round n we take a random draw $\theta_i(n)$ from this posterior distribution $p(\theta_i|\mathbf{y})$, for each ad i .
- At each round n we select the ad i that has the highest $\theta_i(n)$.



Multi armed bandit Thompson sampling

Step 1. At each round n , we consider two numbers for each ad i :

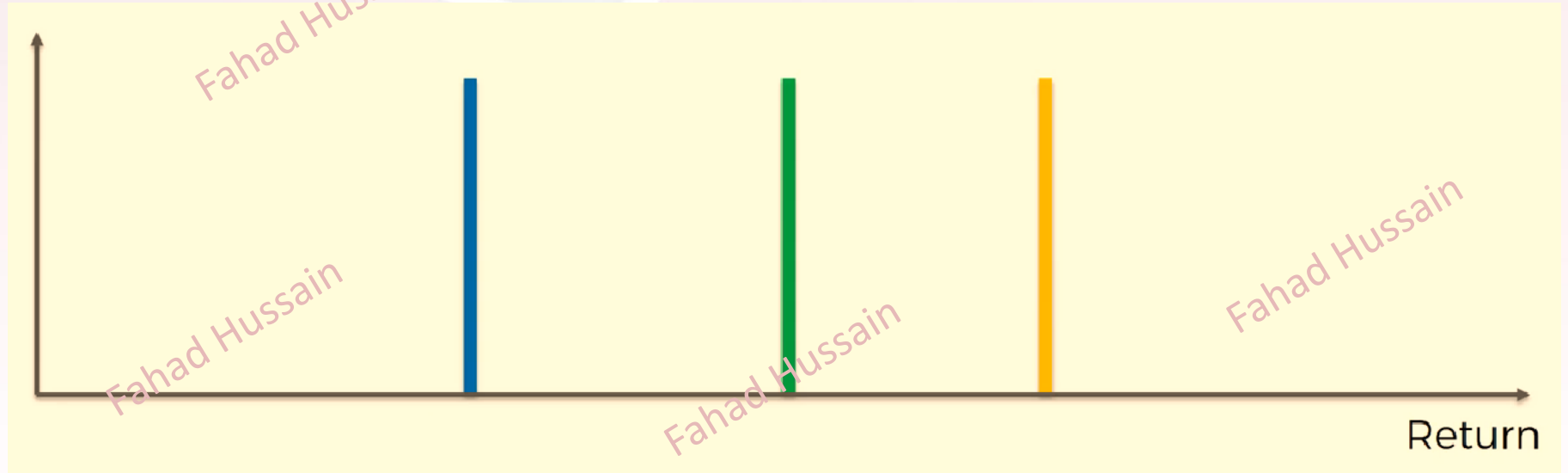
- $N_i^1(n)$ - the number of times the ad i got reward 1 up to round n ,
- $N_i^0(n)$ - the number of times the ad i got reward 0 up to round n .

Step 2. For each ad i , we take a random draw from the distribution below:

$$\theta_i(n) = \beta(N_i^1(n) + 1, N_i^0(n) + 1)$$

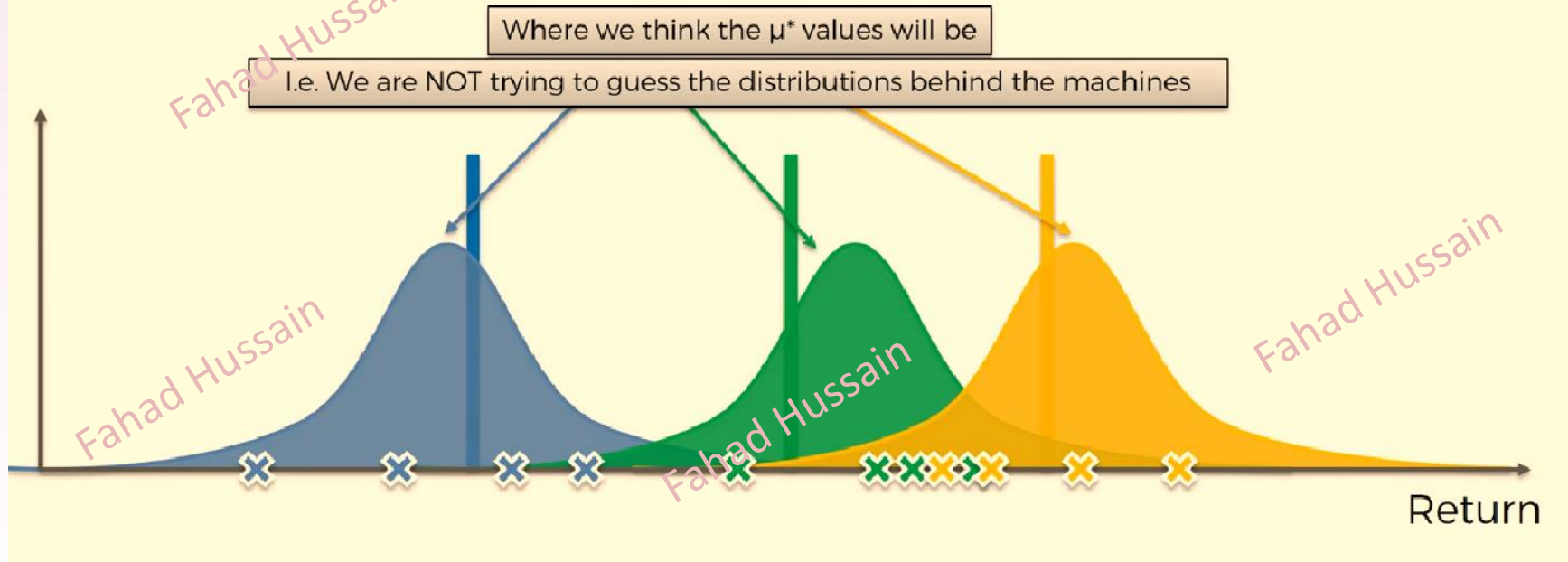
Step 3. We select the ad that has the highest $\theta_i(n)$.

Multi armed bandit Thompson sampling

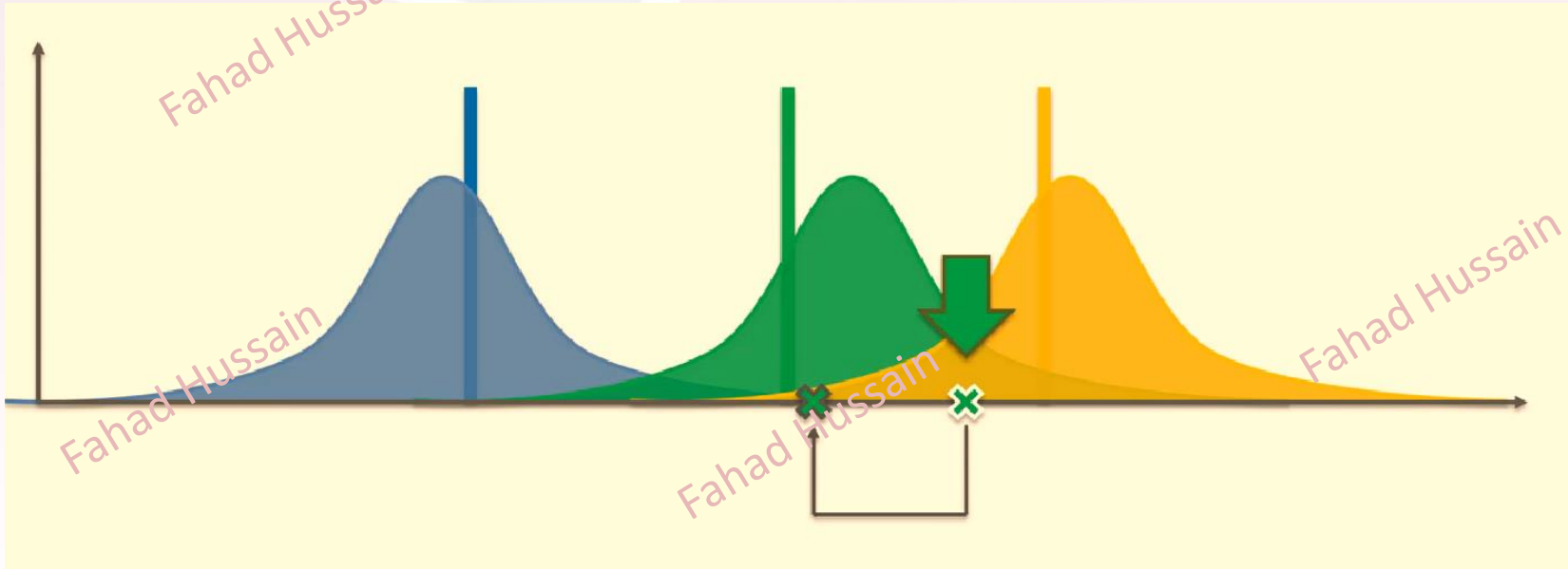


Multi armed bandit

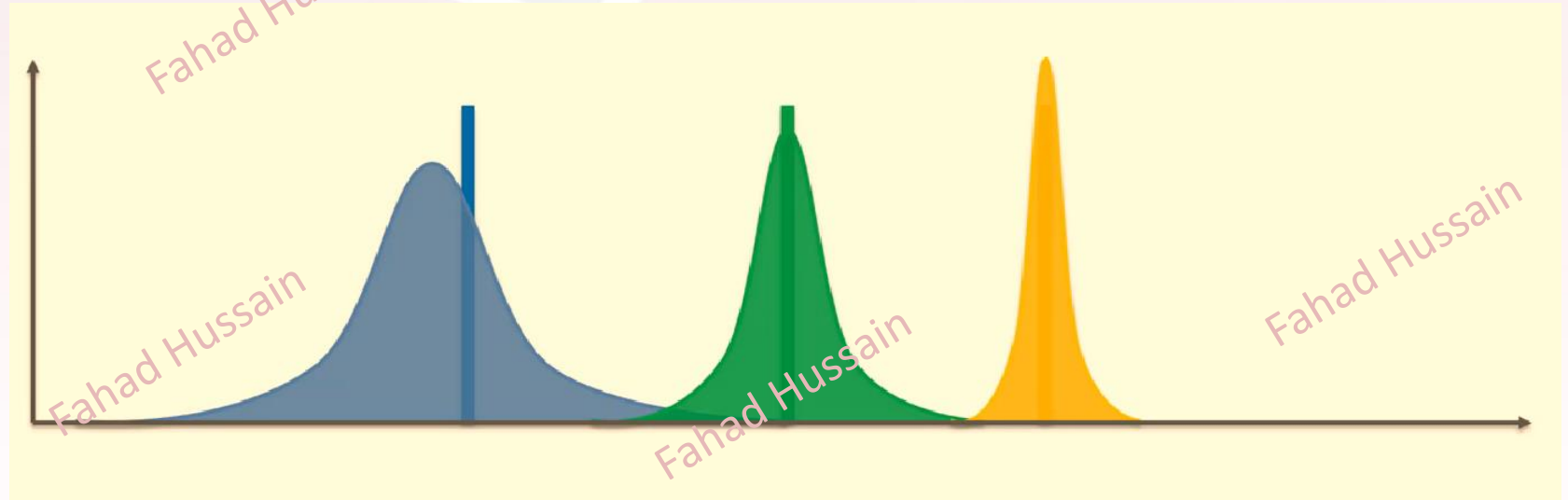
Thompson sampling



Multi armed bandit Thompson sampling



Multi armed bandit Thompson sampling





Dimensionality reduction

In machine learning classification problems, there are often too many factors on the basis of which the final classification is done. These factors are basically variables called features. The higher the number of features, the harder it gets to visualize the training set and then work on it. Sometimes, most of these features are correlated, and hence redundant. This is where dimensionality reduction algorithms come into play. Dimensionality reduction is the process of reducing the number of random variables under consideration, by obtaining a set of principal variables.

It can be divided into ***feature selection and feature extraction***.

What is Predictive Modeling: Predictive modeling is a probabilistic process that allows us to forecast outcomes, on the basis of some predictors. These predictors are basically features that come into play when deciding the final result, i.e. the outcome of the model.



Dimensionality reduction

Feature selection

1.Feature selection: In this, we try to find a subset of the original set of variables, or features, to get a smaller subset which can be used to model the problem. It usually involves three ways:

- 1.Filter (IG, Chi-Square Test, Correlation Coeff.)
- 2.Wrapper (Genetic Algorithm, Recursive Features elimination)
- 3.Embedded (Decision Tree)

$$a + b + c + d = e$$

$$ab = a + b$$

Feature Extraction

$$ab + c + d = e$$

$$c = 0$$

Feature Selection

$$ab + d = e$$



Feature extraction

Feature extraction: This reduces the data in a high dimensional space to a lower dimension space, i.e. a space with lesser no. of dimensions.

Methods of Dimensionality Reduction

The various methods used for dimensionality reduction include:

Principal Component Analysis (PCA)

Linear Discriminant Analysis (LDA)

Kernel Principal Component Analysis (KPCA)

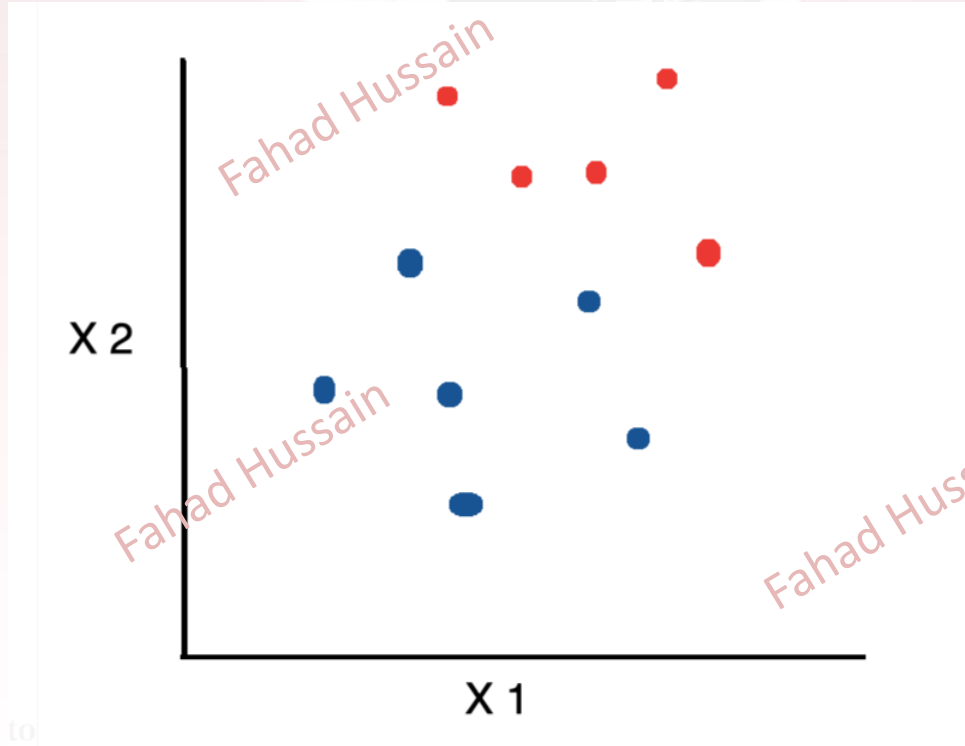


Principal Component Analysis (PCA)

From the m independent variables of your dataset, PCA extracts $p \leq m$ new independent variables that explain the most the variance of the dataset, regardless of the dependent variable.

➡ The fact that the DV is not considered makes PCA an unsupervised model.

Principal Component Analysis (PCA) Example





Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis or **Normal Discriminant Analysis** or **Discriminant Function Analysis** is a dimensionality reduction technique which is commonly used for the classification problems

For example, we have two classes and we need to separate them efficiently. Classes can have multiple features. Using only a single feature to classify them may result in some overlapping. So, we will keep on increasing the number of features for proper classification

Linear Discriminant Analysis (LDA), is a supervised Learning technique!



Linear Discriminant Analysis (LDA)

From the n independent variables of your dataset, LDA extracts $p \leq n$ new independent variables that separate the most the classes of the dependent variable.



The fact that the DV is considered makes LDA a supervised model.



Linear Discriminant Analysis (LDA)

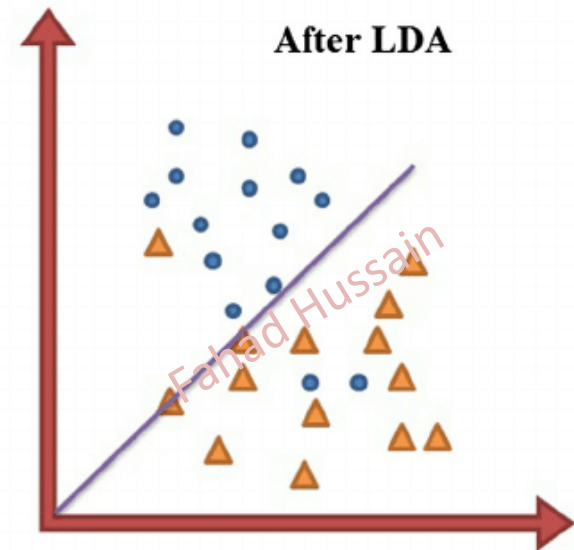
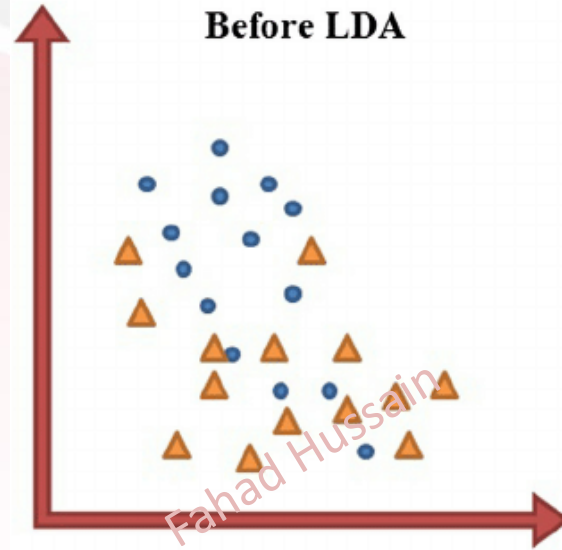
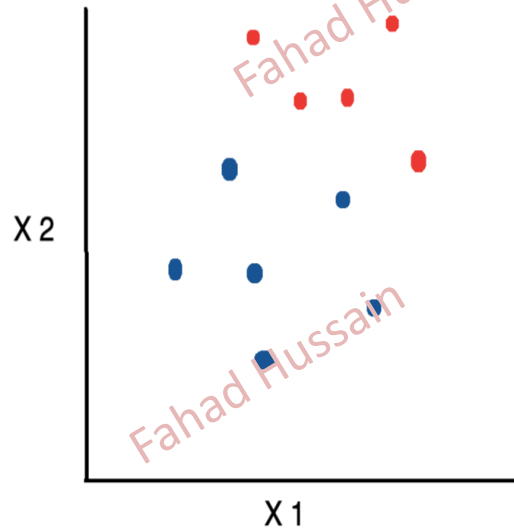
LDA can be achieved in three steps :

The first step is to calculate the separability between different classes (i.e the distance between the mean of different classes) also called as between-class variance

Second Step is to calculate the distance between the mean and sample of each class, which is called the within class variance

The third step is to construct the lower dimensional space which maximizes between class variance and minimizes the within class variance

Linear Discriminant Analysis (LDA)



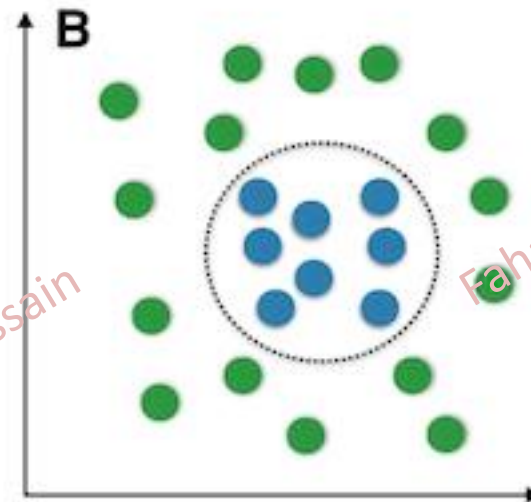
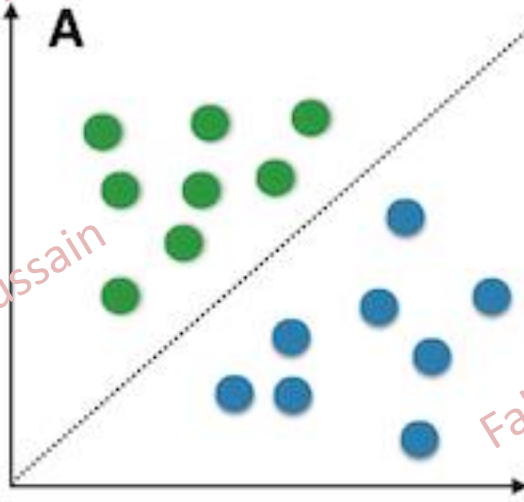


Kernel Principal Component Analysis (KPCA)

That is it can only be applied to datasets which are linearly separable. It does an excellent job for datasets, which are linearly separable. But, if we use it to non-linear datasets, we might get a result which may not be the optimal dimensionality reduction. Kernel PCA uses a kernel function to project dataset into a higher dimensional feature space, where it is linearly separable. It is similar to the idea of Support Vector Machines.

Kernel Principal Component Analysis (KPCA)

Linear vs. nonlinear problems





Queries as Comments

Thank You Very Much

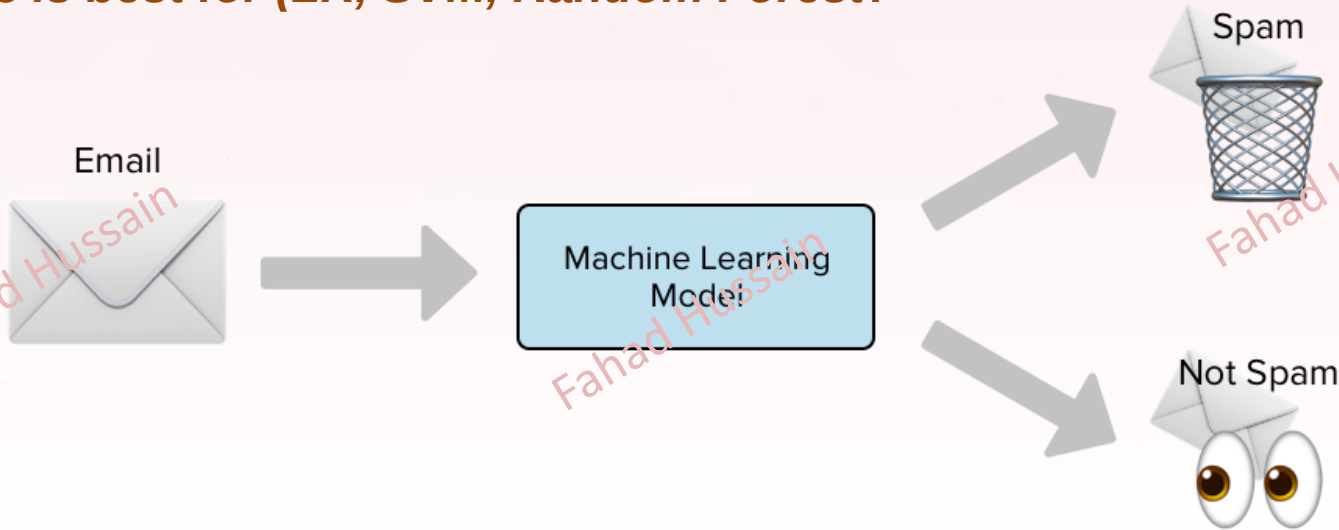
For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>

Cross Validation Session Part 1

For Model Selection, Improving Model Working!!!

*Model Training and Testing,
which one is best for (LR, SVM, Random Forest?)*





Cross Validation

Cross-validation, sometimes called rotation estimation or out-of-sample testing, is any of various similar model validation techniques for assessing how the results of a statistical analysis will generalize to an independent data set. It is mainly used in settings where the goal is prediction, and one wants to estimate how accurately a predictive model will perform in practice.

E.g:

Sample 100, Training 70, 80 Test: 30, 20

`Random_State(0, 50, 100)`

random_state as the name suggests, is used for initializing the internal random number generator, which will decide the splitting of data into train and test indices in your case



Types of Cross Validation

- **Bootstrap Method**
- **Holdout method.**
- **Leave-one-out cross-validation and (P-out)**
- **K-fold cross-validation.**
- **Stratified cross-validation**
- **Time series cross-validation**

• Bootstrap Method

The bootstrap method is a statistical technique for estimating quantities about a population by averaging estimates from multiple small data samples. Importantly, samples are constructed by drawing observations from a large data sample one at a time and returning them to the data sample after they have been chosen. This allows a given observation to be included in a given small sample more than once. This approach to sampling is called sampling with replacement.

Original Dataset

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
-------	-------	-------	-------	-------	-------	-------	-------	-------	----------

Bootstrap 1

x_8	x_6	x_2	x_9	x_5	x_8	x_1	x_4	x_9	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Bootstrap 2

x_{10}	x_1	x_3	x_5	x_1	x_7	x_4	x_2	x_1	x_8
----------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Bootstrap 3

x_6	x_5	x_4	x_1	x_2	x_4	x_2	x_6	x_9	x_2
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

Training Sets

x_3	x_7	x_{10}
-------	-------	----------

x_6	x_9
-------	-------

x_3	x_7	x_8	x_{10}
-------	-------	-------	----------

Test Sets

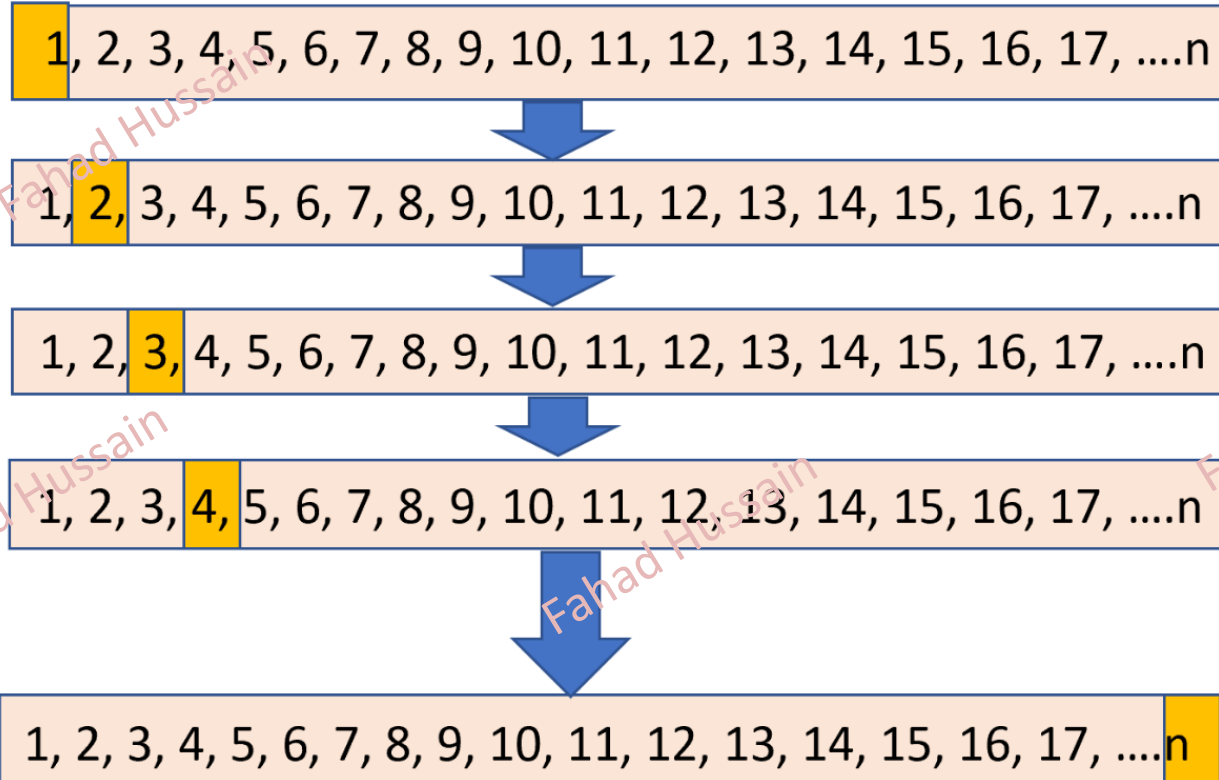
- **Holdout method.**

The **holdout** cross-validation **method** involves removing a certain portion of the training data and using it as test data. The model is first trained against the training set, then asked to predict output from the testing set

TRAINING SAMPLE

HOLD OUT SAMPLE / TESTING
SAMPLE

- **Leave-one-out cross-validation**



K-fold cross-validation

Iteration 1



Iteration 2



Iteration 3



Iteration 4

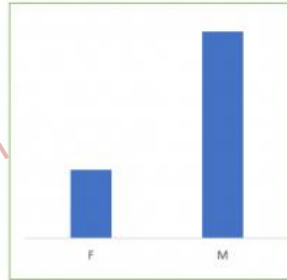


Iteration 5



• Stratified cross-validation

Stratified K-Fold
Cross Validation
(K=5)



Class Distributions



Fold 1

Fold 2

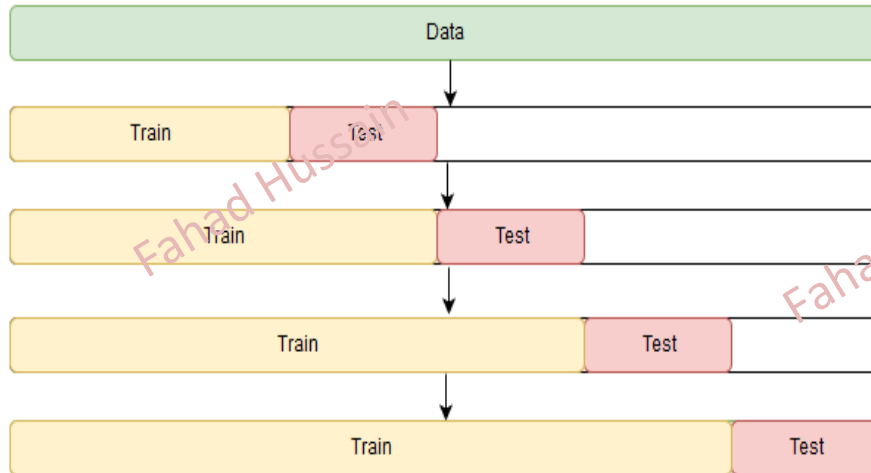
Fold 3

Fold 4

Fold 5

Time series cross-validation

D1	D2	D3	D4	D5	D6	D7	D8	D9
D1	D2	D3	D4	D5	D6	D7	D8	D9
D1	D2	D3	D4	D5	D6	D7	D8	D9
D1	D2	D3	D4	D5	D6	D7	D8	D9
D1	D2	D3	D4	D5	D6	D7	D8	D9





Queries as Comments

Thank You Very Much

For Notes and Code:

Visit: <http://fahadhussaincs.blogspot.com/>



K-fold cross-validation and Grid Search

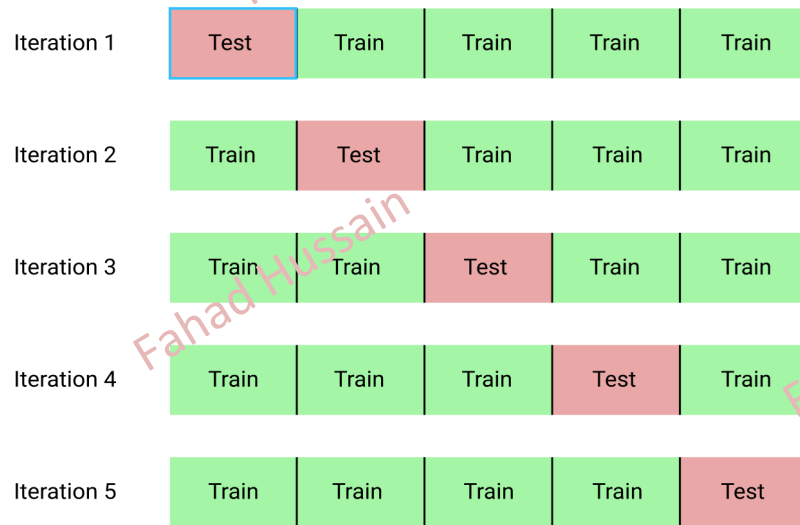
Model Parameters are something that a model learns on its own. For example, 1) Weights or Coefficients of independent variables in Linear regression model. 2) Weights or Coefficients of independent variables SVM. 3) Split points in Decision Tree.

Model hyper-parameters are used to optimize the model performance. For example, 1) Kernel and slack in SVM. 2) Value of K in KNN. 3) Depth of tree in Decision trees

Model Parameters	Hyperparameters
Model parameters are the features of training data that will learn on its own during training.	Model Hyperparameters are the parameters that determine the entire training process.
For example, • Weights and Biases • Split points in Decision Tree	For example, • Learning Rate • Hidden Layers • Hidden Units
They are internal to the model and their value can be estimated from data.	They are external to the model and their value cannot be estimated from data.

K-fold cross-validation and Grid Search

Grid search is the process of performing hyper parameter tuning in order to determine the optimal values for a given model. This is significant as the performance of the entire model is based on the hyper parameter values specified



Hyperparameter tuning vs. model training

Hyperparameter tuning



Best hyperparameters

Model training



Best model parameters



Queries as Comments

Thank You Very Much

For Notes and Code:

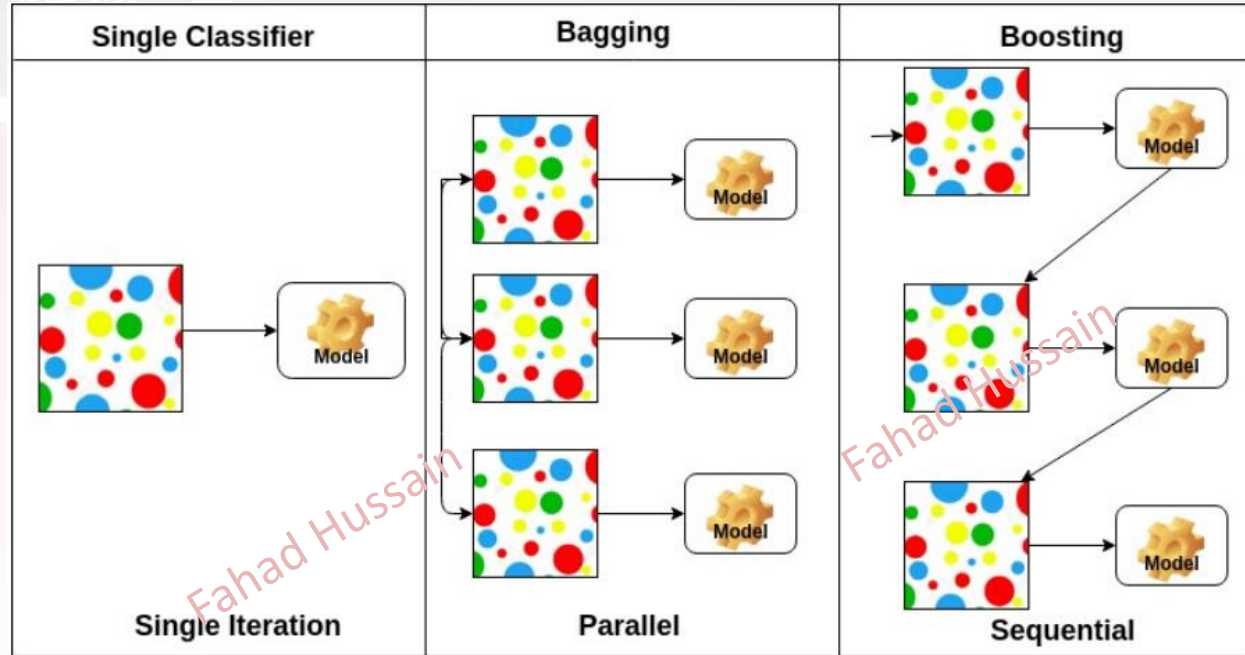
Visit: <http://fahadhussaincs.blogspot.com/>

Grid Search, behind theory (Working)

Ensemble Learning

1) Bagging (RFC, RFR
(Parallelly work)

2) Boosting
(Sequentially tress,
working)





Final words

Congratulation

***for completion of Machine Learning Course
with **Fahad Hussain*****



Final words

Learning Track

[https://www.youtube.com/channel/UCapJpINJKHz
fIWwCQ8Kse2g/playlists](https://www.youtube.com/channel/UCapJpINJKHzfIWwCQ8Kse2g/playlists)



Final words

***Few Very good website for
machine learning OR data science expert***

<https://scikit-learn.org/stable/>

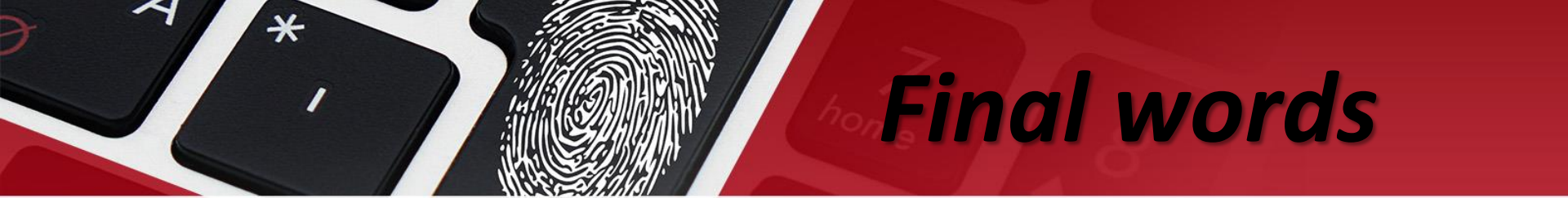
<https://towardsdatascience.com/>

<https://www.datacamp.com/>

<https://www.kaggle.com/>

<https://archive.ics.uci.edu/ml/index.php>

<https://www.whizlabs.com/blog/top-machine-learning-interview-questions/>



Final words

***So, What do you think
Its over Machine leaning***

NO!!!

***Actually it is the start of Machine learning Journey
Join any Company, Org. as internee or Job
To start you career in AI to boost your knowledge practically!!!***

Now, the next step is

Deep Learning

Stay with me, do subscribe and share with friends